Derived file. Generated by tools/build_combined.py from the weekly and support TXT files, which are the only source. Edit those files and regenerate; do not edit this copy.

START HERE: A COURSE BUILT AROUND EXPLANATIONS

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

WHAT THIS PACKAGE CONTAINS

Fourteen detailed instructor lecture plans, each organized into eleven consistent sections. Across the course there are 112 scene briefs: eight per week, including three sustained visual demonstrations and five shorter explanatory scenes. Each scene states what to show, what to ask, what mechanism to reveal, what condition to change, what principle to name, and what the student PDF must preserve.

The weekly TXT files and the numbered support files are the only source. The combined Markdown and PDF are derived reading copies regenerated by tools/build_combined.py and are never edited by hand. The JSON fixture file records the fictional inputs and independently checked results for every Anchor scene; plan_manifest.json is a derived scene manifest for the authoring agents, the index page and the tests.

This is the course plan and authoring package. It does not contain the finished weekly HTML decks, the resulting fourteen student slide PDFs, or a functioning lecture website. Those are the specified outputs of the later authoring workflow. The included PDF is the complete instructor plan, not a substitute for those student decks.

STARTING LEVEL AND TEACHING RHYTHM

This is an introductory software-engineering course for a worldwide general audience: independent learners, people entering software development, and practitioners revisiting the foundations. No university enrollment or particular institutional background is assumed. Familiarity with a short function, a conditional and a collection helps; read each small pseudocode example in plain English and introduce its notation when it first matters. No particular language, framework, cloud account, version-control host or advanced mathematics is required.

Pacing decision, clarified 22 September 2026: the overall target is about 80 minutes per lecture. This is a planning estimate, not a required duration, minimum or maximum; a lecture may be shorter or longer. Do not assign durations, minute budgets or planned timestamps to individual chapters, sections or scenes. Finish the reasoning, comparisons and useful questions at a natural pace; do not rush an explanation or pad it to meet the overall target. Week numbers identify the learning sequence, not an enrollment calendar. Actual navigation timestamps are written from the completed recording. Each weekly file supplies an ordered chapter map and a core/optional depth choice.

The week ordering is a learning sequence. Actual engineering connects requirements, design, construction, verification, operation, and learning repeatedly; it does not complete each once and permanently move on.

THE RECURRING EXPLANATORY CYCLE

Begin with a situation students can understand. Ask them to predict the outcome before revealing it. Step through the mechanism at the instructor's pace. Change one clearly labeled condition. Compare outcomes under the same relevant assumptions. End with a principle that explains the difference.

An Anchor scene sustains this cycle long enough for a real prediction and comparison. A Bridge scene is a shorter explanation or connection, often a static matrix or a few purposeful reveals. These are teaching roles, not required visual components or fixed durations.

Do not turn the detailed instructor prose into slide paragraphs. The screen should contain the situation, visual evidence, a concise question, and the principle when revealed. Students should be able to understand the printed version through selected states and short explanatory captions.

WHAT REMAINS CONSISTENT

Campus Rooms is the fictional recurring domain and an instructor-worked example, not the viewer's university or an assigned implementation. Its rules, names, state distinctions, and numerical fixtures are defined in 19_recurring_case_and_demo_data.txt. Temporary changes are explicitly labeled and restored. All source references use stable S-number keys defined in 18_sources.txt; each source entry records its actual retrieval date and access limitations.

One HTML file is planned for each week. All weeks share the same visual language and control meanings, but not a single diagram type. Slide navigation changes the page; scene controls change the demonstration on that page. Opening a stable scene link always starts at its documented baseline, paused.

HOW TO USE THE WEEKLY FILES

Read Sections 1–3 to establish purpose, outcomes, and preparation. From Week 2, the final answered question applies the same principle to a small example outside room booking. Explain its answer within the lecture; it adds no prerequisite or take-home activity. Section 4 is the ordered chapter/page map. Section 5 is the scene-by-scene teaching brief. Sections 6–7 identify important semantic guardrails and predictable misunderstandings. Section 8 supplies in-lecture questions with answers. Section 9 explains print continuity and pacing flexibility. Section 10 carries the case forward, and Section 11 identifies sources.

Before authoring the whole course, establish the common shell and fully realize Week 1. Also use one dependency scene from Week 5 and one numerical scene from Week 11 to test whether the shell accommodates graphs, controllable state, and print output. Keep these samples inside their eventual weekly files rather than inventing extra student entry pages. Then author one week at a time, reusing the verified shell while allowing the representations to vary. The build plan in 23_build_plan_and_schedule.txt turns this into phases with parallel authoring after Week 1.

CURRENT REVISION

Weeks 2-14 were reviewed on 22 September 2026 against the recorded Week 1 materials and selected current primary references. The order, scene identifiers and canonical case remain stable. Section 9 of each revised week selects core and optional depth; its closing checks cover the new reasoning. See ../notes/curriculum-review-2026-09-22.md for the review rationale and validation record. The detailed content remains in weekN.txt, the source register and the fixtures. Week 1 is retained as delivered.

COURSE SPINE

01 | Engineering software that people can depend on | What makes working code an engineered system? 02 | From ambiguous requests to verifiable requirements | How would we know that a requirement has been satisfied? 03 | Modeling behavior, data, and user interaction | Which model answers the question we are actually asking? 04 | Development flow, uncertainty, and engineering decisions | Where is work waiting, and what would actually improve the flow? 05 | Modularity, interfaces, and the cost of change | When one requirement changes, which parts should need to change? 06 | Architecture, interactions, and distributed failure | What changes when a dependency becomes slow, unavailable, or uncertain? 07 | Collaboration, version history, and reviewable change | How can several people change one system without losing intent or trust? 08 | Testing behavior and building credible evidence | Which checks would challenge our assumptions rather than merely repeat them? 09 | Security, privacy, and trustworthy dependencies | What must remain true when a request, dependency, or assumption cannot be trusted? 10 | Automated delivery and evidence-based release | What evidence should travel with a change before it reaches users? 11 | Reliability, observability, and performance reasoning | Which observation would help distinguish the possible causes of a failure? 12 | Maintenance, evolution, and sustainable change | How can a useful system change without losing the behavior people rely on? 13 | AI-assisted engineering and accountable automation | Does plausible-looking output satisfy the intended behavior, and who can verify that? 14 | Connecting the lifecycle through one engineering decision | Can we explain a change from human need to operational evidence without losing the original promise?

DECISIONS IN FORCE (UPDATED 22 SEPTEMBER 2026)

Recorded here once; other files refer to this list instead of restating it.

Public lecture-only course. Apply the public-audience rule in F:/0_tutorial_videos_project/docs/channel_style_guide.md. The course has no required coursework, homework, assigned projects, submissions, exams or grading, and assumes no separate university activity. Worked software examples and answered prediction questions belong inside the explanation. Keep academic administration out of the decks, PDFs and narration.

AI-assisted engineering is the through-line: framed in Week 1, carried by one beat inside an existing scene in Weeks 2, 7, 8, 9, 10 and 12, examined in Week 13 with synthetic candidate code only. The decks never say how the course itself is made; Week 13's provenance example is a fictional change record.

Everything is in English; there is no other language edition. The dark theme is the default and the print stylesheet is light. Every code snippet is Python-like pseudocode. Diagram grammar, type scale and palette are left to the authoring agents within the visual brief.

Tools and products may be named as dated examples, in each week's "In practice" paragraph (Section 1) and wherever a name helps a student connect a concept to everyday work. No step, diagram or snippet depends on a particular tool, and tools are named factually, without ranking.

Every week is recorded as a narrated video under the tutorial workspace rules (22_recording_track.txt): 3840 by 2160 at 60 fps, full canvas with the presenter overlay yielding to content, every reveal operated by a real pointer, and deck text and narration under the instructional stance defined in that file. There is no instructor rehearsal: a deck is accepted on its automated checks and a native read of its states rendered at the recording resolution.

No speaking cues (17 September 2026). The package and the repository hold no cue lists, shot lists, narration text or briefs, shot-duration budgets or recorded routes. The recording agent analyzes the week's plan, the fixtures and the deck at run time and generates the shot list, the narration and speech, and the tutorial itself (22_recording_track.txt).

The plan is machine-checkable. The TXT files are the single source; plan_manifest.json and the combined Markdown and PDF are derived and checked for freshness; every Anchor scene has structured fixture data; tools/check_plan.py and the tests run before any change is accepted.

The course repository uses Git; publication follows the user's instructions. Week 1 has been recorded. The build sequence in 23_build_plan_and_schedule.txt now starts with Week 2 and advances through verified deliverables without a fixed teaching calendar. The stack of any future Campus Rooms reference implementation is decided later.

PACKAGE MAP

week1.txt through week14.txt: individual lecture briefs; the only source for the weekly content. 15_curriculum_currency_and_coverage.txt: current references, coverage, and deliberate boundaries. 16_reference_plan_analysis.txt: what was retained and transformed from the Blender package. 17_visual_lecture_and_pdf_brief.txt: shared experience, control meanings, local operation, and print expectations. 18_sources.txt: primary source register with each entry's use (curriculum or authoring) and access limitations. 19_recurring_case_and_demo_data.txt: case rules, planted threads, and the meaning of every teaching model. 20_agent_authoring_brief.txt: how the build agents are briefed; current course authoring rules live in ../AGENTS.md. 21_instructor_preflight.txt: acceptance checks, and which of them the tests automate. 22_recording_track.txt: what the decks provide for recording, and what the recording agent generates at run time under the tutorial workspace rules. 23_build_plan_and_schedule.txt: repository, three phases, parallel authoring, minimum viable deck, schedule. assets/demo-fixtures.json: the teaching truth: fixtures and structured data for every Anchor scene. plan_manifest.json: derived scene manifest; regenerate with tools/build_manifest.py, never edit by hand. tools/: build_manifest.py, check_plan.py, build_combined.py and models.py (see tools/README.txt). tests/: the plan checker as a test, the deck contract tests and the PDF checks (pytest.ini, requirements-dev.txt). The former repository starter files are now at the course root: ../AGENTS.md, ../docs/scene-contract.md, ../scripts/, ../tasks/ and ../.github/workflows/. Software_Engineering_14_Week_Plan.md and .pdf: derived reading copies regenerated by tools/build_combined.py.

WEEK 01 | ENGINEERING SOFTWARE THAT PEOPLE CAN DEPEND ON

Research and curriculum date: 16 September 2026

What makes working code an engineered system?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-01.html

1. PURPOSE, CASE STATE, AND SCOPE

Open with a familiar problem rather than a history of programming languages. Campus Rooms appears to reserve a room successfully, yet two people receive confirmations for the same time. The lecture gradually widens the frame from a function that runs to a service whose behavior, users, changes, and consequences must be understood.

Entry knowledge is the ability to read a small function, an if-statement, and a collection. Do not require a particular programming language or familiarity with enterprise tooling. Establish vocabulary through the example: stakeholder, requirement, constraint, quality attribute, evidence, lifecycle, and trade-off. The weekly ordering is a learning sequence, not a claim that real engineering happens in fourteen isolated phases. [S01, S02, S24]

State the through-line in the opening and return to it: every habit this course builds, a precise rule, an independent check, a bounded permission, a reviewable change, is also what it takes to direct and check automated engineering assistance. Weeks 2, 7, 8, 9, 10 and 12 each carry one short beat on that claim inside an existing scene, and Week 13 examines it fully. Decisions in force for the whole package are listed in 00_START_HERE.txt.

In practice (dated examples, 2026): teams keep requirements and incident reports in issue trackers such as GitHub Issues or Jira, check accessibility with axe or Lighthouse next to a keyboard-only walk-through, and draw context maps in Mermaid or draw.io.

2. OBSERVABLE LEARNING OUTCOMES

1. Distinguish a locally correct function from a dependable user-facing service.

2. Identify stakeholders, system boundaries, and competing quality concerns.

3. Explain why delivery and maintenance belong in the engineering lifecycle.

4. Separate an observation, an assumption, and evidence supporting a claim.

5. Explain one technical decision in terms of benefit, cost, and affected people.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare a fictional booking screen, two conflicting confirmation cards, a simple room calendar, and a small map of people and systems. Use Room C101 and the 10:00-11:00 interval throughout. Keep an alternative correct outcome ready: one confirmation and one clear unavailability response. Establish that all numbers and incidents in this course are authored teaching examples unless explicitly attributed.

Opening page: show only the reassuring success message and ask whether it is enough. Recall page: invite students to explain what a program they already know does and what could make it fail outside the happy path. No installation or command-line demonstration is needed.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Two green confirmations cannot both be right | Anchor

week-01.html#/two-confirmations

02 | The booking button is not the whole system | Bridge

week-01.html#/system-boundary

03 | The same screen can serve one person and exclude another | Anchor

week-01.html#/stakeholder-lenses

04 | Faster is not a complete description of better | Bridge

week-01.html#/quality-tradeoffs

05 | A failure can teach the next requirement | Anchor

week-01.html#/feedback-lifecycle

06 | What would make us believe this works? | Bridge

week-01.html#/evidence-not-confidence

07 | Who carries the cost of an engineering shortcut? | Bridge

week-01.html#/responsibility-and-impact

08 | One domain, many kinds of explanation | Bridge

week-01.html#/course-evidence-map

5. DETAILED SCENE TEACHING NOTES

01. TWO GREEN CONFIRMATIONS CANNOT BOTH BE RIGHT

Anchor scene | ID: two-confirmations

VISUAL IDEA: Two browser cards beside one shared room calendar. Three controlled beats reveal request, stored state, and user consequence.

SITUATION, PREDICTION, AND MECHANISM: Situation and prediction: Ari and Bo each see “Confirmed” for C101 at 10:00. Ask, “What evidence would let us decide whether the system worked?” Reveal the calendar containing two overlapping confirmed bookings, then the people arriving at the same door. Show that an attractive interface and a successful response are not the whole behavior.

CHANGE ONE CONDITION AND COMPARE: Change only the confirmation rule: a shared decision now admits one request and rejects the conflicting one. Compare inconvenience from an honest rejection with harm from a false promise. Do not explain transactions yet; preserve that

question for later.

PRINCIPLE TO REVEAL: Engineering judges behavior against an intended outcome and its consequences, not appearance alone.

STUDENT PDF TREATMENT: Print the two-confirmation outcome and the one-confirmation outcome, with the violated rule written below both.

02. THE BOOKING BUTTON IS NOT THE WHOLE SYSTEM

Bridge scene | ID: system-boundary

VISUAL IDEA: A context map with the service inside a boundary and users, campus identity, room records, and notification delivery outside.

SITUATION, PREDICTION, AND MECHANISM: Ask which parts the team can change directly. Reveal one interaction at a time and label arrows with their meaning. A student request, an identity assertion, and a notification are not interchangeable kinds of information.

CHANGE ONE CONDITION AND COMPARE: Make the notification provider unavailable without changing room storage. Ask which user outcomes remain possible. Keep the boundary fixed so the class sees a dependency rather than a new diagram.

PRINCIPLE TO REVEAL: A system boundary identifies responsibility and dependencies; it does not make external behavior disappear.

STUDENT PDF TREATMENT: Use one labeled context map with the unavailable dependency marked by both text and a broken-line symbol.

03. THE SAME SCREEN CAN SERVE ONE PERSON AND EXCLUDE ANOTHER

Anchor scene | ID: stakeholder-lenses

VISUAL IDEA: A stakeholder map connected to concrete concerns; an accessible keyboard path sits beside the visual interface.

SITUATION, PREDICTION, AND MECHANISM: Ask who is affected besides the person clicking Reserve. Reveal a student needing step-free access, room staff handling closures, support staff diagnosing mistakes, and a maintainer interpreting old decisions. Translate each concern into a question the system must answer.

CHANGE ONE CONDITION AND COMPARE: Change the user interaction from mouse to keyboard. The visually polished version initially has no reachable confirmation control; the corrected version does. Compare task completion, not the attractiveness of the two screens. Introduce accessibility as part of whether the service works. [S25]

PRINCIPLE TO REVEAL: Quality depends on the people and conditions included in the definition of success.

STUDENT PDF TREATMENT: Print the stakeholder-concern table and the successful keyboard route; describe the failed route in text.

04. FASTER IS NOT A COMPLETE DESCRIPTION OF BETTER

Bridge scene | ID: quality-tradeoffs

VISUAL IDEA: A comparison table with response time, correctness, recoverability, privacy, and change effort; avoid an unexplained radar chart.

SITUATION, PREDICTION, AND MECHANISM: Offer two fictional designs: immediate confirmation from an unchecked local view, and confirmation after an authoritative availability decision. Ask which is better before revealing their behavior. Separate quality attributes from optional features. Name the standard vocabulary once: ISO/IEC 25010:2023 lists nine product quality characteristics, among them performance efficiency, reliability, security, interaction capability and maintainability. [S46]

CHANGE ONE CONDITION AND COMPARE: Keep correctness mandatory and change the response-time target. Discuss which optimizations preserve the rule and which merely hide a delay. Clearly label illustrative measurements and never imply that accuracy must always be traded for speed.

PRINCIPLE TO REVEAL: Compare alternatives under stated constraints; some requirements are conditions of acceptance rather than points in a score.

STUDENT PDF TREATMENT: Print the alternatives, assumptions, and a short rationale rather than a winner badge.

05. A FAILURE CAN TEACH THE NEXT REQUIREMENT

Anchor scene | ID: feedback-lifecycle

VISUAL IDEA: A loop connecting need, model, implementation, checks, release, operation, and revision; an incident card travels through it.

SITUATION, PREDICTION, AND MECHANISM: Predict where a missing “notification failed” behavior would be discovered. Reveal that it may emerge during a requirements conversation, an integration check, or operation. Follow the incident back to an updated user message and a new verification check.

CHANGE ONE CONDITION AND COMPARE: Move the first useful feedback earlier while keeping the defect the same. Compare the kinds of rework involved, without presenting an invented universal cost-of-defect multiplier. Explain iteration as revisiting understanding and increment as adding usable capability.

PRINCIPLE TO REVEAL: The lifecycle is a network of feedback loops, not a one-way conveyor ending when code runs.

STUDENT PDF TREATMENT: Print the full loop and two annotated feedback paths, including what information changed.

06. WHAT WOULD MAKE US BELIEVE THIS WORKS?

Bridge scene | ID: evidence-not-confidence

VISUAL IDEA: Three columns labeled Claim, Check, and Remaining uncertainty; cards move only when a reason is supplied.

SITUATION, PREDICTION, AND MECHANISM: Present “It looks right,” “It passed one example,” and “It met the stated checks under these conditions.” Ask which evidence supports which claim. Reveal a counterexample outside the tested conditions without implying that all prior testing was worthless.

CHANGE ONE CONDITION AND COMPARE: Change the workload from one user to simultaneous users. Compare the unchanged claim with the newly exposed assumption. Introduce the course habit of stating both evidence and its limits.

PRINCIPLE TO REVEAL: A check supports a bounded claim; confidence without scope is difficult to evaluate.

STUDENT PDF TREATMENT: Print the completed claim-check-uncertainty table, retaining the counterexample.

07. WHO CARRIES THE COST OF AN ENGINEERING SHORTCUT?

Bridge scene | ID: responsibility-and-impact

VISUAL IDEA: A consequence map linking a decision to users, support effort, data exposure, and future change.

SITUATION, PREDICTION, AND MECHANISM: Show a support log containing more personal information than needed to diagnose a failure. Ask who benefits and who takes the risk. Reveal an alternative containing a request identifier and failure category instead of private booking details.

CHANGE ONE CONDITION AND COMPARE: Change the support question so that one additional field is genuinely needed. Compare deliberate, justified collection with collecting everything by default. Keep this an engineering discussion, not jurisdiction-specific legal advice.

PRINCIPLE TO REVEAL: Professional responsibility includes the consequences imposed on people who did not make the technical choice.

STUDENT PDF TREATMENT: Print the minimal diagnostic record and the rationale for each retained field.

08. ONE DOMAIN, MANY KINDS OF EXPLANATION

Bridge scene | ID: course-evidence-map

VISUAL IDEA: A course map that changes representation: requirement card, state diagram, dependency graph, test table, release path, and incident timeline.

SITUATION, PREDICTION, AND MECHANISM: Ask whether one flowchart could explain all of these questions equally well. Reveal each representation with a single example of the question it answers. Introduce the recurring Campus Rooms case without exposing authoring tools.

CHANGE ONE CONDITION AND COMPARE: Keep the booking behavior fixed and switch from “Who depends on this?” to “What happened first?” Compare the dependency graph with the sequence diagram. Then change the actor: the same booking change is drafted by an automated assistant instead of a colleague. Ask which representations and checks are no longer needed. None; that is the through-line the course returns to in Week 13.

PRINCIPLE TO REVEAL: Choose a representation for the question, not a diagram style for the whole course.

STUDENT PDF TREATMENT: Print the course map as a question-to-representation reference, including the row for a change drafted by an automated assistant.

6. TEACHING GUARDRAILS

Define software engineering broadly through change, coordination, quality, and the system's lifetime rather than a contest between programmers and engineers. The course does not teach that every small script needs heavyweight process. Keep process proportional to consequence and uncertainty. Treat security, accessibility, and maintainability as recurring concerns from the first week. [S01, S02, S24]

In every visual, distinguish a hypothetical failure from an observed real incident. Do not use fabricated industry percentages to make the opening dramatic. The booking conflict is a counterexample that motivates inquiry, not an explanation of its implementation yet.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“The interface says success, so the operation succeeded.” Ask which authoritative state supports that message.

“More process always means better engineering.” Compare a disposable local calculation with a shared booking service and ask what risks differ.

“Testing happens only at the end.” Revisit the feedback loop and identify an early check on a requirement.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: What is wrong with two successful overlapping confirmations?

ANSWER: They violate the intended exclusivity rule even when each screen and request appears successful.

QUESTION 2: Why include support staff in the stakeholder map?

ANSWER: Their diagnostic needs influence observability, documentation, privacy, and recovery decisions.

QUESTION 3: Does passing one booking example establish correctness for simultaneous requests?

ANSWER: No. The evidence does not cover that interaction condition.

QUESTION 4: Which representation answers “What happens after a timeout?”

ANSWER: A state model or a sequence/timeline view, depending on whether the question concerns possible states or a particular interaction.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

The student PDF should stand alone: include the conflict and corrected outcome, the stakeholder-concern table, the lifecycle feedback paths, and a small terminology panel. Put the answered understanding checks after the questions. Keep the instructor's prompting notes out of the student edition. Use “Example assumption” labels wherever a rule or number is introduced.

SHORTER AND LONGER MEETING ROUTES

Preserve the conflicting confirmations, stakeholder comparison, and feedback loop as the three sustained demonstrations. For a shorter meeting, combine the responsibility discussion with the stakeholder map. For a longer meeting, compare the same engineering question in a tiny local utility without introducing another elaborate business domain.

10. HANDOFF

Carry forward the context map, the exclusivity concern, and a list of unresolved assumptions. Week 2 turns these into precise, testable requirements. Show one ambiguous sentence on the final page: “Make room booking fair and easy.”

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S24, S25, S46. Full titles, URLs, dated status, and access limitations appear in the source register. Research check: 16 September 2026. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 02 | FROM AMBIGUOUS REQUESTS TO VERIFIABLE REQUIREMENTS

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

How would we know that a requirement has been satisfied?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-02.html

1. PURPOSE, CASE STATE, AND SCOPE

Use the unresolved sentence from Week 1 to teach elicitation, scope, prioritization, acceptance criteria, and traceability. Students should experience ambiguity before hearing a definition of a good requirement. Requirements are negotiated descriptions of intended behavior and constraints, not a form whose completion guarantees understanding.

The core output within the lecture is a small connected set of requirement examples for Campus Rooms. Distinguish functional behavior, quality targets, business rules, and implementation decisions. Introduce stable identifiers that reappear later in designs and verification evidence. [S01, S02]

In practice (dated examples, 2026): requirements live as issues or user stories in GitHub Issues, Jira or Azure Boards, and acceptance examples are often written as Given/When/Then scenarios that tools such as Cucumber or pytest-bdd run as checks.

2. OBSERVABLE LEARNING OUTCOMES

1. Identify ambiguity, missing stakeholders, and hidden assumptions in a request.

2. Turn a request into observable behavior with boundary and failure cases.

3. Distinguish a requirement from a solution decision and a supporting assumption.

4. Connect a requirement to a design decision and an independent verification check.

5. Explain how a changed requirement affects linked evidence.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare short stakeholder statements with deliberate disagreement: “Anyone should reserve,” “Only campus members should reserve,” and “Staff need to close rooms.” Prepare time strips showing 29, 30, 120, and 121 minutes. Keep the canonical rules in the shared case guide visible to the instructor.

Use R-01 for non-overlap, R-02 for a 30-120 minute duration, R-03 for cancellation authorization, R-04 for notification independence, and R-05 for accessible completion. Requirements and examples are course-authored rules, not claims about a real university.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | One sentence, three incompatible interpretations | Anchor

week-02.html#/ambiguous-request

02 | An omitted user can hide an entire requirement | Bridge

week-02.html#/elicitation-and-scope

03 | Turn “short booking” into a rule we can examine | Anchor

week-02.html#/observable-behavior

04 | “Fast” needs a workload and a measurement boundary | Bridge

week-02.html#/quality-with-conditions

05 | A precise requirement can still describe the wrong thing | Bridge

week-02.html#/acceptance-and-validation

06 | A requirement should lead somewhere | Anchor

week-02.html#/traceability-matrix

07 | The loudest request is not automatically the next one | Bridge

week-02.html#/priority-and-risk

08 | A changed rule should not leave old evidence pretending to be current | Bridge

week-02.html#/requirements-under-change

5. DETAILED SCENE TEACHING NOTES

01. ONE SENTENCE, THREE INCOMPATIBLE INTERPRETATIONS

Anchor scene | ID: ambiguous-request

VISUAL IDEA: An editable-looking request card branches into three concrete screen outcomes; no actual language-model call is involved.

SITUATION, PREDICTION, AND MECHANISM: Open with the sentence Week 1 ended on, “Make room booking fair and easy.” Ask students to predict the behavior implied by “fair.” Reveal first-come access, per-user limits, and staff-priority interpretations. Show that all three could be defended from the same sentence. Add questions about who decides, what is scarce, and what outcome matters. Mark “easy” as a second, separate ambiguity (easy for whom: a keyboard user, a first-time visitor, room staff?) and record it as an open quality question for scene 04 and R-05, keeping this scene on fairness.

CHANGE ONE CONDITION AND COMPARE: Change only the stakeholder answering the question. Compare the resulting rules and surface the conflict rather than silently selecting one. Settle this lecture's scope on exclusive booking of available rooms; fairness policy remains an explicit open decision.

PRINCIPLE TO REVEAL: Ambiguity is resolved through examples and stakeholder decisions, not through more confident wording.

STUDENT PDF TREATMENT: Print the ambiguous sentence, the competing interpretations, the clarified scope, and the open question about “easy.”

02. AN OMITTED USER CAN HIDE AN ENTIRE REQUIREMENT

Bridge scene | ID: elicitation-and-scope

VISUAL IDEA: A context map gains questions at its boundaries; facts, assumptions, and open decisions have different labels and shapes.

SITUATION, PREDICTION, AND MECHANISM: Ask what must be learned before accepting a request from a guest. Reveal questions about identity, cancellation rights, accessibility, and room closures. [S25] Separate something a stakeholder stated from something a developer inferred. Make elicitation concrete with a fictional request for another confirmation email. Ask what happened on the person's last booking, then reveal an authored observation: after submitting, the person cannot tell whether the booking was stored. Compare the requested solution with the underlying need to know the outcome. A small interface prototype distinguishes Submitting, Confirmed and Unknown outcome; its walkthrough informs the requirement, while the authoritative stored result remains a separate engineering concern. Label this as a simulated observation, not user research that the course actually conducted.

CHANGE ONE CONDITION AND COMPARE: Introduce an external visitor as a new stakeholder but keep this release's campus-member boundary. Compare explicitly deferred behavior with an accidental omission.

PRINCIPLE TO REVEAL: An explicit boundary and an open-question register make uncertainty discussable.

STUDENT PDF TREATMENT: Print the scope boundary and a compact fact-assumption-question table. Retain the requested email, observed uncertainty and prototype comparison so the elicitation method is visible.

03. TURN “SHORT BOOKING” INTO A RULE WE CAN EXAMINE

Anchor scene | ID: observable-behavior

VISUAL IDEA: A duration ruler, a requirement card, and example outcomes update together.

SITUATION, PREDICTION, AND MECHANISM: Show 29, 30, 120, and 121 minute requests. Ask which should be accepted before revealing R-02: durations are at least 30 and at most 120 minutes, inclusive. Explain actor, condition, expected behavior, and rejection behavior. Keep the rule separate from a proposed slider widget.

CHANGE ONE CONDITION AND COMPARE: Change only the upper limit from 120 to 90 minutes in a clearly labeled temporary variant. Compare the affected examples and restore the canonical 120-minute rule afterward. Include an invalid end-before-start request to show that one boundary rule does not describe every validity condition. Then add the AI beat: two prepared, synthetic drafting requests go to an automated assistant, one carrying the sentence “keep bookings short” and one carrying R-02. Show the two candidate rules side by side: the first invents a limit, the second implements 30 through 120 inclusive. No live model call; the candidates are authored fixtures. A precise request makes the candidate easier to judge; it does not guarantee a correct candidate. Reuse the synthetic exclusive-boundary candidate from Week 13 as a counterexample even when R-02 is supplied, and let the independent 120-minute example settle the decision.

PRINCIPLE TO REVEAL: A useful requirement makes meaningful differences between acceptable and unacceptable outcomes visible.

STUDENT PDF TREATMENT: Print the canonical rule and an input-expected-outcome table; label the 90-minute variant as a comparison only. Print the two candidate rules with the request that produced each.

04. “FAST” NEEDS A WORKLOAD AND A MEASUREMENT BOUNDARY

Bridge scene | ID: quality-with-conditions

VISUAL IDEA: A response-time distribution appears beside workload and measurement-condition cards.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a 200 millisecond response proves the system is fast. Reveal Q-01: in the specified teaching fixture, the 95th percentile of search API response time should not exceed 500 milliseconds under 50 concurrent users. Explain that this is an authored target, not a universal service standard. Map the course's quality words to the ISO/IEC 25010:2023 characteristics, for example response time to performance efficiency and keyboard-accessible completion to interaction capability. [S46]

CHANGE ONE CONDITION AND COMPARE: Change the workload while holding the target and code constant. Compare what was measured with what was claimed. Distinguish service-boundary response time from the viewer's whole browser experience. Contrast Q-01 with R-05: follow a keyboard-only booking path and inspect whether the outcome is reachable and understandable. Observable acceptance can be a categorical outcome as well as a numerical threshold.

PRINCIPLE TO REVEAL: A quality requirement needs observable acceptance, a boundary, and stated conditions.

STUDENT PDF TREATMENT: Print the distribution, the explicit target, the fixture assumptions, and the measurement boundary. Keep a worked keyboard-completion example for R-05 beside the performance target, so quality is not reduced to speed.

05. A PRECISE REQUIREMENT CAN STILL DESCRIBE THE WRONG THING

Bridge scene | ID: acceptance-and-validation

VISUAL IDEA: A three-part strip: stakeholder need, precise statement, and observed behavior.

SITUATION, PREDICTION, AND MECHANISM: Show a system that perfectly enforces a 30-minute maximum because someone transcribed the desired 120-minute maximum incorrectly. Ask whether implementation checks against the written statement would catch the misunderstanding. Reveal the original stakeholder example. Separate the requested feature from the outcome the person needs: shortening every reservation may satisfy a written limit while making a useful session impossible. Compare the proposed behavior with a concrete stakeholder example before accepting the specification. Return to the unresolved fairness choice without selecting a new policy.

CHANGE ONE CONDITION AND COMPARE: Keep the implementation unchanged and correct the intended requirement. Compare verification against a specification with validation of the intended need, acknowledging that actual practices can overlap.

PRINCIPLE TO REVEAL: We must question both whether the system matches its description and whether the description matches the need.

STUDENT PDF TREATMENT: Print the two questions and the mismatch connecting them.

06. A REQUIREMENT SHOULD LEAD SOMEWHERE

Anchor scene | ID: traceability-matrix

VISUAL IDEA: A small matrix links requirement IDs to design decisions, examples, and checks. Selecting a cell highlights its evidence chain.

SITUATION, PREDICTION, AND MECHANISM: Begin with R-01 and a blank matrix. Ask what evidence should connect “no overlapping confirmations” to an implementation. Reveal an authoritative conflict decision, boundary examples, and a concurrent-request check. Add R-03 with an ownership decision and an authorization check. Explain why a matrix suits coverage better than a moving pipeline.

CHANGE ONE CONDITION AND COMPARE: Change R-03 so a delegated staff role may cancel under a recorded reason. Highlight only the decisions and checks that genuinely depend on that rule. Leave unrelated duration tests unchanged and mark the new policy as a variant requiring a stakeholder decision.

PRINCIPLE TO REVEAL: Traceability makes relationships inspectable; a link is useful only when its meaning and evidence are current.

STUDENT PDF TREATMENT: Print the readable matrix, a legend, and the change-impact highlight with text labels.

07. THE LOUDEST REQUEST IS NOT AUTOMATICALLY THE NEXT ONE

Bridge scene | ID: priority-and-risk

VISUAL IDEA: A decision table compares user benefit, dependency, uncertainty, consequence, and effort range.

SITUATION, PREDICTION, AND MECHANISM: Offer three requests: correct duplicate confirmations, add decorative room photos, and make cancellation accessible. Ask what information is needed to prioritize them. Reveal consequences and dependencies without hiding judgments inside a mysterious score.

CHANGE ONE CONDITION AND COMPARE: Change one condition: a room closure is scheduled tomorrow in the fictional scenario. Compare how urgency and risk influence order without rewriting the underlying value of the requests.

PRINCIPLE TO REVEAL: Priority is a reasoned choice under constraints, not a permanent property of a feature.

STUDENT PDF TREATMENT: Print the decision table and the rationale for the selected order.

08. A CHANGED RULE SHOULD NOT LEAVE OLD EVIDENCE PRETENDING TO BE CURRENT

Bridge scene | ID: requirements-under-change

VISUAL IDEA: A versioned requirement card is connected to a change note, an owner, and stale/current evidence badges.

SITUATION, PREDICTION, AND MECHANISM: Ask what must happen when a duration rule changes after tests have been written. Reveal the changed statement, its rationale, the linked examples requiring revision, and a clearly dated decision. Retain prior wording as history rather than making it disappear.

CHANGE ONE CONDITION AND COMPARE: Change the request from a rule update to a wording correction that preserves behavior. Compare semantic change with editorial change and avoid triggering the same response mechanically.

PRINCIPLE TO REVEAL: Manage the meaning of a change, its rationale, and its evidence together.

STUDENT PDF TREATMENT: Print the before-and-after requirement and the explicit affected/unaffected list.

6. TEACHING GUARDRAILS

Use concise examples rather than a giant requirements document. Do not equate a user-story sentence with a complete specification. Acceptance examples, quality constraints, and open questions may be needed beside it. A solution such as “use a relational database” is not the same thing as “never confirm overlapping reservations.”

Numerical quality targets are synthetic and bounded by a stated fixture. R-01 concerns confirmed bookings for the same room; adjacent half-open intervals do not overlap. Do not accidentally introduce a 30-minute scheduling grid: durations in this case are whole minutes within the permitted range, and start times are not restricted to half-hour boundaries.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Testable means measurable only with a number.” A concrete permission or state-transition outcome can also be checked.

“A traceability matrix proves completeness.” Ask what missing requirement would have no row in the first place.

“The stakeholder knows every requirement in advance.” Use examples and unexpected conditions to reveal what remains unknown.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Is “Use Framework X to make booking fast” a clear behavior requirement?

ANSWER: It mixes a solution choice with an underspecified quality objective; define the needed outcome and conditions separately.

QUESTION 2: Should a 120-minute booking satisfy R-02?

ANSWER: Yes. The upper limit is inclusive; other validity and availability checks still apply.

QUESTION 3: What is missing from “Search must finish in 500 milliseconds”?

ANSWER: At least the measurement boundary, workload, proportion or percentile, and conditions.

QUESTION 4: What does selecting R-03 in the matrix tell us?

ANSWER: Which decisions and checks are linked to cancellation authorization; those links need review when the rule changes.

QUESTION 5: Does supplying R-02 to an assistant guarantee that its candidate is correct?

ANSWER: No. The explicit rule makes an independent judgment possible; the inclusive boundary still needs to be checked.

QUESTION 6: A desktop editor request says 'save my work safely.' What must be clarified before choosing autosave?

ANSWER: Clarify what counts as saved, acceptable loss after interruption, recovery feedback and overwrite behavior. Choose an observable recovery example with the user; autosave is a possible design, not the requirement itself.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Include a one-page requirement register, boundary examples, Q-01's measurement assumptions, and the traceability matrix. Preserve explicit “unknown” and “deferred” labels rather than replacing them with guessed facts. Temporary variants must never be confused with the course's canonical rules. In print, highlight the changed matrix cells with symbols as well as color.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect ambiguity, observable boundaries and traceability. Treat the quality vocabulary as labels for the worked cases; compress the taxonomy and tool examples. Optional depth: compare stakeholder outcome evidence with feature completion.

10. HANDOFF

Carry the requirement register and traceability matrix into Week 3. End by asking whether a written rule alone makes the sequence of booking, cancellation, and failure states understandable.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S25, S46. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 03 | MODELING BEHAVIOR, DATA, AND USER INTERACTION

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Which model answers the question we are actually asking?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-03.html

1. PURPOSE, CASE STATE, AND SCOPE

Turn the requirements into complementary models rather than a tour of diagram notation. Students compare a user journey, a state model, an entity relationship view, a decision table, and a sequence diagram. The important learning is what each representation includes, omits, and can contradict.

Use lightweight, clearly labeled UML-style state and sequence notation where useful, without claiming every drawing is a formally complete UML model. The model of the booking service remains independent of any language, database product, or interface framework. [S01, S02, S08]

In practice (dated examples, 2026): teams keep diagrams as text next to the code in Mermaid or PlantUML, draw them in draw.io, and design interface states in Figma.

2. OBSERVABLE LEARNING OUTCOMES

1. Choose a model suited to a behavior, structure, or interaction question.

2. Read a state transition with an event, condition, and outcome.

3. Distinguish a stored booking from a transient interface or request state.

4. Explain entity identity and relationships without duplicating shared facts.

5. Locate a contradiction between a requirement, a diagram, and an interface message.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare the Week 2 requirement cards and three mock screens: choose room, submit request, and observe outcome. Prepare separate diagrams for request/UI state and stored booking state. The distinction prevents “Rejected request” from silently becoming a confirmed booking record.

Use synthetic IDs U-01, U-02, C101, C202, and B-001. Default stored booking states are Confirmed and Cancelled. Draft, Submitting, and Unknown outcome belong to the request/UI model. A rejected request does not create a confirmed booking.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Follow the person before drawing the internals | Bridge

week-03.html#/journey-before-boxes

02 | A booking cannot be in every state at once | Anchor

week-03.html#/state-machine

03 | Two copies of a room name can disagree | Anchor

week-03.html#/identity-and-data

04 | Some rules are clearer as rows than as arrows | Bridge

week-03.html#/decision-table

05 | The same final state can hide different interaction histories | Anchor

week-03.html#/sequence-not-state

06 | Loading, empty, error, and success are different promises | Bridge

week-03.html#/interface-states

07 | A beautiful diagram can contradict the requirement | Bridge

week-03.html#/model-consistency

08 | A model is allowed to leave things out | Bridge

week-03.html#/abstraction-and-zoom

5. DETAILED SCENE TEACHING NOTES

01. FOLLOW THE PERSON BEFORE DRAWING THE INTERNALS

Bridge scene | ID: journey-before-boxes

VISUAL IDEA: A user journey strip with goal, action, visible feedback, and uncertainty at each step.

SITUATION, PREDICTION, AND MECHANISM: Ask what a student needs to know after pressing Reserve. Reveal the difference between “request sent,” “booking confirmed,” and “could not determine the result.” Follow one ordinary request before adding technical actors.

CHANGE ONE CONDITION AND COMPARE: Remove the confirmation message but retain a successful stored booking. Compare system state with user understanding and discuss the resulting duplicate-click risk.

PRINCIPLE TO REVEAL: A user journey exposes information needs that an internal component diagram can miss.

STUDENT PDF TREATMENT: Print the journey with the visible message and underlying state on separate rows.

02. A BOOKING CANNOT BE IN EVERY STATE AT ONCE

Anchor scene | ID: state-machine

VISUAL IDEA: Two deliberately separate state models: request/UI states above, stored booking states below. A current-state marker moves only on valid events.

SITUATION, PREDICTION, AND MECHANISM: Start in Draft and ask what Submit means. Reveal Submitting, then Confirmed, Rejected, or Unknown outcome when a response is lost. Link confirmed results to a stored booking record. In the stored model, a permitted cancellation changes

Confirmed to Cancelled. Label events and conditions before discussing terminology. Name an invariant as a statement that must remain true across the permitted transitions. Contrast that safety question with a progress question: will pending work eventually be resolved? A diagram of allowed transitions alone establishes neither implementation correctness nor eventual completion.

CHANGE ONE CONDITION AND COMPARE: Change the current state to Cancelled and replay Cancel. Compare a stable, clearly defined repeated response with an accidental second state change. Ask which transitions are intentionally absent. Keep UI timeout separate from deletion of the stored booking.

PRINCIPLE TO REVEAL: State models clarify which changes are possible, what triggers them, and which facts are actually known.

STUDENT PDF TREATMENT: Print both models with a legend and a short example explaining why a timeout is not proof of failure.

03. TWO COPIES OF A ROOM NAME CAN DISAGREE

Anchor scene | ID: identity-and-data

VISUAL IDEA: Entity cards for User, Room, and Booking; relationship lines highlight identity and cardinality.

SITUATION, PREDICTION, AND MECHANISM: Show two booking records containing copied room capacity values. Ask what happens when the room capacity changes. Reveal a Room entity with a stable identifier and bookings referencing it. Explain that a booking belongs to one user and one room, while each user or room can be associated with many bookings over time.

CHANGE ONE CONDITION AND COMPARE: Change a room's display name while keeping its identifier. Compare stable references with matching on a mutable name. Also show a deliberately preserved historical snapshot and explain that duplication may be intentional when the meaning is historical rather than current. Distinguish a well-formed identifier from an identifier that references an existing room. Relate that distinction to referential integrity and explain why a data constraint, a domain rule and a permission decision answer different questions.

PRINCIPLE TO REVEAL: Model identity and the meaning of a fact before deciding where to store it.

STUDENT PDF TREATMENT: Print the relationship diagram and current-fact versus historical-snapshot comparison.

04. SOME RULES ARE CLEARER AS ROWS THAN AS ARROWS

Bridge scene | ID: decision-table

VISUAL IDEA: A decision table combines membership, ownership, booking state, and start time for cancellation.

SITUATION, PREDICTION, AND MECHANISM: Ask whether an authenticated campus member may cancel every booking. Reveal R-03: a requester may cancel their own future confirmed booking. Fill representative allowed and denied rows, then expose a missing past-booking row.

CHANGE ONE CONDITION AND COMPARE: Change only ownership while holding authentication true. Compare the decision and point to the exact rule responsible. Avoid inventing a universal staff override.

PRINCIPLE TO REVEAL: A decision table makes combinations and missing cases easier to inspect.

STUDENT PDF TREATMENT: Print all core rows, the rule, and a note identifying the policy boundary.

05. THE SAME FINAL STATE CAN HIDE DIFFERENT INTERACTION HISTORIES

Anchor scene | ID: sequence-not-state

VISUAL IDEA: A sequence diagram with browser, booking service, authoritative store, and notification worker.

SITUATION, PREDICTION, AND MECHANISM: Ask what must happen before a true confirmation can be shown. Reveal request validation, an authoritative booking decision, persistence, and the response. Add notification as separate work after confirmation. Use numbered events and distinguish messages from elapsed time.

CHANGE ONE CONDITION AND COMPARE: Lose the response after the booking has been stored. The user sees Unknown outcome while the stored record remains Confirmed. Compare this with a rejection before persistence. Preview status reconciliation without teaching the full retry mechanism yet. [S09]

PRINCIPLE TO REVEAL: An interaction history explains why two apparently similar user symptoms can require different recovery actions.

STUDENT PDF TREATMENT: Print the successful sequence and the lost-response sequence as two annotated panels.

06. LOADING, EMPTY, ERROR, AND SUCCESS ARE DIFFERENT PROMISES

Bridge scene | ID: interface-states

VISUAL IDEA: A small interface state gallery with keyboard focus and text status indicators.

SITUATION, PREDICTION, AND MECHANISM: Ask what “No rooms” means while a search is still loading. Reveal distinct Loading, Empty result, Validation error, Service unavailable, and Results states. Attach the promised meaning to each, rather than cycling through cosmetic designs.

CHANGE ONE CONDITION AND COMPARE: Change one condition from an empty valid result to a failed request. Compare wording, available recovery actions, and focus movement. Make the distinction perceptible without color. [S25]

PRINCIPLE TO REVEAL: An interface should communicate what is known and what the user can do next.

STUDENT PDF TREATMENT: Print the state gallery with meaningful text alternatives and a recovery-action column.

07. A BEAUTIFUL DIAGRAM CAN CONTRADICT THE REQUIREMENT

Bridge scene | ID: model-consistency

VISUAL IDEA: Three synchronized panels show R-01, a sequence diagram, and the calendar after the interaction.

SITUATION, PREDICTION, AND MECHANISM: Present a sequence that sends Confirmed before deciding availability. Ask students to find the contradiction. Reveal the resulting overlapping calendar and the precise message order that permitted the false promise.

CHANGE ONE CONDITION AND COMPARE: Move the confirmation message after the authoritative decision while leaving the other panels intact. Compare consistency across the representations rather than awarding correctness for diagram neatness.

PRINCIPLE TO REVEAL: Models are useful when their claims can be checked against one another and against intended behavior.

STUDENT PDF TREATMENT: Print the flawed sequence beside the corrected one, with the contradiction named.

08. A MODEL IS ALLOWED TO LEAVE THINGS OUT

Bridge scene | ID: abstraction-and-zoom

VISUAL IDEA: A representation selector switches among context, sequence, state, and data views of one booking.

SITUATION, PREDICTION, AND MECHANISM: Ask which view answers “Who owns B-001?” and which answers “What happened before the timeout?” Reveal the appropriate view without suggesting that it contains every implementation detail.

CHANGE ONE CONDITION AND COMPARE: Change the audience from a student using the service to a maintainer investigating the failure. Compare the information each needs and the irrelevant detail each can omit.

PRINCIPLE TO REVEAL: A useful abstraction preserves what matters for its question and makes its omissions intentional.

STUDENT PDF TREATMENT: Print a question-to-model matrix and the explicit limitations of each view.

6. TEACHING GUARDRAILS

A diagram is not evidence that its implementation exists or is correct. Label the level and purpose of each model. Keep arrow meanings consistent within a diagram; a dependency arrow is not automatically a message or a chronological event. Explain that class/entity representations need not become one class or one table per box.

Do not collapse unknown outcome into failed operation. Preserve the distinction between a request's status and the durable booking state. Model examples are intentionally small; Week 6 returns to concurrent decisions and failure across boundaries, and Week 8 derives tests from these models.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Every noun should become a class.” Ask which identity, responsibility, or relationship the proposed class actually represents.

“A sequence diagram lists all possible states.” Contrast one interaction path with the state model of permitted transitions.

“An empty page means no results.” Ask whether loading and service failure were distinguished.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: A response times out after persistence. Is the booking necessarily absent?

ANSWER: No. The requester may not know the result even though the booking exists.

QUESTION 2: Why reference a room ID instead of copying its current name everywhere?

ANSWER: Identity remains stable when the displayed name changes; historical snapshots should be modeled deliberately.

QUESTION 3: Which model best exposes combinations of ownership and status?

ANSWER: A decision table.

QUESTION 4: How can a diagram be tested before code exists?

ANSWER: Walk concrete examples through it and compare outcomes with requirements and complementary models.

QUESTION 5: Can a valid state diagram establish that pending notification work will eventually complete?

ANSWER: No. Permitted states and transitions do not establish the scheduling and recovery assumptions needed for progress.

QUESTION 6: A mobile note says 'saved on this device' while synchronization is pending. Does that contradict the cloud copy being older?

ANSWER: No. Local persistence and remote synchronization are separate states. Model both and make the message identify which promise has been met; pending synchronization still needs a defined completion or recovery path.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

The PDF should contain the separate request and booking state models, a readable entity relationship view, the cancellation table, and both sequence outcomes. Include a small notation legend. Retain the short explanatory sentence beneath each visual; arrows alone will not reproduce the explanation delivered aloud.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect state separation, identity and the lost-response sequence. Optional depth: enumerate a bounded set of transition paths and search for an invariant violation. Exhaustion of that small model is not proof of an unbounded implementation; omit formal-method notation and time-zone extensions from the core.

10. HANDOFF

Carry the requirement IDs, models, and unresolved decisions into Week 4. The next question is how a team organizes and learns from the work needed to turn these descriptions into dependable behavior.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S08, S09, S25. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 04 | DEVELOPMENT FLOW, UNCERTAINTY, AND ENGINEERING DECISIONS

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Where is work waiting, and what would actually improve the flow?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-04.html

1. PURPOSE, CASE STATE, AND SCOPE

Use a team workflow as an observable system. Explain iterative and incremental development, feedback, work-in-progress, bottlenecks, estimation, risk, and prioritization through controlled comparisons. Introduce Scrum and Kanban as identifiable approaches with different emphases, not synonyms for a board or for “being agile.” [S03, S04, S05]

The case is a small team improving Campus Rooms. Students do not need to learn a commercial planning interface. Use a transparent tabletop-like simulation with labeled work cards and explicit capacities. Avoid converting every judgment into a score or implying that all engineering work has predictable service times. [S01, S02]

In practice (dated examples, 2026): boards and queues live in tools such as Jira, GitHub Projects, Linear or Trello, whose reports chart the same work-in-progress, cycle-time and throughput measures this lecture builds by hand.

2. OBSERVABLE LEARNING OUTCOMES

1. Distinguish iteration, increments, and a single large batch of delivery.

2. Locate a bottleneck using queues and completion evidence.

3. Distinguish work-in-progress, throughput, cycle time, and lead time.

4. Explain an estimate as conditional information rather than a guarantee.

5. Choose a next action using risk, dependency, feedback value, and capacity.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare eight fictional work cards: conflict rule, duration boundary, cancellation ownership, room closure, accessible feedback, notification recovery, search response time, and support diagnosis. Use a discrete-round board with Ready, Implementing, Review, and Done. Default processing capacities are two cards per round in implementation and one in review. Cards completed in a round enter the next station for the following round, not instantly.

Prepare identical arrival schedules for every comparison. A Run control may advance the model, but Step advances one labeled round; the default is paused. All timings are teaching units, not measured developer performance.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Eight finished parts can still be one late discovery | Anchor

week-04.html#/feedback-batches

02 | A meeting does not create a feedback loop by itself | Bridge

week-04.html#/methods-not-rituals

03 | Making one stage faster can make the next queue longer | Anchor

week-04.html#/queue-and-bottleneck

04 | A shorter internal cycle can hide a longer external wait | Bridge

week-04.html#/wip-and-time

05 | A precise number can be less honest than a range | Bridge

week-04.html#/estimates-with-assumptions

06 | The most useful next step may be the one that reduces uncertainty | Anchor

week-04.html#/risk-and-learning

07 | “Coded” and “ready to rely on” are different claims | Bridge

week-04.html#/definition-of-done

08 | A process change is a hypothesis, too | Bridge

week-04.html#/retrospective-as-test

5. DETAILED SCENE TEACHING NOTES

01. EIGHT FINISHED PARTS CAN STILL BE ONE LATE DISCOVERY

Anchor scene | ID: feedback-batches

VISUAL IDEA: Two timelines compare one large batch with small usable slices; a requirement misunderstanding is revealed at a chosen feedback point.

SITUATION, PREDICTION, AND MECHANISM: Ask when the team would learn that students need clear cancellation feedback. Reveal one timeline where everything is integrated at the end and another where a small booking-and-cancellation slice is inspected earlier. Explain the difference between adding usable capability and revising understanding.

CHANGE ONE CONDITION AND COMPARE: Keep the misunderstanding constant but move the feedback point. Compare what must be reconsidered at that point. Then introduce a high-coordination integration constraint so students see that small slices still require thoughtful interfaces and sequencing.

PRINCIPLE TO REVEAL: Shorter feedback loops can expose wrong assumptions earlier; useful increments must still make sense as a system.

STUDENT PDF TREATMENT: Print the two timelines with discovery points, affected work, and stated assumptions.

02. A MEETING DOES NOT CREATE A FEEDBACK LOOP BY ITSELF

Bridge scene | ID: methods-not-rituals

VISUAL IDEA: A three-column comparison of plan-driven coordination, iterative timeboxes, and flow-based policies.

SITUATION, PREDICTION, AND MECHANISM: Ask what information each approach makes visible and when decisions change. Introduce Scrum through goals, inspection, and adaptation within its defined framework, and Kanban through explicit workflow, controlling work-in-progress, and improving flow. Do not claim a generic board is the whole of either. Name Scrum's three accountabilities in one line, Product Owner, Scrum Master and Developers, as the places where decisions about value, process and the work itself are made visible, not as job titles. [S04, S05] Give plan-driven coordination its strongest use case: costly external integration or a required evidence handoff. Name an early prototype or risk investigation as another way to learn before commitment. Compare when feedback arrives and which decisions remain reversible, rather than teaching method names as competing slogans.

CHANGE ONE CONDITION AND COMPARE: Change the work from a planned capability to unpredictable incident arrivals. Compare which coordination mechanisms help and what policies need revision; do not declare one universal winner.

PRINCIPLE TO REVEAL: Choose coordination practices for the uncertainty and constraints of the work, not for their labels.

STUDENT PDF TREATMENT: Print the comparison with a note that simplified lecture examples are not complete method definitions.

03. MAKING ONE STAGE FASTER CAN MAKE THE NEXT QUEUE LONGER

Anchor scene | ID: queue-and-bottleneck

VISUAL IDEA: A work board and a numerical queue chart share the same discrete-round data.

SITUATION, PREDICTION, AND MECHANISM: Run implementation at two cards per round and review at one. Ask where the next unfinished card will wait. Reveal the review queue accumulating, with completed work counted separately from work started. Let the viewer predict the result before doubling implementation capacity.

CHANGE ONE CONDITION AND COMPARE: Change only implementation capacity to four and replay the same arrival schedule. Compare completion rate and queue length. Restore the baseline, then change review capacity to two. Explain that the bottleneck may move once a constraint is relieved.

PRINCIPLE TO REVEAL: Local activity is not the same as system throughput; improve the limiting flow, not merely the busiest-looking stage.

STUDENT PDF TREATMENT: Print selected rounds and the corresponding queue/completion table, including capacities and transfer rules.

04. A SHORTER INTERNAL CYCLE CAN HIDE A LONGER EXTERNAL WAIT

Bridge scene | ID: wip-and-time

VISUAL IDEA: Each card has arrival, start, and finish markers; a work-in-progress limit controls admission.

SITUATION, PREDICTION, AND MECHANISM: Ask which clock a stakeholder experiences. Reveal lead time from request arrival to completion and cycle time from the declared start boundary to completion. Mark the difference visually and state the boundaries before using the terms. [S05]

CHANGE ONE CONDITION AND COMPARE: Reduce allowed in-progress work while keeping demand constant. Compare internal work age and the queue waiting before admission. Avoid claiming that moving waiting outside the measurement boundary is a real improvement.

PRINCIPLE TO REVEAL: Timing metrics are meaningful only when their boundaries and excluded waiting are explicit.

STUDENT PDF TREATMENT: Print the time markers and both waits; retain the ready queue instead of cropping it away.

05. A PRECISE NUMBER CAN BE LESS HONEST THAN A RANGE

Bridge scene | ID: estimates-with-assumptions

VISUAL IDEA: An estimate range sits beside known work, dependencies, and unresolved questions.

SITUATION, PREDICTION, AND MECHANISM: Ask whether “three days” means effort, elapsed time, or a commitment. Reveal the dependencies and review capacity behind the estimate. Compare a range based on several similar examples with a single unsupported number.

CHANGE ONE CONDITION AND COMPARE: Change one assumption: the identity interface is not yet agreed. Widen or qualify the estimate and show the value of resolving that uncertainty. Do not turn fictional ranges into statistical confidence intervals without a model and data. Also separate effort spent doing the change from elapsed waiting and continuing support cost. These form a small lifecycle cost discussion, not just an implementation estimate.

PRINCIPLE TO REVEAL: An estimate is a conditional statement whose assumptions should travel with it.

STUDENT PDF TREATMENT: Print the estimate with its assumptions and the condition that would trigger revision.

06. THE MOST USEFUL NEXT STEP MAY BE THE ONE THAT REDUCES UNCERTAINTY

Anchor scene | ID: risk-and-learning

VISUAL IDEA: A decision table contrasts impact, uncertainty, reversibility, cost of delay, and information gained.

SITUATION, PREDICTION, AND MECHANISM: Offer a visual refinement, a proof that simultaneous bookings cannot overlap, and an investigation into an unclear identity dependency. Ask what the team should learn before committing more work. Reveal what each step would clarify and what failure would cost. Include the option to defer or reduce scope. Explain opportunity cost through the valuable work displaced by the chosen step; no invented monetary multiplier is needed.

CHANGE ONE CONDITION AND COMPARE: Change only the reversibility of a decision: a temporary interface choice becomes a difficult data migration. Compare the amount of evidence worth seeking before proceeding. Use ordinal judgments and explanatory notes rather than multiplying invented probability scores.

PRINCIPLE TO REVEAL: Prioritize not only output, but also the decisions and uncertainties that could make later work wasteful.

STUDENT PDF TREATMENT: Print the decision table and a short rationale, including what remains uncertain.

07. "CODED" AND "READY TO RELY ON" ARE DIFFERENT CLAIMS

Bridge scene | ID: definition-of-done

VISUAL IDEA: A work card accumulates evidence badges for intended behavior, review, checks, documentation, and operability.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a cancellation change should be considered complete when the happy-path function runs. Reveal missing ownership checks and missing user feedback. Introduce a shared completion policy appropriate to this team's risk.

CHANGE ONE CONDITION AND COMPARE: Change the work to an internal wording correction with no behavioral effect. Compare the relevant evidence instead of demanding identical ceremony for every change. Distinguish acceptance of a particular requirement from a broader completion policy. [S04]

PRINCIPLE TO REVEAL: A shared completion policy should make quality expectations visible and proportionate.

STUDENT PDF TREATMENT: Print the two changes with their relevant evidence, not an unexplained universal checklist.

08. A PROCESS CHANGE IS A HYPOTHESIS, TOO

Bridge scene | ID: retrospective-as-test

VISUAL IDEA: Before-and-after panels connect a workflow observation, proposed change, measure, and possible side effect.

SITUATION, PREDICTION, AND MECHANISM: Ask what should happen after noticing a long review queue. Reveal a proposal to share review work, the expected effect, and what would count as contrary evidence. Keep the discussion about the system rather than blaming an individual, and explain why: people report delays and mistakes honestly only where reporting them is safe, so blameless discussion and psychological safety are the conditions for trustworthy process data.

CHANGE ONE CONDITION AND COMPARE: Increase review speed but show more missed issues. Compare throughput with quality and rework. Leave the viewer with the idea of a balanced improvement experiment rather than one target number.

PRINCIPLE TO REVEAL: Change a process deliberately, observe its consequences, and revise the explanation when the evidence disagrees.

STUDENT PDF TREATMENT: Print the process hypothesis and the before/after evidence with the side effect included.

6. TEACHING GUARDRAILS

The board is a deliberately simplified queue model, not a forecast of human productivity. Work cards are similar-sized only for the core demonstration; unequal sizes are a useful later complication. Keep chart axes and simulation rounds identical across A/B runs.

Do not teach story points as hours or compare teams through velocity. A work-in-progress limit does not remove incoming demand or guarantee shorter stakeholder lead time. Avoid a caricature in which plan-driven work never receives feedback or agile work needs no planning. Use the current Kanban Guide edition identified in the source register; the cited Scrum Guide is its November 2020 text. [S04, S05]

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Busy people mean fast delivery.” Point to completed outcomes and the queue.

“An estimate is a promise.” Ask which dependencies and uncertainties the number assumes.

“A lower cycle time proves a better user experience.” Inspect the arrival-to-completion boundary and excluded waits.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Review processes one item per round. Will doubling implementation necessarily double completion?

ANSWER: No. Review remains the limiting stage in the stated model.

QUESTION 2: What is the difference between iteration and increment?

ANSWER: Iteration revisits or refines understanding or a solution; an increment adds usable capability.

QUESTION 3: Why might a risky assumption deserve attention before an easy visible feature?

ANSWER: Resolving it can prevent later work from relying on a false premise.

QUESTION 4: What should accompany a process improvement claim?

ANSWER: A baseline, a stated change, a relevant outcome measure, and attention to quality or other side effects.

QUESTION 5: Why can the smallest implementation estimate still be the more expensive choice?

ANSWER: Waiting, integration, continuing support, exit cost and displaced valuable work can change the lifecycle comparison.

QUESTION 6: A library team writes changes faster, but maintainers can review no more than before. What should it inspect next?

ANSWER: Inspect waiting work, review capacity, accepted changes and rework. Faster drafting can grow the queue without improving completion; choose the next improvement from the constraint and its quality consequences.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve the actual values behind every chart: rounds, capacities, queue lengths, and completed items. Include a glossary with measurement boundaries. Print the estimate assumptions and risk rationale. Do not distribute a chart that implies a measured industry result when it came from the lecture model.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect feedback batches, queue behavior and the risk decision. Fit lifecycle cost into the existing estimate and risk tables; shorten the framework comparison. Optional depth: change the support or exit assumption in the same decision, without a new scoring system.

10. HANDOFF

Carry forward one prioritized change: changing a booking policy without causing widespread edits. Week 5 examines how design structure affects the cost and safety of that change.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S03, S04, S05. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 05 | MODULARITY, INTERFACES, AND THE COST OF CHANGE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

When one requirement changes, which parts should need to change?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-05.html

1. PURPOSE, CASE STATE, AND SCOPE

Teach design through change propagation. Compare two deliberately small implementations with the same visible behavior, then alter one requirement and follow the dependencies. Introduce cohesion, coupling, encapsulation, interface contracts, dependency direction, and refactoring only after their effects have been seen.

Keep code excerpts short and readable as pseudocode. This is not an exhaustive catalog of design patterns or a memorization session about acronyms. Students should be able to explain why a boundary is helpful, when it is costly, and what information must cross it. [S01, S02, S07, S24]

In practice (dated examples, 2026): IDEs such as VS Code and IntelliJ IDEA automate small refactorings like extracting a function, and many teams keep decision records as short Markdown files next to the code.

2. OBSERVABLE LEARNING OUTCOMES

1. Trace the effect of a requirement change through dependent components.

2. Distinguish cohesive responsibility from unrelated work gathered in one module.

3. Explain how an interface can hide changeable implementation details.

4. Recognize why fewer lines or more abstractions do not automatically mean better design.

5. Explain refactoring as structural change with preserved intended behavior.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare two fixed dependency graphs. Design A repeats duration policy in the browser, booking handler, and reminder logic. Design B has one authoritative policy component plus interface-facing validation that is explicitly non-authoritative. Use consistent component names across weeks: Booking, Policy, Room Catalog, Notification, and Storage.

Prepare a 120-to-90 minute temporary policy change and a second independent change replacing the notification provider. Keep baseline requirements available so students can distinguish a demonstration variant from the canonical case.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Two designs can look identical until something changes | Bridge

week-05.html#/same-behavior-different-design

02 | Watch a single policy change spread | Anchor

week-05.html#/dependency-example

03 | A module name is not a reason for its contents to belong together | Bridge

week-05.html#/cohesion-and-responsibility

04 | A replacement fits only when its behavior fits | Anchor

week-05.html#/interface-contract

05 | Who should know about the replaceable detail? | Bridge

week-05.html#/dependency-direction

06 | Change the structure without quietly changing the promise | Anchor

week-05.html#/refactoring-under-checks

07 | A flexible design can still be unnecessarily complicated | Bridge

week-05.html#/abstraction-cost

08 | A future maintainer needs the reason, not only the diagram | Bridge

week-05.html#/design-rationale

5. DETAILED SCENE TEACHING NOTES

01. TWO DESIGNS CAN LOOK IDENTICAL UNTIL SOMETHING CHANGES

Bridge scene | ID: same-behavior-different-design

VISUAL IDEA: Two matching booking screens above two different internal dependency maps.

SITUATION, PREDICTION, AND MECHANISM: Ask whether the screens reveal which design will be easier to change. Reveal the internal responsibilities only after the viewer commits a prediction. Trace one successful request through each version.

CHANGE ONE CONDITION AND COMPARE: Change the maximum booking duration while holding the screen and user goal constant. Stop before highlighting affected components so students identify likely edits first.

PRINCIPLE TO REVEAL: Design quality becomes visible through the changes and failures a structure must accommodate.

STUDENT PDF TREATMENT: Print both internal maps with the same user-visible behavior stated above them.

02. WATCH A SINGLE POLICY CHANGE SPREAD

Anchor scene | ID: dependency-example

VISUAL IDEA: A stable dependency graph progressively marks changed files, required rechecks, and unaffected components using distinct symbols.

SITUATION, PREDICTION, AND MECHANISM: In Design A, reduce the maximum duration from 120 to 90 minutes. Ask which components contain the rule and which merely consume its result. Reveal duplicated decisions in the browser, handler, and reminder code. Show a missed copy accepting a duration that another component rejects.

CHANGE ONE CONDITION AND COMPARE: Apply the same change to Design B. Highlight the authoritative policy edit and the affected interface/tests, while leaving genuinely unrelated modules untouched. Compare edit locations, revalidation needs, and inconsistency risk; do not promise that one module means only one file ever changes.

PRINCIPLE TO REVEAL: Useful boundaries localize decisions while keeping their dependents and evidence visible.

STUDENT PDF TREATMENT: Print the baseline graphs and changed graphs with an edit/recheck/unaffected legend.

03. A MODULE NAME IS NOT A REASON FOR ITS CONTENTS TO BELONG TOGETHER

Bridge scene | ID: cohesion-and-responsibility

VISUAL IDEA: Responsibility cards cluster by reason to change rather than by file extension.

SITUATION, PREDICTION, AND MECHANISM: Show a “Utilities” module containing booking rules, email formatting, date display, and storage access. Ask which tasks change for the same reason. Reveal a regrouping around meaningful responsibilities. Make construction visible with the existing duration predicate: isolate the deterministic policy from clock, storage and notification effects. Walk an input through validation, a domain decision and an explicit result. Clear names and small responsibilities help a maintainer locate the decision; short code alone is not the objective.

CHANGE ONE CONDITION AND COMPARE: Add a new notification format while keeping booking policy unchanged. Compare the groups affected before and after regrouping. Acknowledge that a boundary that separates tightly related behavior can also create unnecessary coordination.

PRINCIPLE TO REVEAL: Cohesion concerns meaningful related responsibility, not simply making modules small.

STUDENT PDF TREATMENT: Print the responsibility grouping and the change that explains each boundary.

04. A REPLACEMENT FITS ONLY WHEN ITS BEHAVIOR FITS

Anchor scene | ID: interface-contract

VISUAL IDEA: A component plugs into a labeled interface; inputs, outputs, errors, and side effects are shown as contract cards.

SITUATION, PREDICTION, AND MECHANISM: Replace a notification implementation while keeping its call shape unchanged. Ask whether matching function names guarantees compatibility. Reveal differences in failure reporting, repeated delivery, and whether the call blocks confirmation. Use R-04 to state the relevant behavior. Use the return value to distinguish a successful booking decision, a domain rejection and unavailable infrastructure. A well-shaped

value can still violate a domain constraint, and a type annotation does not enforce authorization or an atomic booking decision. Keep these distinctions in Python-like pseudocode and a small result table.

CHANGE ONE CONDITION AND COMPARE: Change only the error behavior of the replacement. Compare a clearly reported failed notification with an implementation that throws into the booking path and undoes the user promise. Introduce preconditions, outcomes, and error expectations without a long type-system lecture.

PRINCIPLE TO REVEAL: An interface contract includes behavior and failure semantics, not just matching names or parameter lists.

STUDENT PDF TREATMENT: Print the two contract cards and the mismatch that breaks substitutability.

05. WHO SHOULD KNOW ABOUT THE REPLACEABLE DETAIL?

Bridge scene | ID: dependency-direction

VISUAL IDEA: A policy component and storage adapter exchange a small domain-shaped interface; dependency arrows are distinct from runtime message arrows.

SITUATION, PREDICTION, AND MECHANISM: Ask whether the booking rule should depend on a particular database client's response shape. Reveal an interface describing the domain decision instead. Keep runtime calls visible on a separate line to avoid confusing execution with source-code dependency.

CHANGE ONE CONDITION AND COMPARE: Change the storage implementation while preserving the required atomic booking behavior. Compare which code needs knowledge of the change and what must still be verified.

PRINCIPLE TO REVEAL: Depend on the behavior a responsibility needs, and isolate details that change for other reasons.

STUDENT PDF TREATMENT: Print the source dependency view and runtime interaction view with explicit legends.

06. CHANGE THE STRUCTURE WITHOUT QUIETLY CHANGING THE PROMISE

Anchor scene | ID: refactoring-under-checks

VISUAL IDEA: A short pseudocode excerpt transforms in small steps beside a fixed table of expected outcomes.

SITUATION, PREDICTION, AND MECHANISM: Start with duplicated duration checks and several named examples. Ask whether extracting a policy function should alter the answers. Reveal each structural change, then replay the same normal, boundary, and invalid inputs. Name refactoring only after the behavior comparison.

CHANGE ONE CONDITION AND COMPARE: Introduce an accidental inclusive-boundary error during one step. The 120-minute example changes outcome. Compare a structural intention with the actual behavioral effect and restore the correct condition. Explain that checks provide evidence, not a proof of all behavior.

PRINCIPLE TO REVEAL: Refactoring should preserve intended observable behavior; small steps and relevant checks make deviations easier to locate.

STUDENT PDF TREATMENT: Print before/after pseudocode and the regression example that exposed the mistaken boundary.

07. A FLEXIBLE DESIGN CAN STILL BE UNNECESSARILY COMPLICATED

Bridge scene | ID: abstraction-cost

VISUAL IDEA: A comparison matrix contrasts duplication, a small policy function, and a generic rule framework.

SITUATION, PREDICTION, AND MECHANISM: Ask which option fits two clear policy rules and a small team. Reveal setup, understanding, extension, and debugging costs as qualitative judgments. Avoid invented precision or a decorative three-dimensional score surface.

CHANGE ONE CONDITION AND COMPARE: Change one condition: several independently changing policies now exist. Revisit the decision and identify what evidence might justify more structure. Keep flexibility as a costed option, not a virtue without limits. Offer building a small adapter, reusing a maintained library, and buying a managed capability as alternatives. Compare integration, licensing, support, data portability and exit cost using qualitative evidence. Buying or reusing changes responsibility; it does not remove the need to verify the required behavior.

PRINCIPLE TO REVEAL: Abstraction should earn its complexity by simplifying real responsibilities or plausible change.

STUDENT PDF TREATMENT: Print the alternatives and the condition under which the recommendation changes.

08. A FUTURE MAINTAINER NEEDS THE REASON, NOT ONLY THE DIAGRAM

Bridge scene | ID: design-rationale

VISUAL IDEA: A compact decision record connects context, alternatives, choice, consequences, and revisit trigger.

SITUATION, PREDICTION, AND MECHANISM: Ask what will be missing when a maintainer sees only “Use a Policy component.” Reveal why the boundary was chosen, what it deliberately does not generalize, and which assumptions could make the choice obsolete.

CHANGE ONE CONDITION AND COMPARE: Change the scope from one campus to several with different rules. Compare blindly preserving the old decision with revisiting its stated trigger.

PRINCIPLE TO REVEAL: A design decision is easier to maintain when its reasoning and limits are recorded.

STUDENT PDF TREATMENT: Print the concise decision record with the relevant requirement IDs.

6. TEACHING GUARDRAILS

Avoid presenting SOLID, DRY, or a pattern name as an automatic answer. Similar-looking code does not necessarily represent one shared concept, and duplicated policy is different from deliberately independent verification. A local client check may improve feedback but must not be

the authority for a server-side rule.

Dependency graphs should distinguish “must edit” from “must reconsider or recheck.” Lower visual edge count does not by itself establish a superior design. Keep the examples focused enough that students can trace every highlighted change. Source-code dependency direction and runtime call direction may differ; label which diagram is on screen.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Less coupling means no dependencies.” A useful system still needs dependencies; the question is which ones and on what behavior.

“Refactoring means any improvement.” A feature change or changed requirement is not purely behavior-preserving restructuring.

“One interface guarantees easy replacement.” Revisit error behavior, side effects, and the contract.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Why is a duplicated duration rule risky?

ANSWER: Copies can change inconsistently and create conflicting decisions.

QUESTION 2: Does a centralized rule mean only one check is needed?

ANSWER: No. Different checks should still independently examine its important behavior and integration.

QUESTION 3: Why did the replacement notifier break the design despite matching parameters?

ANSWER: It violated the expected failure behavior and affected confirmation.

QUESTION 4: What should an architecture or design decision record preserve?

ANSWER: Context, alternatives, choice, consequences, and the assumptions or triggers that warrant revisiting it.

QUESTION 5: Does a well-typed result prove that cancellation is authorized?

ANSWER: No. Shape, domain validity and permission are separate claims with separate evidence.

QUESTION 6: An image library changes its file decoder while retaining the same function name. What does a caller still need to check?

ANSWER: Check accepted inputs, pixel and color meaning, error results and side effects promised by the interface. A matching name or type shape does not settle behavioral compatibility.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

The PDF must preserve both designs before and after the same change. Include the change-impact legend, the behavioral interface contract, and the refactoring check table. Do not print only the final clean diagram: the comparison is the explanation. The stable main anchor is week-05.html#/dependency-example.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect dependency spread, behavioral contracts and refactoring. Construction is the mechanism inside those demonstrations. Optional depth: name Strategy or Adapter only after its responsibility and trade-off are visible; omit a SOLID or pattern catalogue.

10. HANDOFF

Carry the module responsibilities, contracts, and decision record into Week 6. The next lecture asks what changes when those responsibilities cross process, network, and data-consistency boundaries.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S07, S24. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 06 | ARCHITECTURE, INTERACTIONS, AND DISTRIBUTED FAILURE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

What changes when a dependency becomes slow, unavailable, or uncertain?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-06.html

1. PURPOSE, CASE STATE, AND SCOPE

Scale the Week 5 design from internal responsibilities to runtime structure. Compare a modular single application with a separated notification worker and with a more distributed alternative. Teach architecture through responsibility, deployability, data ownership, contracts, and failure—not through a shopping list of infrastructure products.

The three main mechanisms are a request traveling across components, two concurrent booking attempts competing for one invariant, and a lost response creating an uncertain outcome. The viewer should leave with disciplined trade-off questions rather than a rule that a newer architectural style is always better. [S01, S02, S08, S09, S10]

In practice (dated examples, 2026): HTTP contracts are often described in OpenAPI; PostgreSQL can enforce a no-overlap rule for time ranges with an exclusion constraint; queued work commonly runs on RabbitMQ or Kafka; and services ship as Docker images, which are a packaging format, not the containers of the C4 model.

2. OBSERVABLE LEARNING OUTCOMES

1. Distinguish a module boundary from a process or deployment boundary.

2. Read context, component, and runtime views at an appropriate level.

3. Explain how a concurrent check-then-write can violate a shared invariant.

4. Distinguish timeout, failure, and unknown outcome.

5. Compare synchronous and asynchronous interactions, including their costs.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare the same named components as Week 5 and two architecture alternatives using the same workload. Use simple C4-inspired zoom levels, explicitly explaining that a C4 container is an application or data store boundary, not necessarily an operating-system container. [S08]

Prepare a deterministic interleaving: A reads available, B reads available, A confirms, B confirms. Prepare a corrected authoritative decision that prevents the overlap, plus a retry fixture where the original write succeeded but the response was lost. No actual distributed cluster is required for the lecture.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | A box can mean a responsibility, a process, or a machine | Bridge

week-06.html#/architecture-zoom

02 | Follow one request across real boundaries | Anchor

week-06.html#/request-journey

03 | More deployable pieces create more boundaries to manage | Bridge

week-06.html#/monolith-and-services

04 | Two individually sensible actions can violate one shared rule | Anchor

week-06.html#/concurrent-booking

05 | A timeout does not tell us whether a write happened | Anchor

week-06.html#/retry-and-idempotency

06 | A confirmed booking and a pending email can both be true | Bridge

week-06.html#/async-and-consistency

07 | An added field and a changed meaning are not equivalent | Bridge

week-06.html#/contracts-and-evolution

08 | Choose the smallest structure that protects the required promises | Bridge

week-06.html#/architecture-decision

5. DETAILED SCENE TEACHING NOTES

01. A BOX CAN MEAN A RESPONSIBILITY, A PROCESS, OR A MACHINE

Bridge scene | ID: architecture-zoom

VISUAL IDEA: A controlled zoom moves from system context to runtime containers to components, preserving labels and the current question.

SITUATION, PREDICTION, AND MECHANISM: Ask what a box labeled Booking represents. Reveal three distinct meanings on separate views. Trace the same user request without pretending every component runs on its own server.

CHANGE ONE CONDITION AND COMPARE: Move Notification into a separate worker while leaving Policy and Booking in one application. Compare what changed operationally with what stayed the same logically. [S08]

PRINCIPLE TO REVEAL: Architecture views should state their level; logical separation does not automatically imply network separation.

STUDENT PDF TREATMENT: Print the three labeled views with a brief legend defining each boundary.

02. FOLLOW ONE REQUEST ACROSS REAL BOUNDARIES

Anchor scene | ID: request-journey

VISUAL IDEA: A sequence diagram and runtime map highlight the same request ID, with latency segments and failure points.

SITUATION, PREDICTION, AND MECHANISM: Ask which dependency the user is waiting for at each step. Reveal identity checking, policy validation, authoritative persistence, response, and later notification. Label the difference between a call that blocks the response and work that continues after it.

CHANGE ONE CONDITION AND COMPARE: Make notification unavailable while preserving the authoritative store. Compare a design that makes all confirmation wait for email with one that records notification work separately. State the extra responsibility: pending delivery must be observable and recoverable, not merely forgotten.

PRINCIPLE TO REVEAL: Interaction structure determines which failures affect which user promises.

STUDENT PDF TREATMENT: Print the two request sequences with blocking and asynchronous steps labeled in text.

03. MORE DEPLOYABLE PIECES CREATE MORE BOUNDARIES TO MANAGE

Bridge scene | ID: monolith-and-services

VISUAL IDEA: A side-by-side architecture comparison and a qualitative trade-off table.

SITUATION, PREDICTION, AND MECHANISM: Ask what a separate service buys a small team. Reveal possible independent release or scaling benefits beside network failures, contract coordination, data ownership, and operational work. Keep the single-application version modular rather than drawing it as an unstructured blob.

CHANGE ONE CONDITION AND COMPARE: Change only one condition: notification volume grows independently of booking writes. Compare the targeted benefit of separating that workload with splitting every component.

PRINCIPLE TO REVEAL: Choose deployment boundaries for demonstrated needs; modularity and distribution are different decisions.

STUDENT PDF TREATMENT: Print both alternatives and the changed condition, with no universal winner label.

04. TWO INDIVIDUALLY SENSIBLE ACTIONS CAN VIOLATE ONE SHARED RULE

Anchor scene | ID: concurrent-booking

VISUAL IDEA: Two request lanes advance through Read, Decide, and Write beside one room calendar.

SITUATION, PREDICTION, AND MECHANISM: Pause after both requests read an available interval. Ask what each will decide. Reveal two confirmations in the naive check-then-write design. The key issue is the shared invariant, not which user clicked first by a few milliseconds.

CHANGE ONE CONDITION AND COMPARE: Replace the separate check and write with an authoritative, atomic enforcement strategy for the no-overlap invariant. Step the same competing requests: one confirms, the other receives a conflict or retries appropriately. Explain that isolation choices, constraints, locking, and retries must be selected correctly; merely writing “transaction” around code is not a universal fix. [S10]

PRINCIPLE TO REVEAL: An invariant shared by concurrent operations needs enforcement that survives their interleavings.

STUDENT PDF TREATMENT: Print the violating interleaving and corrected interleaving with the invariant beside both.

05. A TIMEOUT DOES NOT TELL US WHETHER A WRITE HAPPENED

Anchor scene | ID: retry-and-idempotency

VISUAL IDEA: A message timeline, an operation identifier, and a stored-result panel show a lost response and retry.

SITUATION, PREDICTION, AND MECHANISM: Store B-001, then lose its response. Ask whether the requester should submit a new operation. Reveal how an unrecognized retry can create a duplicate logical action. Introduce an operation key scoped to the requester and intended action.

CHANGE ONE CONDITION AND COMPARE: Retry with the same key and identical request. Compare a stable stored result with repeating the effect. Then change the payload while reusing the key and show an explicit conflict. Distinguish this application mechanism from assuming every HTTP POST is idempotent or that an “exactly once” guarantee appears automatically. [S09]

PRINCIPLE TO REVEAL: Retries need defined semantics for identity, repeated effects, and uncertain outcomes.

STUDENT PDF TREATMENT: Print the original request, lost response, repeated request, and reconciled result, including key scope.

06. A CONFIRMED BOOKING AND A PENDING EMAIL CAN BOTH BE TRUE

Bridge scene | ID: async-and-consistency

VISUAL IDEA: A booking record and a pending notification record advance on separate timelines.

SITUATION, PREDICTION, AND MECHANISM: Ask whether delayed email means the booking failed. Reveal R-04: confirmation depends on the booking decision, while notification may complete later. Show pending, delivered, and failed-delivery observations without hiding them from support.

CHANGE ONE CONDITION AND COMPARE: Duplicate a delivery attempt while keeping the booking unchanged. Compare a handler that tolerates repeat work with one that produces unintended extra effects. Mention a durable handoff pattern without requiring implementation detail. Name the durable handoff as a transactional outbox: the booking change and a pending notification record are committed together, then a worker attempts delivery. Step a crash after commit but before delivery; pending work remains discoverable. A crash after sending but before acknowledgement can still lead to a duplicate attempt. Keep delivery semantics explicit rather than implying exactly-once email.

PRINCIPLE TO REVEAL: Asynchronous work changes the timing and failure contract; it does not remove responsibility for completion.

STUDENT PDF TREATMENT: Print the two timelines and the user-facing meaning of each combination.

07. AN ADDED FIELD AND A CHANGED MEANING ARE NOT EQUIVALENT

Bridge scene | ID: contracts-and-evolution

VISUAL IDEA: A request/response contract appears beside old and new consumers.

SITUATION, PREDICTION, AND MECHANISM: Ask whether adding a response field is always harmless. Reveal that compatibility depends on consumer behavior; strict consumers may reject additions. Contrast that with silently changing the meaning of an existing field. Make one contract concrete: select a booking request, its response shape, the error meaning and the retry rule, then run an old consumer against a changed response. Identify OpenAPI as an HTTP interface-description standard; a schema can describe shape without establishing ownership, no-overlap or retry correctness. [S52]

CHANGE ONE CONDITION AND COMPARE: Change a booking status representation and replay old and new consumers. Compare an explicit transition strategy with an abrupt breaking change.

PRINCIPLE TO REVEAL: Compatibility is a relationship between actual producers and consumers, not a label on a release.

STUDENT PDF TREATMENT: Print a producer-consumer compatibility matrix with the assumptions behind each cell.

08. CHOOSE THE SMALLEST STRUCTURE THAT PROTECTS THE REQUIRED PROMISES

Bridge scene | ID: architecture-decision

VISUAL IDEA: A decision record ties quality scenarios, trade-offs, ownership, and failure behavior to one selected architecture.

SITUATION, PREDICTION, AND MECHANISM: Ask which evidence supports the default architecture for this case. Reveal a modular application with an authoritative store and separately recoverable notification work as a teaching baseline, not a universal prescription.

CHANGE ONE CONDITION AND COMPARE: Change the constraints to multiple independently operated campuses. Identify what must be reconsidered without pretending the current lecture has solved cross-region consistency.

PRINCIPLE TO REVEAL: Architecture decisions should explain which requirements they protect and which assumptions would change the choice.

STUDENT PDF TREATMENT: Print the decision record with the unresolved questions and revisit triggers.

6. TEACHING GUARDRAILS

Do not teach that eventual consistency is suitable for every datum: display availability may be stale, but the canonical no-overlap decision still needs authoritative enforcement. Do not equate transactions with a guarantee at every isolation level. Locking an existing row may also fail to protect a gap when no conflicting row exists; an implementation must enforce the interval invariant, not merely a superficial uniqueness condition. [S10]

A timeout is an observation at one boundary, not proof of the remote effect. Retry demonstrations use explicit operation identities and defined duplicate handling. Keep technical implementation choices open for authoring; preserve the semantics in every animation.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Microservices mean loose coupling.” A distributed design can still require coordinated changes and releases.

“A transaction always prevents double booking.” Ask which invariant and concurrent anomaly the selected strategy actually prevents.

“An error message means nothing was saved.” Revisit the lost-response timeline.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Why can two availability checks both return true?

ANSWER: They can observe the same earlier state before either write has been enforced.

QUESTION 2: Why not block confirmation on every notification attempt?

ANSWER: It couples the booking promise to an external dependency unnecessarily under R-04; separate work still needs reliable tracking.

QUESTION 3: What should happen when an operation key is reused with a different payload?

ANSWER: A defined conflict or rejection, not silently treating the altered request as the original operation.

QUESTION 4: Is a C4 container always a Docker container?

ANSWER: No. It denotes an application or data-store boundary in that model, not a specific packaging technology.

QUESTION 5: What survives a crash after an atomic booking-and-pending-notification commit?

ANSWER: The booking and the pending work remain recorded; delivery still needs recovery, and a later retry can repeat a delivery attempt.

QUESTION 6: A batch importer crashes after storing a record but before marking its input complete. What makes a restart safe?

ANSWER: Define a stable identity for the intended operation and coordinate its effect with durable completion evidence, or make replay preserve the intended result. A crash does not establish that no write occurred; reusing an identity for changed input needs an explicit policy.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve the successful path, dependency failure, concurrent conflict, and lost-response storyboards. Keep event numbering consistent between HTML and PDF. Include the compatibility matrix and the architecture rationale. Every asynchronous arrow must retain a caption explaining what the user can rely on before that work finishes.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect request boundaries, concurrent enforcement and retry identity. Keep the API contract and outbox inside their current scenes. Optional depth: compare cached search with authoritative confirmation; leave consensus, full CAP theory and multi-region design for later study.

10. HANDOFF

Carry the contracts and decision record into Week 7. Now several developers need to change this system together without losing intent, context, or evidence.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S08, S09, S10, S52. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 07 | COLLABORATION, VERSION HISTORY, AND REVIEWABLE CHANGE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

How can several people change one system without losing intent or trust?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-07.html

1. PURPOSE, CASE STATE, AND SCOPE

Move from the structure of the software to the structure of shared work. The central object is a change with a reason, a history, and evidence—not a sequence of Git commands. Make branching and merging visible, then reveal why combining text successfully does not establish that the combined behavior is correct.

Keep the interface neutral: a commit graph, readable diffs, review comments, and a small evidence panel. Explain version control, code review, configuration management, and documentation as complementary forms of coordination. Students do not need accounts on a hosting platform. [S01, S02, S06, S07]

In practice (dated examples, 2026): Git records the history; GitHub and GitLab host the pull or merge requests where review happens; lockfiles such as package-lock.json or uv.lock record the resolved dependency set; linters and formatters such as Ruff, ESLint and Prettier run before a human reviews the change. [S62, S63]

2. OBSERVABLE LEARNING OUTCOMES

1. Read a small commit graph and distinguish commits, branch references, and merges.

2. Explain the difference between a textual conflict and an incompatible behavioral change.

3. Identify the context and evidence needed to review a change.

4. Explain how dependency and configuration choices affect reproducibility.

5. Use history and decision records to explain why a change was made.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare a six-commit fictional history, two short readable diffs, and one deliberately misleading clean merge. Use a temporary policy variant reducing the maximum duration from 120 to 90 minutes. One branch changes the policy; another adds a hard-coded message saying 120 minutes. Label the variant clearly and restore the canonical rule after the scene.

Recall the Week 5 change-impact graph. Ask how two people would discover that their apparently separate edits affect the same promise. Keep code fragments small enough to compare without scrolling; syntax coloring must not carry the meaning alone.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | History is a graph of states and relationships | Anchor

week-07.html#/commit-graph

02 | A conflict asks for a decision about meaning | Anchor

week-07.html#/conflict-and-intent

03 | No conflict does not mean no contradiction | Bridge

week-07.html#/clean-merge-wrong-behavior

04 | A diff is easier to judge when the claim is visible | Anchor

week-07.html#/review-with-context

05 | Distance from shared reality can grow quietly | Bridge

week-07.html#/integration-frequency

06 | A source snapshot is only part of a reproducible run | Bridge

week-07.html#/same-code-different-result

07 | The reason for a change should survive the conversation | Bridge

week-07.html#/history-as-explanation

08 | Shared confidence comes from several kinds of evidence | Bridge

week-07.html#/ready-to-integrate

5. DETAILED SCENE TEACHING NOTES

01. HISTORY IS A GRAPH OF STATES AND RELATIONSHIPS

Anchor scene | ID: commit-graph

VISUAL IDEA: A commit graph grows one node at a time beside a tiny snapshot of the booking policy; branch labels move separately from nodes.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a branch is a second permanent copy of every file. Reveal commits as recorded snapshots linked to parent commits, then move a branch reference when a new commit is added. Draw time consistently but explain that graph ancestry, not horizontal position alone, determines the relationship. [S06] Before growing the graph, distinguish the working tree, the selected changes to commit, the local recorded commit and a shared remote reference. Committing and sharing are separate actions; the graph is not a backup of every unsaved edit.

CHANGE ONE CONDITION AND COMPARE: Create a second line of development from the same ancestor. Compare two descendants with different changes, then reveal a merge with two parents. Contrast a fast-forward case without suggesting that every integration needs a merge commit.

PRINCIPLE TO REVEAL: History records relationships between changes; branch names are movable references, not the changes themselves.

STUDENT PDF TREATMENT: Print the common ancestor, divergent descendants, and merged state in three labeled panels.

02. A CONFLICT ASKS FOR A DECISION ABOUT MEANING

Anchor scene | ID: conflict-and-intent

VISUAL IDEA: Two edits to one policy line appear beside the requirement they were intended to implement.

SITUATION, PREDICTION, AND MECHANISM: Ask whether taking the left or right version is sufficient. Reveal one edit implementing the 90-minute variant and another reorganizing the same validation. Show the shared original so the viewer can distinguish changed behavior from changed structure.

CHANGE ONE CONDITION AND COMPARE: Resolve the text while accidentally losing the upper bound. Compare a syntactically valid result with a requirement-based resolution that preserves both the intended rule and the structural improvement. Reveal the boundary check that detects the mistake.

PRINCIPLE TO REVEAL: A merge resolution must reconcile intent and verify the combined behavior, not merely remove conflict markers.

STUDENT PDF TREATMENT: Print ancestor, two edits, chosen resolution, and the verification result with compact annotations.

03. NO CONFLICT DOES NOT MEAN NO CONTRADICTION

Bridge scene | ID: clean-merge-wrong-behavior

VISUAL IDEA: A clean-merge indicator sits above a policy value and a user message that disagree.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a successful merge means the feature is ready. Reveal a policy rejecting durations above 90 while the newly added message still says 120. The files differ, so the version-control tool had no textual disagreement to flag.

CHANGE ONE CONDITION AND COMPARE: Replace the duplicated message value with a response derived from the same approved policy source. Compare the user-visible behavior before and after the change.

PRINCIPLE TO REVEAL: Textual integration and semantic integration are different kinds of evidence.

STUDENT PDF TREATMENT: Print the clean merge and the contradictory outcomes; do not leave only the green merge indicator.

04. A DIFF IS EASIER TO JUDGE WHEN THE CLAIM IS VISIBLE

Anchor scene | ID: review-with-context

VISUAL IDEA: A review panel connects requirement, diff, checks, decision rationale, and focused comments.

SITUATION, PREDICTION, AND MECHANISM: Show the diff alone and ask what a reviewer cannot yet decide. Reveal the purpose, affected rule, alternatives, and checks. Sort comments into correctness, security, design, understandability, and optional preference. Explain that linters and formatters take style out of the conversation without replacing judgment, and that a useful comment addresses the change rather than the person and says whether it is required or optional. [S07]

CHANGE ONE CONDITION AND COMPARE: Split an unrelated rename out of the same change. Compare the reviewer's task with and without the unrelated material, without claiming an exact universal speed improvement. Keep one substantive concern unresolved until its supporting evidence appears. AI beat: label the same diff as drafted by an automated assistant with the description "fix stuff". Ask what changes for the reviewer. Nothing: the requirement, the checks and the rationale are still required, and the empty description is the documentation failure of scene 07 arriving early. Resolve the remaining disagreement by restating the claim, identifying the missing evidence and recording the decision owner. Compare asynchronous review with a short paired investigation when shared context is missing; either can work when its findings survive the conversation.

PRINCIPLE TO REVEAL: Review is an evidence-based conversation about a change, not a vote on how convincing its author sounds.

STUDENT PDF TREATMENT: Print the compact review packet and two example comments with their supporting reasons.

05. DISTANCE FROM SHARED REALITY CAN GROW QUIETLY

Bridge scene | ID: integration-frequency

VISUAL IDEA: Two timelines show the same intended changes integrated in small steps or accumulated on a long-lived branch.

SITUATION, PREDICTION, AND MECHANISM: Ask where incompatibilities would become visible. Reveal the integration points and overlapping assumptions, not just the number of commits. Explain that smaller steps still need useful checks and a safe way to keep unfinished behavior unavailable.

CHANGE ONE CONDITION AND COMPARE: Introduce one contract change halfway through both timelines. Compare when each branch learns about it. Avoid presenting one branching convention as mandatory for every team.

PRINCIPLE TO REVEAL: Choose a collaboration strategy that exposes incompatible assumptions early enough to act on them.

STUDENT PDF TREATMENT: Print both timelines with the contract-change event and discovery points.

06. A SOURCE SNAPSHOT IS ONLY PART OF A REPRODUCIBLE RUN

Bridge scene | ID: same-code-different-result

VISUAL IDEA: A source revision connects to dependency versions, runtime, configuration, data fixture, and build artifact.

SITUATION, PREDICTION, AND MECHANISM: Ask why the same code might behave differently on two machines. Reveal one changed dependency and one different configuration value. Distinguish a recorded version constraint from the actual resolved dependency set.

CHANGE ONE CONDITION AND COMPARE: Hold the source fixed and restore the declared dependency and configuration inputs. Compare repeatability of the teaching result. Keep secrets out of the diagram and out of the repository; use placeholder references.

PRINCIPLE TO REVEAL: Reproducibility requires control of the relevant inputs, not just possession of the source files.

STUDENT PDF TREATMENT: Print the input manifest and the single difference responsible for the changed behavior.

07. THE REASON FOR A CHANGE SHOULD SURVIVE THE CONVERSATION

Bridge scene | ID: history-as-explanation

VISUAL IDEA: A short change description and decision record are linked to the commit and requirement.

SITUATION, PREDICTION, AND MECHANISM: Ask what a maintainer learns from “fix stuff.” Reveal an alternative explaining the observed problem, intended behavior, and a constraint that ruled out an obvious solution. Separate durable rationale from a transcript of every discussion. Then show history as a debugging tool: when the boundary error appears and nobody knows which change introduced it, bisection checks the middle commit of the suspect range and halves the range until one change remains. [S61]

CHANGE ONE CONDITION AND COMPARE: Remove the original author from the scenario. Compare how much each record helps a future reviewer understand whether the constraint still applies.

PRINCIPLE TO REVEAL: Documentation earns its place by preserving information that cannot be reliably reconstructed from the code.

STUDENT PDF TREATMENT: Print the weak and improved descriptions with the useful information highlighted in words, and the bisection steps that located the change.

08. SHARED CONFIDENCE COMES FROM SEVERAL KINDS OF EVIDENCE

Bridge scene | ID: ready-to-integrate

VISUAL IDEA: A change card collects review, checks, integration status, and remaining uncertainty without turning them into a numerical score.

SITUATION, PREDICTION, AND MECHANISM: Ask which missing evidence prevents this change from being integrated responsibly. Reveal that passing checks does not settle an unresolved authorization question, while approval does not fix a failing integration check.

CHANGE ONE CONDITION AND COMPARE: Resolve the actual concern, rerun the affected checks, and compare the evidence packet. Restore the canonical 120-minute policy and show that later weeks use that baseline.

PRINCIPLE TO REVEAL: Integration readiness is a justified judgment about the change and its context, not a single green indicator.

STUDENT PDF TREATMENT: Print the final evidence packet and an explicit baseline-restoration note.

6. TEACHING GUARDRAILS

Do not turn the lecture into a hosting-platform tour or a memorization exercise for commands. A commit identifier identifies recorded content in its version-control context; it is not proof that the content is safe or correct. A successful merge establishes neither acceptance nor release readiness.

Show behavior and intent before naming a workflow. Keep examples of branch rewriting conceptual; do not prescribe rewriting shared history without coordination. Reproducible dependency selection also does not mean a dependency is trustworthy. That distinction leads into Week 9.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Git chooses the correct business rule.” It can combine content; people and checks must resolve intended behavior.

“Approval proves correctness.” Review is bounded evidence and may miss assumptions.

“A small diff is automatically safe.” A one-character authorization or boundary change can have large consequences.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: What distinguishes a branch label from a commit?

ANSWER: A branch label points to a commit and can move; the recorded commit represents a particular snapshot and ancestry.

QUESTION 2: Can two changes merge cleanly and still conflict?

ANSWER: Yes. Their behavior or assumptions can contradict even when the edited text does not overlap.

QUESTION 3: What context should accompany a policy change?

ANSWER: The intended requirement, affected behavior, relevant design constraints, and checks demonstrating the change.

QUESTION 4: Why record dependency inputs?

ANSWER: The same source can run differently when its resolved dependencies or configuration differ.

QUESTION 5: What should resolve a substantive review disagreement?

ANSWER: A clear claim, the evidence needed to decide it and an accountable decision; durable rationale should survive the discussion.

QUESTION 6: A library's code and documentation merge cleanly, but they describe different default behavior. Is the change ready?

ANSWER: Resolve the intended contract and check the implementation and documentation against it. A textual merge detects some overlapping edits; it does not reconcile meaning across different files.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve the commit graph at meaningful points, the conflict resolution, and the clean-merge counterexample. Provide readable diffs as text rather than tiny full-screen captures. The review example must retain the requirement and evidence, not only comments. State that the 90-minute rule was a temporary variant.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect commit ancestry, conflict intent and focused review. The working-tree-to-shared-history strip is preparation for the graph. Optional depth: compare revert and history rewriting on shared work; do not add a Git command catalogue.

10. HANDOFF

Carry the review packet and its unanswered question into Week 8: what makes a verification check useful, and what can a passing suite still fail to establish?

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S06, S07, S61, S62, S63. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. Week 7 source re-verification: 28 October 2026; Git bisection and the two lockfile references were added to support those explanations. S01 returned HTTP 403; its last recorded edition is retained. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 08 | TESTING BEHAVIOR AND BUILDING CREDIBLE EVIDENCE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Which checks would challenge our assumptions rather than merely repeat them?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-08.html

1. PURPOSE, CASE STATE, AND SCOPE

Build testing from the behavior students already understand. Start with a booking-duration function that passes a friendly example but rejects a valid boundary. Connect examples, properties, interfaces, and system-level checks to the particular claims they support.

The lecture should make evidence visible without presenting one test taxonomy, framework, or fixed test-pyramid ratio as a universal formula. Treat testability, independent expected results, representative conditions, and useful failure messages as engineering choices. [S01, S02, S40]

In practice (dated examples, 2026): pytest and JUnit run example checks, Hypothesis generates property-based cases, Stryker and mutmut run mutation analysis, coverage.py measures executed lines, and Playwright drives complete browser journeys.

2. OBSERVABLE LEARNING OUTCOMES

1. Derive ordinary, boundary, and invalid cases from a stated rule.

2. Choose a suitable scope for a check and explain what it leaves unexamined.

3. Distinguish executing code from asserting the intended result.

4. Explain a property and a small mutation-based challenge to a test suite.

5. Recognize shared assumptions, uncontrolled inputs, and flaky observations.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare the canonical duration rule: whole-minute durations from 30 through 120 inclusive, with positive end-after-start ordering. Use 29, 30, 31, 119, 120, and 121 as a boundary table. Do not introduce a 30-minute slot grid. Prepare the incorrect comparison “duration < 120” and its corrected inclusive version in readable pseudocode.

Prepare a clock-controlled cancellation example, an overlap example accepting adjacent intervals, and a deterministic fake notification failure. Recall the requirement-to-check links from Week 2. All results must be computed or independently verified rather than assigned to make the animation persuasive.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | One passing example can hide the wrong boundary | Anchor

week-08.html#/friendly-example

02 | Input categories make omissions easier to see | Bridge

week-08.html#/partition-the-inputs

03 | Different checks answer different questions | Anchor

week-08.html#/choose-test-scope

04 | The expected answer needs its own justification | Bridge

week-08.html#/oracle-independence

05 | A useful property describes more than one example | Anchor

week-08.html#/properties-and-counterexamples

06 | Executed is not the same as checked | Bridge

week-08.html#/coverage-and-mutation

07 | A moving clock can make a stable rule look unreliable | Bridge

week-08.html#/repeatable-observation

08 | A check can guide a change and later guard its meaning | Bridge

week-08.html#/testing-as-feedback

5. DETAILED SCENE TEACHING NOTES

01. ONE PASSING EXAMPLE CAN HIDE THE WRONG BOUNDARY

Anchor scene | ID: friendly-example

VISUAL IDEA: Input cards move through a small predicate; accepted and rejected outcomes are shown beside the requirement.

SITUATION, PREDICTION, AND MECHANISM: Run 60 minutes and ask whether the rule is implemented correctly. Reveal that both the correct and incorrect comparisons pass this example. Step through 119, then pause before 120. Ask for a prediction before revealing the rejection caused by the exclusive upper bound.

CHANGE ONE CONDITION AND COMPARE: Change only the comparison to inclusive and replay the same cases. Compare 120 and 121 so the fix does not accidentally remove the upper limit.

PRINCIPLE TO REVEAL: Test cases should distinguish plausible incorrect behavior from the intended behavior.

STUDENT PDF TREATMENT: Print the rule, both predicates, and the completed six-case boundary table.

02. INPUT CATEGORIES MAKE OMISSIONS EASIER TO SEE

Bridge scene | ID: partition-the-inputs

VISUAL IDEA: A number line and a small two-dimensional table separate valid durations, too-short values, too-long values, and malformed or reversed intervals.

SITUATION, PREDICTION, AND MECHANISM: Ask whether testing every minute is the only way to gain confidence. Reveal categories with shared expectations, then focus attention on boundaries where the decision changes. Add invalid input shape and end-before-start as different concerns from an out-of-range duration.

CHANGE ONE CONDITION AND COMPARE: Change the rule to the explicitly labeled 90-minute variant. Compare which cases remain useful and which boundary cases must change, then restore 120.

PRINCIPLE TO REVEAL: Partitions organize expectations; boundaries expose where one expectation becomes another.

STUDENT PDF TREATMENT: Print the baseline partitions and a small variant annotation without mixing their expected outcomes.

03. DIFFERENT CHECKS ANSWER DIFFERENT QUESTIONS

Anchor scene | ID: choose-test-scope

VISUAL IDEA: A layered system view highlights a pure policy function, component interaction, storage enforcement, and complete user journey.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a unit check of overlap arithmetic proves that concurrent requests cannot double-book. Reveal the missing shared-storage behavior. Place focused function checks, integration checks, and a complete booking journey where they observe their respective claims. Use the same keyboard booking journey to explain exploratory testing: start with the question of how someone recovers from a lost response, then choose the next check from what the interface reveals. In the authored comparison, reopening the stored booking retrieves confirmation but the interface still says Submitting. Inspect the corrected Confirmed state and capture the discovered mismatch as a repeatable regression example. This is guided investigation with recorded observations; scripted checks preserve known expectations. A separate usability study would ask representative people to attempt the task and explain what they understood. The scene is a simulated example, not a claim that such a study occurred. [S60]

CHANGE ONE CONDITION AND COMPARE: Replace the real notification boundary with a controlled test double. Compare faster isolation of booking behavior with the remaining uncertainty about actual integration. Show a separate contract or integration check for that uncertainty. Use a consumer expectation that the actual provider violates to explain contract testing, then keep the cross-component journey check for behavior that neither side can establish alone. A mock returning the desired value is not evidence that the real boundary behaves that way.

PRINCIPLE TO REVEAL: The appropriate test scope follows the claim and failure risk, not the prestige of a test label.

STUDENT PDF TREATMENT: Print a claim-to-check-scope matrix with one explicit limitation per row, and retain the exploration question, discovered status mismatch and resulting regression example.

04. THE EXPECTED ANSWER NEEDS ITS OWN JUSTIFICATION

Bridge scene | ID: oracle-independence

VISUAL IDEA: Requirement, implementation, and expected-result cards form a triangle; a copied assumption is visibly traced through two cards.

SITUATION, PREDICTION, AND MECHANISM: Show a test whose expected value is calculated by the same incorrect helper used in production. Ask why agreement is weak evidence. Reveal an independently derived expectation from the stated rule and a hand-checked interval example.

CHANGE ONE CONDITION AND COMPARE: Change the shared helper while leaving the requirement fixed. Compare the self-confirming test with the independent check that detects the discrepancy. AI beat: name the case where the tests and the helper were drafted by the same assistant from the same misreading of R-02. Their agreement is this loop again. Week 13 returns to it as the shared-assumption scene.

PRINCIPLE TO REVEAL: A test oracle—the basis for the expected result—must not simply repeat the defect being examined.

STUDENT PDF TREATMENT: Print the dependency of each expected result and the independent derivation.

05. A USEFUL PROPERTY DESCRIBES MORE THAN ONE EXAMPLE

Anchor scene | ID: properties-and-counterexamples

VISUAL IDEA: A calendar receives a controlled sequence of candidate intervals; a property panel shows whether confirmed bookings overlap.

SITUATION, PREDICTION, AND MECHANISM: Ask what should remain true after any accepted booking. Reveal the no-overlap invariant and several generated-but-fixed examples, including adjacency. Explain that property-based tools can explore many cases and help reduce a failure to a smaller counterexample. [S11]

CHANGE ONE CONDITION AND COMPARE: Change the overlap predicate from strict interval intersection to a comparison that also rejects adjacency. Replay [10:00,11:00) and [11:00,12:00). Compare the property about disallowing real overlaps with a separate property that valid adjacent bookings remain possible.

PRINCIPLE TO REVEAL: Properties expose classes of behavior, but the properties themselves must express the intended rule.

STUDENT PDF TREATMENT: Print the properties, deterministic examples, and smallest displayed counterexample; retain half-open interval notation.

06. EXECUTED IS NOT THE SAME AS CHECKED

Bridge scene | ID: coverage-and-mutation

VISUAL IDEA: A coverage overlay sits beside assertions; a one-character mutation changes a boundary while execution remains similar.

SITUATION, PREDICTION, AND MECHANISM: Ask what a highlighted executed line tells us. Reveal that a test can reach the line without asserting the result that matters. Introduce mutation as intentionally changing behavior to ask whether a suite notices. [S12]

CHANGE ONE CONDITION AND COMPARE: Use the exclusive-boundary mutation and first omit the 120-minute case, then add it. Compare surviving and detected changes. Mention that equivalent or irrelevant mutations can require judgment; a mutation score is not a proof of correctness.

PRINCIPLE TO REVEAL: Coverage and mutation results are diagnostic signals about evidence, not substitutes for requirements.

STUDENT PDF TREATMENT: Print the mutation, missing case, added assertion, and the limited conclusion supported by the result.

07. A MOVING CLOCK CAN MAKE A STABLE RULE LOOK UNRELIABLE

Bridge scene | ID: repeatable-observation

VISUAL IDEA: Two runs share the same code but use different clocks or uncontrolled message timing; the fixture inputs are displayed.

SITUATION, PREDICTION, AND MECHANISM: Ask why “cancel before the start time” sometimes passes and sometimes fails. Reveal that the wall clock crossed the boundary between runs. Replace it with an explicit teaching clock and deterministic dependency behavior.

CHANGE ONE CONDITION AND COMPARE: Keep the clock fixed and inject one genuine notification failure. Compare a repeatable meaningful failure with a test that randomly waits for enough time to pass.

PRINCIPLE TO REVEAL: Control irrelevant variation so a failing check tells us something about the behavior being examined.

STUDENT PDF TREATMENT: Print the uncontrolled inputs, controlled fixture, and the rule-specific result.

08. A CHECK CAN GUIDE A CHANGE AND LATER GUARD ITS MEANING

Bridge scene | ID: testing-as-feedback

VISUAL IDEA: A short requirement-to-failing-check-to-small-change-to-recheck loop ends with a refactoring comparison.

SITUATION, PREDICTION, AND MECHANISM: Ask what should happen before fixing the exclusive upper bound. Debug in four visible moves first: reproduce the failure with the 120-minute case, shrink it to the smallest failing input, state one hypothesis about the comparison, and confirm it with one targeted observation. Then reveal a focused failing check, a minimal correction, and rerun evidence. Explain test-first development as one way to make the expected behavior explicit, not a guarantee of good requirements. Relate the existing examples to verification before and after execution: review and static analysis examine some claims without running the service; dynamic checks observe particular executions. A precondition, postcondition or invariant must still be justified by the requirement.

CHANGE ONE CONDITION AND COMPARE: Refactor the corrected predicate without changing its contract. Compare unchanged behavior under the checks while acknowledging untested conditions and higher-level integration evidence.

PRINCIPLE TO REVEAL: Tests are feedback instruments whose value depends on the claims, observations, and assumptions they preserve.

STUDENT PDF TREATMENT: Print the four debugging moves, the feedback loop and a final claim-evidence-remaining-uncertainty table.

6. TEACHING GUARDRAILS

A passing suite does not prove absence of defects. Do not imply that random generation alone finds every important case or that a test double demonstrates real integration. Coverage percentages require a defined measure and scope; avoid prescribing an arbitrary universal target. [S11, S12, S40]

For interval examples use strict overlap: a.start < b.end and b.start < a.end for valid intervals. A function-level proof or test of this predicate does not establish safe concurrent storage. Keep expected results independent of the implementation and preserve the rule's inclusivity in every table.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

"More tests automatically mean stronger evidence." Repeated happy-path tests can leave the same boundary unexamined.

"A mock proves the external service works." It observes behavior under the assumptions encoded in the substitute.

"Complete coverage means correct software." Execution evidence does not establish that assertions or requirements are sufficient.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Which two cases distinguish an inclusive 120-minute limit from no upper limit?

ANSWER: 120 must be accepted and 121 rejected, under otherwise valid conditions.

QUESTION 2: Do adjacent intervals overlap under the course rule?

ANSWER: No. The end of one may equal the start of the next because the intervals are half-open.

QUESTION 3: Why test the storage boundary as well as the overlap function?

ANSWER: Correct local arithmetic alone does not protect the invariant against concurrent operations.

QUESTION 4: Why can implementation and test agree while both are wrong?

ANSWER: They may share the same mistaken assumption or calculate the expected result using the defective logic.

QUESTION 5: Does a controlled test double establish that the real provider honors its contract?

ANSWER: No. It isolates a claim; evidence at the real boundary is still needed.

QUESTION 6: A file converter passes a test whose expected output is produced by the same conversion function. What would strengthen the test?

ANSWER: Use an independently derived small example and a property justified by the format's meaning, including a relevant boundary. If round-trip conversion is promised, check it too; a shared defect can make both directions agree while still producing the wrong result.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

The student PDF must contain the actual input values, expected outcomes, and reasons—not only pass/fail colors. Preserve the test-scope matrix, the oracle counterexample, and the mutation comparison. Print the fixed clock value used in the cancellation scene. Include the principle and a short statement of remaining uncertainty after each major demonstration.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect discriminating examples, test scope and properties. Use one existing provider mismatch for contract testing and the existing journey for the exploratory comparison, replacing a repeated catalogue of test labels. Optional depth: reduce a malformed-input counterexample to explain fuzzing; bounded model exploration belongs after the Week 3 invariant, without a separate tool survey.

10. HANDOFF

Carry independent checks and explicit invariants into Week 9. Correctness now includes whether an action is permitted, which data may cross a boundary, and what happens when a dependency or input cannot be trusted.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S11, S12, S40, S60. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 09 | SECURITY, PRIVACY, AND TRUSTWORTHY DEPENDENCIES

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

What must remain true when a request, dependency, or assumption cannot be trusted?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-09.html

1. PURPOSE, CASE STATE, AND SCOPE

Treat security as part of the service's behavior rather than an isolated scanner at the end. Return to cancellation: the interface can look correct and the duration checks can pass while a user is allowed to cancel someone else's booking. Build the explanation around assets, trust boundaries, identity, permission, safe handling, and evidence.

Use the OWASP Top 10:2025 as an awareness reference and NIST SSDF 1.1 as the final secure-development baseline in this plan. The retrieved SSDF 1.2 document is still an initial public draft, so its status belongs in instructor currency notes, not in a claim that a new final standard has been adopted. [S01, S02, S13, S14, S15]

In practice (dated examples, 2026): Dependabot and OSV-Scanner report dependencies with known vulnerabilities, Syft generates software bills of materials in SPDX or CycloneDX format, and Sigstore signs and verifies build artifacts.

2. OBSERVABLE LEARNING OUTCOMES

1. Identify an asset, trust boundary, threat, and proposed mitigation in a simple system.

2. Distinguish authentication from authorization and apply ownership rules.

3. Explain why validation and authoritative enforcement belong at trusted boundaries.

4. Compare useful diagnostics with unnecessary personal-data exposure.

5. Explain what dependency inventory and provenance evidence do and do not establish.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Use synthetic users U-01 and U-02, with B-001 owned by U-01. Define the authenticated requester independently of any owner identifier supplied in a form. Prepare a cancellation decision matrix, a simple data-flow diagram, a redacted log comparison, and a fictional dependency chain without real vulnerability claims.

Keep every demonstration local and conceptual. No external targets, real credentials, actual personal data, or live exploit tooling are needed. The aim is to reason about protected behavior and boundaries, not to rehearse attack commands.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | What are we trying to protect, and from whom? | Anchor

week-09.html#/assets-and-boundaries

02 | Being signed in does not authorize every action | Anchor

week-09.html#/identity-is-not-permission

03 | A disabled button is guidance, not a security boundary | Bridge

week-09.html#/trusted-enforcement

04 | Input should not acquire unintended authority | Bridge

week-09.html#/data-not-instructions

05 | Useful evidence does not require collecting everything | Anchor

week-09.html#/privacy-and-diagnostics

06 | A component brings dependencies and assumptions with it | Bridge

week-09.html#/dependency-chain

07 | Failure behavior can preserve or violate the promise | Bridge

week-09.html#/exceptional-conditions

08 | Security evidence belongs throughout the lifecycle | Bridge

week-09.html#/security-through-change

5. DETAILED SCENE TEACHING NOTES

01. WHAT ARE WE TRYING TO PROTECT, AND FROM WHOM?

Anchor scene | ID: assets-and-boundaries

VISUAL IDEA: A data-flow diagram marks booking records, identity assertions, browser input, storage, and notification boundaries.

SITUATION, PREDICTION, AND MECHANISM: Ask which arrows carry information the service can accept without checking. Reveal that user-supplied room, time, and booking identifiers are claims, not authority. Identify unwanted outcomes: false cancellation, data disclosure, denied service, and misleading state.

CHANGE ONE CONDITION AND COMPARE: Change one condition: the browser sends a different booking identifier while the authenticated identity remains U-02. Follow the request across the same boundary and ask which decision must stop an unauthorized effect.

PRINCIPLE TO REVEAL: Threat modeling connects a protected outcome to an entry point, an assumption, and a concrete control.

STUDENT PDF TREATMENT: Print the boundary diagram and a small asset-threat-control table with residual questions.

02. BEING SIGNED IN DOES NOT AUTHORIZE EVERY ACTION

Anchor scene | ID: identity-is-not-permission

VISUAL IDEA: A decision matrix shows requester, owner, booking state, future start time, and cancellation result.

SITUATION, PREDICTION, AND MECHANISM: Ask whether U-02 may cancel B-001 simply because the request is authenticated. Reveal R-03: only the authenticated owner may cancel their own future confirmed booking in the baseline. Show the permission decision next to the requested state transition.

CHANGE ONE CONDITION AND COMPARE: Change only the requester to U-01, then separately change the booking to already started. Compare these independent conditions. A staff closure role does not silently grant a cancellation override; any delegation would be an explicitly revised policy.

PRINCIPLE TO REVEAL: Authentication establishes an identity claim; authorization decides whether that identity may perform this action on this resource.

STUDENT PDF TREATMENT: Print the full decision matrix, including same-user but too-late rejection.

03. A DISABLED BUTTON IS GUIDANCE, NOT A SECURITY BOUNDARY

Bridge scene | ID: trusted-enforcement

VISUAL IDEA: A polished interface sits above a service-side decision point; both paths converge on the same authoritative rule.

SITUATION, PREDICTION, AND MECHANISM: Ask whether hiding the Cancel button prevents an unauthorized request. Reveal that the service still receives requests independently of the visible interface. The trusted boundary must check identity, ownership, state, and time rather than relying on the button.

CHANGE ONE CONDITION AND COMPARE: Bypass only the interface affordance in the illustration, without showing attack tooling. Compare a service that trusts supplied ownership with one that obtains ownership from the protected booking record.

PRINCIPLE TO REVEAL: User-interface constraints help users, while trusted enforcement protects the rule when those constraints are absent.

STUDENT PDF TREATMENT: Print the two paths and the same required service-side decision.

04. INPUT SHOULD NOT ACQUIRE UNINTENDED AUTHORITY

Bridge scene | ID: data-not-instructions

VISUAL IDEA: A structured request is separated into data fields and the operation interpreting them; a small query-template diagram illustrates the boundary.

SITUATION, PREDICTION, AND MECHANISM: Ask what could go wrong when a text value becomes part of the instructions used to access storage. Reveal the conceptual distinction between validated data, parameterized operations, and authorization. Explain that none replaces the other. [S13]

CHANGE ONE CONDITION AND COMPARE: Keep the text value unchanged and change only how the operation treats it: as structure or as data. Compare the interpretation without providing an operational exploitation sequence. AI beat: the same boundary holds when the reader is an automated assistant. A note inside a file it reads is data, not an instruction to the reader. Week 13 examines that case as prompt injection.

PRINCIPLE TO REVEAL: Validate the domain, preserve the data-versus-instruction boundary, and independently check permission.

STUDENT PDF TREATMENT: Print the structured-data illustration with the three distinct responsibilities labeled.

05. USEFUL EVIDENCE DOES NOT REQUIRE COLLECTING EVERYTHING

Anchor scene | ID: privacy-and-diagnostics

VISUAL IDEA: Two incident records are compared field by field, linked to the actual support question.

SITUATION, PREDICTION, AND MECHANISM: Ask which fields distinguish a failed notification from a failed booking. Reveal that a request identifier, outcome category, and dependency status may answer the question without a message body or private meeting description. Mark fictional identifiers as pseudonymous, not magically anonymous.

CHANGE ONE CONDITION AND COMPARE: Add a narrowly justified diagnostic need and choose the minimal additional field, access restriction, and retention rationale. Compare purposeful collection with permanent detailed logging by default. Name the legal context in one line: KVKK (Law No. 6698) in Türkiye and the GDPR in the EU make collecting only what a purpose needs, and keeping it no longer than necessary, a legal duty; the lecture teaches the engineering and gives no compliance advice. [S47, S48]

PRINCIPLE TO REVEAL: Diagnostic value, access, retention, and disclosure risk should be considered together.

STUDENT PDF TREATMENT: Print the before-and-after records and a field-purpose-retention rationale table, without real personal information.

06. A COMPONENT BRINGS DEPENDENCIES AND ASSUMPTIONS WITH IT

Bridge scene | ID: dependency-chain

VISUAL IDEA: A dependency graph expands one level, then an inventory and build-provenance card attach to the selected release.

SITUATION, PREDICTION, AND MECHANISM: Ask whether knowing the name of a direct library tells us everything included in a build. Reveal a transitive dependency. Explain an inventory or software bill of materials as visibility into components, and provenance as evidence about an artifact’s production. [S16]

CHANGE ONE CONDITION AND COMPARE: Change one dependency version while keeping application source fixed. Compare the need to evaluate the new component, rebuild under controlled inputs, and recheck affected behavior. A familiar name, inventory entry, or signature alone does not prove safety. Licensing beat: change only the license of the transitive dependency in the fictional chain, from a permissive license to a copyleft license with a source-availability obligation. Compare what the license inventory now requires before release. Use SPDX identifiers as the vocabulary and keep this an engineering question about obligations and inventory, not legal advice. [S42] Name the legal context in one line: for products within its scope, the EU Cyber Resilience Act links component documentation to lifecycle vulnerability handling. Reporting obligations apply from 11 September 2026; the main obligations, including the component-documentation requirements, apply from 11 December 2027. Applicability and exemptions need

their own legal analysis; do not imply that every website is covered. [S49] Finish the lifecycle: inventory locates the affected artifact, the team evaluates exposure, applies a supported update or mitigation, verifies it and records follow-up. Include a secret as a reference with an owner and revocation path, never as a value placed in source or logs.

PRINCIPLE TO REVEAL: Supply-chain evidence makes dependencies inspectable; it does not eliminate the need for risk judgment and verification.

STUDENT PDF TREATMENT: Print the exact fictional dependency set, the changed node, and the bounded claims of each evidence type. Add the license column with the changed identifier.

07. FAILURE BEHAVIOR CAN PRESERVE OR VIOLATE THE PROMISE

Bridge scene | ID: exceptional-conditions

VISUAL IDEA: A permission check, booking write, and notification step each receive a separate failure card.

SITUATION, PREDICTION, AND MECHANISM: Ask whether every failure should have the same response. Reveal that an unavailable permission decision must not silently grant access, while an email failure must not undo a confirmed booking under R-04. State the protected property at each boundary.

CHANGE ONE CONDITION AND COMPARE: Move the failure from notification to authorization while keeping the request fixed. Compare safe outcomes and user messages rather than applying a blanket “always continue” or “always roll back everything” rule.

PRINCIPLE TO REVEAL: Handling failure safely requires knowing which promise each operation is responsible for preserving.

STUDENT PDF TREATMENT: Print a failure-location-to-safe-outcome table with R-03 and R-04 references.

08. SECURITY EVIDENCE BELONGS THROUGHOUT THE LIFECYCLE

Bridge scene | ID: security-through-change

VISUAL IDEA: The familiar lifecycle loop receives threat questions, permission checks, review, dependency evidence, and incident feedback.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a final scanner can discover an unstated ownership policy. Reveal distinct contributions from requirements, design, implementation checks, review, release integrity, and operation. Use a small selected set of risks rather than reciting a top-ten list. Connect a selected risk to a concrete verification requirement, implementation control, evidence and owner using OWASP ASVS 5.0.0. Contrast the Top 10 awareness list with ASVS verification requirements and SSDF development practices; none is a certificate that this application is secure. Use a few relevant controls rather than reading a checklist. [S51]

CHANGE ONE CONDITION AND COMPARE: Introduce a new delegated-cancellation policy as an explicit hypothetical. Trace which model, checks, review questions, and diagnostic fields must be reconsidered before restoring the baseline.

PRINCIPLE TO REVEAL: Security is maintained through connected decisions and feedback, not added by one tool at the end.

STUDENT PDF TREATMENT: Print the lifecycle evidence map and the policy-change trace.

6. TEACHING GUARDRAILS

OWASP's Top 10 is an awareness document, not a complete application verification standard. Do not equate cryptography with authorization, input validation with all injection prevention, or an SBOM with proof of a secure build. Use the selected SLSA specification to clarify provenance and integrity claims, not to award a maturity level to the fictional system. [S13, S16]

Name laws only as one-line context (KVKK, the GDPR, the Cyber Resilience Act); do not present privacy or supply-chain examples as legal compliance. Avoid collecting real class information for the demonstration. Keep credentials as obvious placeholders. The no-overlap invariant remains enforced at the authoritative boundary even when every requester is authenticated.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

"Logged in means permitted." The owner, action, resource state, and time still matter.

"The interface prevents that request." A trusted service must enforce the rule independently.

"A dependency scan proves the release is secure." It observes a limited set of known issues and does not establish every design or behavioral property.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: May U-02 cancel a future booking owned by U-01 in the baseline?

ANSWER: No. Authentication alone does not satisfy the ownership requirement.

QUESTION 2: Why is a request identifier preferable to a full private message in this diagnostic example?

ANSWER: It can connect relevant observations while avoiding content that is unnecessary for the stated support question.

QUESTION 3: What does a component inventory establish?

ANSWER: Which recorded components are included or expected within its defined scope; it does not by itself establish that they are safe.

QUESTION 4: Should notification failure automatically cancel an already confirmed booking?

ANSWER: No. Under R-04 the booking remains confirmed while notification work is explicitly pending or failed and recoverable.

QUESTION 5: How do a Top 10 risk, an ASVS requirement and an SSDF practice differ?

ANSWER: They describe an awareness concern, a verifiable application control expectation and a development practice respectively; each informs a different decision.

QUESTION 6: A desktop application hides its export button, but its export entry point can still be called. Where does the permission decision belong?

ANSWER: At the trusted boundary that performs the export, using protected identity and data-access rules. Interface visibility guides interaction; it cannot enforce a restriction when another path reaches the operation.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Retain the permission matrix, trust boundaries, and safe-failure comparison. Do not use a lock icon as the only explanation of a control. Print the minimal diagnostic record with fictional-data labels and the dependency evidence limitations. Put dated standards status in the sources page rather than on every conceptual scene.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect threat boundaries, authorization and minimal diagnostics. Replace part of the security-through-change recap with the ASVS mapping. Keep the legal context short and put its dates in the reading copy. Optional depth: examine session lifetime or a secrets revocation path; avoid a cryptography or compliance tour.

10. HANDOFF

Carry the review, verification, permission, and dependency evidence into Week 10. The next question is how a change reaches users while preserving the relationship between what was checked and what is released.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S13, S14, S15, S16, S42, S47, S48, S49, S51. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 10 | AUTOMATED DELIVERY AND EVIDENCE-BASED RELEASE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

What evidence should travel with a change before it reaches users?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-10.html

1. PURPOSE, CASE STATE, AND SCOPE

Animate the movement of a change, but keep attention on the evidence attached to it. A pipeline is not useful simply because boxes turn green. Show what each check observes, which artifact was actually checked, how configuration changes behavior, and what can be recovered when release evidence proves incomplete.

Use current DORA terminology in a compact measurement scene: change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate. Keep these at the service and delivery-system level, not as individual rankings. [S01, S02, S17]

In practice (dated examples, 2026): GitHub Actions and GitLab CI run the gates, container images built with Docker are common release artifacts, Terraform or OpenTofu describe environments as code, and OpenFeature or Unleash control feature flags.

2. OBSERVABLE LEARNING OUTCOMES

1. Explain continuous integration, continuous delivery, and continuous deployment without tying them to a vendor.

2. Identify which evidence a release gate provides and its limitations.

3. Explain why the tested artifact and production artifact must be related unambiguously.

4. Compare deployment, user exposure, rollback, and forward recovery.

5. Interpret a small balanced set of delivery measures with explicit denominators.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare one booking-policy correction and a second change that breaks an authorization check. Give each change, build, and release a stable identifier. Display source revision, dependency inputs, test outcomes, artifact identity, and configuration separately. Use synthetic evidence rather than a real delivery platform.

For the measurement scene, use the declared ten-deployment fixture in the case guide: a seven-day window, two failed deployments requiring intervention, two unplanned corrective deployments, lead times with median 3.5 hours, and recovery durations of 15 and 25 minutes. These are teaching values, not performance benchmarks.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | A release path should make its evidence visible | Anchor

week-10.html#/change-through-gates

02 | Integration, readiness, and deployment are different decisions | Bridge

week-10.html#/continuous-does-not-mean-uncontrolled

03 | Release the thing you checked, with the intended settings | Anchor

week-10.html#/artifact-and-configuration

04 | Installed code and visible behavior can change at different times | Bridge

week-10.html#/deployment-and-exposure

05 | Undoing code does not necessarily undo effects | Anchor

week-10.html#/recover-after-release

06 | Count outcomes without turning one number into the goal | Bridge

week-10.html#/delivery-measures

07 | Passing gates does not end verification | Bridge

week-10.html#/feedback-after-release

08 | Ready means the remaining risk has been considered | Bridge

week-10.html#/release-rationale

5. DETAILED SCENE TEACHING NOTES

01. A RELEASE PATH SHOULD MAKE ITS EVIDENCE VISIBLE

Anchor scene | ID: change-through-gates

VISUAL IDEA: A change token moves through integration, build, static checks, focused checks, broader checks, artifact recording, and release decision; evidence cards remain attached.

SITUATION, PREDICTION, AND MECHANISM: Ask which stage would catch the incorrect 120-minute boundary and which would check ownership. Reveal distinct checks and pause at a failure. The token cannot advance merely because the animation has reached its next step. Label the static checks with their scope: linters, type checkers and security scanners find unused code, type mismatches and known insecure patterns, but generic checks without a specification of R-02 need not reject “duration < 120”; the independent boundary check does. Specialized static analysis can check richer properties when those properties are supplied. Explain that gate placement follows risk and feedback needs.

CHANGE ONE CONDITION AND COMPARE: Fix the boundary but introduce an authorization regression. Replay the same pipeline and compare which evidence changes. Show a passing gate with an explicit scope label so it is not mistaken for a universal guarantee. AI beat: let

the second change be proposed by an automated assistant. It enters the same path and is blocked by the same gate. Proposing a change and passing a gate are different authorities, which Week 13 develops as bounded action.

PRINCIPLE TO REVEAL: Automation should preserve the connection between a claim, a check, its result, and the change being released.

STUDENT PDF TREATMENT: Print the successful path and blocked path with the exact failed condition and artifact identity.

02. INTEGRATION, READINESS, AND DEPLOYMENT ARE DIFFERENT DECISIONS

Bridge scene | ID: continuous-does-not-mean-uncontrolled

VISUAL IDEA: Three lanes distinguish frequent integration, a releasable state, and automatic production deployment.

SITUATION, PREDICTION, AND MECHANISM: Ask whether automatically running tests is the whole meaning of continuous integration. Reveal frequent integration into shared work with useful checks. Contrast a system kept ready for release with one that automatically deploys changes passing the declared gates.

CHANGE ONE CONDITION AND COMPARE: Add an explicit human release decision without changing build and verification automation. Compare continuous delivery with continuous deployment; neither label determines whether the underlying checks are sufficient.

PRINCIPLE TO REVEAL: Describe which decision is automated rather than using “continuous” as a synonym for fast or safe.

STUDENT PDF TREATMENT: Print the three lanes and identify the production-release decision in each.

03. RELEASE THE THING YOU CHECKED, WITH THE INTENDED SETTINGS

Anchor scene | ID: artifact-and-configuration

VISUAL IDEA: A recorded artifact travels through environments while configuration cards remain separate and visible.

SITUATION, PREDICTION, AND MECHANISM: Ask whether rebuilding from the same source later necessarily gives the same result. Reveal a changed resolved dependency. Compare promoting the recorded artifact with silently rebuilding an unverified one. Keep provenance evidence distinct from functional correctness. [S16]

CHANGE ONE CONDITION AND COMPARE: Hold artifact identity fixed and change a configuration value affecting authorization or a dependency endpoint. Compare why environment-specific configuration still needs validation, even when the artifact is unchanged. Secrets remain references, never visible values. Inspect the build identity and permissions as inputs too: a proposed change may run checks without permission to publish or access release secrets. Compare isolated verification with the trusted release path and bind provenance to the artifact digest and expected producer. A signature proves neither correct behavior nor the suitability of every included dependency. [S16]

PRINCIPLE TO REVEAL: Artifact identity controls one source of uncertainty; configuration and environment behavior still need evidence.

STUDENT PDF TREATMENT: Print the source-to-artifact relationship and the two configuration outcomes.

04. INSTALLED CODE AND VISIBLE BEHAVIOR CAN CHANGE AT DIFFERENT TIMES

Bridge scene | ID: deployment-and-exposure

VISUAL IDEA: A release appears in the runtime map while a feature flag and a small cohort control exposure.

SITUATION, PREDICTION, AND MECHANISM: Ask whether deploying code must expose a new behavior to everyone immediately. Reveal a disabled path, then a labeled small-cohort rollout. Make clear that hidden code is still deployed and may carry operational or security implications.

CHANGE ONE CONDITION AND COMPARE: Change cohort size while holding the artifact fixed. Compare observations from a narrow rollout with the uncertainty about other users and workloads. A small cohort is not automatically representative. Attach a responsible owner and retirement condition to the flag. Inspect the recovery decision when the small cohort crosses the stated user-outcome limit; a staged rollout needs a response policy as well as a percentage.

PRINCIPLE TO REVEAL: Separating deployment from exposure can create additional decision points, provided the controls and observations are trustworthy.

STUDENT PDF TREATMENT: Print deployment state, exposure state, and cohort assumptions in a small matrix.

05. UNDOING CODE DOES NOT NECESSARILY UNDO EFFECTS

Anchor scene | ID: recover-after-release

VISUAL IDEA: A deployment timeline sits beside database compatibility and already-sent notifications.

SITUATION, PREDICTION, AND MECHANISM: Ask whether returning to the previous binary restores the previous world. Reveal records written in a new format and messages already delivered. Compare a reversible code change with an irreversible external effect or incompatible schema change.

CHANGE ONE CONDITION AND COMPARE: Use a compatibility-preserving transition that lets the earlier application continue reading data, then compare rollback feasibility. Where reversal is not safe, show a forward correction and explicit reconciliation rather than a magical rewind.

PRINCIPLE TO REVEAL: Recovery must account for state, compatibility, and external effects—not only the application version.

STUDENT PDF TREATMENT: Print the before-and-after state and a recovery-choice table with prerequisites.

06. COUNT OUTCOMES WITHOUT TURNING ONE NUMBER INTO THE GOAL

Bridge scene | ID: delivery-measures

VISUAL IDEA: A deployment timeline feeds five small numerical summaries with the observation window and denominators visible.

SITUATION, PREDICTION, AND MECHANISM: Ask whether twice as many deployments necessarily means better delivery. Reveal the ten-deployment fixture and calculate the measures from the event records. Distinguish a failed deployment from an unplanned deployment that repairs an incident; they contribute to different numerators. [S17]

CHANGE ONE CONDITION AND COMPARE: Split an otherwise identical release into several bookkeeping deployments. Compare the inflated count with unchanged user value and other evidence. Explain why an honest interpretation needs context, stable definitions, and several measures.

PRINCIPLE TO REVEAL: Measures support improvement when their definitions and consequences remain visible.

STUDENT PDF TREATMENT: Print the event table, five results, and explicit limits of the synthetic comparison.

07. PASSING GATES DOES NOT END VERIFICATION

Bridge scene | ID: feedback-after-release

VISUAL IDEA: The release path loops back from user outcomes and service observations to earlier checks and requirements.

SITUATION, PREDICTION, AND MECHANISM: Ask what happens when production traffic differs from the test fixture. Reveal a slow path absent from the earlier workload. Connect the observation to a new representative check and a revised capacity assumption rather than blaming the pipeline for not predicting everything.

CHANGE ONE CONDITION AND COMPARE: Change only the workload mix. Compare why the same release artifact can behave differently and what additional evidence would distinguish code, configuration, and demand effects.

PRINCIPLE TO REVEAL: Production feedback can refine the evidence model without making earlier checks pointless.

STUDENT PDF TREATMENT: Print the release-to-observation-to-improved-check loop.

08. READY MEANS THE REMAINING RISK HAS BEEN CONSIDERED

Bridge scene | ID: release-rationale

VISUAL IDEA: A concise release record combines intended behavior, checked artifact, known limitations, exposure plan, and recovery conditions.

SITUATION, PREDICTION, AND MECHANISM: Ask what the release decision-maker still needs beyond a green build. Reveal the affected users, compatibility constraints, monitoring question, and recovery plan. Keep the record brief enough to read while explaining.

CHANGE ONE CONDITION AND COMPARE: Remove a safe recovery path from the scenario. Compare how that changes the required evidence and exposure decision even when the tests remain unchanged.

PRINCIPLE TO REVEAL: Release readiness is an evidence-based decision under stated risk, not a decorative status badge.

STUDENT PDF TREATMENT: Print the release record with the missing-evidence variant and its effect on the decision.

6. TEACHING GUARDRAILS

Do not imply that a particular automation product defines continuous integration or that every deployment should automatically expose every feature. Avoid using approval as an excuse for absent verification. A stored artifact digest can support identity checks but does not establish useful behavior by itself.

The DORA scene uses all five current measures, but does not blend them into an invented score or prescribe targets. Failed deployment recovery time concerns deployment failures requiring intervention; it is not a generic mean recovery time for every kind of incident. Deployment rework rate counts unplanned incident-driven deployments, not every ordinary follow-up change. [S17]

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Green pipeline means safe release.” The checks have defined scope and can miss environmental or requirement assumptions.

“The same source always builds the same artifact.” Relevant inputs and nondeterminism can differ.

“Rollback restores everything.” Stored state, external effects, and compatibility may prevent that.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: What is the distinction between continuous delivery and continuous deployment here?

ANSWER: A releasable state with an explicit production decision versus automatic production deployment of changes that pass the declared gates.

QUESTION 2: Why keep configuration visible when promoting one artifact?

ANSWER: Configuration can change behavior independently of artifact identity.

QUESTION 3: What is the change fail rate in the ten-deployment fixture?

ANSWER: Two of ten deployments required immediate intervention, so the rate is 20%.

QUESTION 4: Does a notification already delivered disappear after code rollback?

ANSWER: No. External effects need their own reconciliation or compensating behavior.

QUESTION 5: Why can the same artifact require new release evidence under different settings?

ANSWER: Configuration, identity, permissions and dependencies can change behavior even when the artifact digest stays fixed.

QUESTION 6: A command-line release was tested from source, but users download a separately rebuilt package. What connects their result to those checks?

ANSWER: Identify the distributed artifact and its resolved build inputs. Promote the verified package where possible, or verify the rebuilt artifact and the relevant environment; matching source alone leaves other inputs unsettled.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve the failed gate, artifact identity chain, configuration comparison, and recovery prerequisites. The delivery-measure page must include data and units, not only KPI tiles. Print the release rationale with its uncertainty. The student version should not contain credentials, platform screenshots, or authoring controls.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect gates, artifact/configuration identity and state-aware recovery. Keep build permissions and flag retirement within those scenes. Optional depth: environment drift and a self-service deployment path with bounded permissions, without infrastructure-platform installation.

10. HANDOFF

Carry the released artifact and observation questions into Week 11. Now the service is in use, and the viewer must distinguish a user-visible symptom from the evidence needed to explain it.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S16, S17. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 11 | RELIABILITY, OBSERVABILITY, AND PERFORMANCE REASONING

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Which observation would help distinguish the possible causes of a failure?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-11.html

1. PURPOSE, CASE STATE, AND SCOPE

Begin with a user-visible delay, not a dashboard. Establish the service outcome that matters, then reveal measurements, logs, and traces only when they help answer a specific diagnostic question. Keep reliability objectives, performance requirements, and delivery metrics separate: they describe different aspects of the system.

Use selected concepts from service-level objectives and monitoring guidance, with logs, metrics, and traces as the three telemetry forms explored in this lecture. OpenTelemetry supports a broader set of signals; this course deliberately concentrates on these three. [S01, S02, S20, S21, S22]

In practice (dated examples, 2026): OpenTelemetry collects logs, metrics and traces; Prometheus stores metrics and evaluates alert rules; Grafana draws dashboards; Jaeger shows individual traces.

2. OBSERVABLE LEARNING OUTCOMES

1. Define an eligible event, a good event, an indicator, and an objective for a specific service promise.

2. Interpret a latency distribution without relying only on its average.

3. Use correlated observations to distinguish plausible incident causes.

4. Explain how retries can increase load and why recovery controls need conditions.

5. Separate immediate mitigation from later causal learning.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare a fictional seven-day record of 1,000 eligible booking requests. Exactly 980 receive a correct outcome within two seconds; ten are correct but slower, five time out, and five return unexpected server failures. The illustrative objective is 99% good events, so 10 bad events are permitted by that objective and 20 are observed. The 10 permitted bad events are the error budget; observing twice the allowed bad-event rate is a burn rate of 2, which at a steady rate uses up the seven-day budget in 3.5 days. Honest, timely “room unavailable” decisions count as correct outcomes.

Keep the earlier Q-01 search requirement separate: p95 search API response at most 500 milliseconds with 50 concurrent users under its declared fixture. Prepare two fixed latency datasets and one correlated incident packet containing request ID, release marker, trace, queue measurement, and redacted log. None is a real campus incident.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | A successful status code is not the complete promise | Anchor

week-11.html#/define-good-service

02 | The average can hide the experience we need to improve | Bridge

week-11.html#/latency-is-a-distribution

03 | Each signal answers a different part of the question | Anchor

week-11.html#/correlate-the-observations

04 | A plausible story needs a distinguishing observation | Bridge

week-11.html#/hypothesis-before-fix

05 | Trying harder can increase the pressure on a failing dependency | Anchor

week-11.html#/retry-amplification

06 | Restoring a useful service and explaining a cause are different jobs | Bridge

week-11.html#/mitigate-and-communicate

07 | An alert should help someone decide what to do | Bridge

week-11.html#/actionable-alerts

08 | The explanation should improve the system, not find a scapegoat | Bridge

week-11.html#/learn-after-the-incident

5. DETAILED SCENE TEACHING NOTES

01. A SUCCESSFUL STATUS CODE IS NOT THE COMPLETE PROMISE

Anchor scene | ID: define-good-service

VISUAL IDEA: Request tokens enter a good/bad classification table with eligibility, correctness, and timing visible.

SITUATION, PREDICTION, AND MECHANISM: Ask whether every rejected booking request is a service failure. Reveal a timely correct conflict response as a good outcome under this objective. Then classify a fast incorrect confirmation and a correct response arriving too late. The viewer must state the event definition before seeing the percentage. [S21]

CHANGE ONE CONDITION AND COMPARE: Change only the observation: 20 of 1,000 eligible requests miss the promise. Compare 98% observed good events with the illustrative 99% objective. Name the ten-event allowance as the error budget and the twenty observed bad events as a burn rate of 2, without treating the budget as permission to ignore harmful failures. [S45]

PRINCIPLE TO REVEAL: A reliability indicator is meaningful only after the user promise, event population, and observation window are defined.

STUDENT PDF TREATMENT: Print the classifications, arithmetic, window, and objective; retain rejected-but-correct examples.

02. THE AVERAGE CAN HIDE THE EXPERIENCE WE NEED TO IMPROVE

Bridge scene | ID: latency-is-a-distribution

VISUAL IDEA: Two empirical latency distributions share axes and units; mean and nearest-rank p95 markers are shown separately.

SITUATION, PREDICTION, AND MECHANISM: Ask whether equal averages imply equally responsive service. Reveal Dataset A: twenty requests at 200 ms. Dataset B: eighteen at 100 ms and two at 1,100 ms. Both average 200 ms, but their nearest-rank p95 values differ: 200 ms and 1,100 ms.

CHANGE ONE CONDITION AND COMPARE: Change the comparison from the average to the tail, keeping the data fixed. Connect the result to the specific Q-01 fixture without claiming that twenty synthetic points establish real production performance.

PRINCIPLE TO REVEAL: Use a distribution and a defined percentile method when the question concerns uneven user experience.

STUDENT PDF TREATMENT: Print the sorted values or frequency table, both charts, sample sizes, and percentile method.

03. EACH SIGNAL ANSWERS A DIFFERENT PART OF THE QUESTION

Anchor scene | ID: correlate-the-observations

VISUAL IDEA: One request timeline links a latency measurement, structured log entry, and distributed trace; selecting the request highlights matching records.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a spike in aggregate latency identifies the slow component. Reveal a trace with an unusually long storage wait, then a relevant event record. Contrast what the metric suggests, what the trace localizes, and what the log explains. [S20]

CHANGE ONE CONDITION AND COMPARE: Change the cause to delayed notification that is not on the confirmation path. Compare the same user report with the request trace, showing why correlation and a clear interaction model matter.

PRINCIPLE TO REVEAL: Telemetry becomes useful evidence when it connects a symptom to a discriminating question about the system.

STUDENT PDF TREATMENT: Print the correlated packet with a shared request identifier and a caption for what each signal supports.

04. A PLAUSIBLE STORY NEEDS A DISTINGUISHING OBSERVATION

Bridge scene | ID: hypothesis-before-fix

VISUAL IDEA: Three candidate causes sit beside predicted observations and the evidence actually available.

SITUATION, PREDICTION, AND MECHANISM: Offer slow storage, an identity dependency failure, and exhausted request capacity. Ask which observation would separate them. Reveal one trace segment and one queue measurement, then eliminate only the hypotheses contradicted by

that evidence. Distinguish waiting on an external dependency from computation inside a function. A trace helps localize the boundary; a profile of the relevant execution helps locate expensive computation. Do not optimize a visually prominent function without evidence that it limits the observed outcome. Optional performance extension: once storage lookup is the supported hypothesis, compare examining every booking with using an index to locate a relevant subset. Change the amount of stored data while keeping the query and required result fixed. Explain why the work can grow differently and why maintaining an index adds storage and write work. Algorithm and data-structure choices affect the work performed; operation counts alone are not elapsed-time measurements. Ask what representative read/write workload would test the trade-off, and retain the authoritative no-overlap decision. This is a qualitative authored comparison, not a benchmark or a database installation lesson. [S59]

CHANGE ONE CONDITION AND COMPARE: Remove the trace sample from the packet. Compare the weaker conclusion now justified. A missing log entry is not automatically proof that an event never happened; collection may be incomplete.

PRINCIPLE TO REVEAL: Diagnosis advances by testing predictions and acknowledging gaps, not by naming the first familiar cause.

STUDENT PDF TREATMENT: Print the hypothesis-prediction-evidence table and the remaining uncertainty. If the optional lookup comparison is used, retain its fixed query/result, changed data size, index maintenance cost and missing workload evidence; do not attach invented speedup figures.

05. TRYING HARDER CAN INCREASE THE PRESSURE ON A FAILING DEPENDENCY

Anchor scene | ID: retry-amplification

VISUAL IDEA: A controlled queue model shows original requests and separately labeled retry attempts; arrival and completion rates are visible.

SITUATION, PREDICTION, AND MECHANISM: Start with a constrained dependency and ask whether immediate repeated retries will reduce waiting. Reveal extra attempts entering the same limited capacity, with no invented promise about exact real-world timings. Preserve the distinction between unique user operations and repeated attempts.

CHANGE ONE CONDITION AND COMPARE: Change to bounded retries with delay, backoff, and an explicit timeout budget; show a circuit-open state as a deliberate temporary refusal to call the unhealthy dependency. Compare queue pressure and successful outcomes under the declared toy model. Keep jitter optional and seeded for reproducible teaching. Distinguish admission control from retries: bounded queues and load shedding can protect work already admitted when capacity is exhausted. Preserve an honest unavailable result instead of a false confirmation. Backoff with jitter reduces synchronized retries; the deterministic core fixture models bounded retries and a circuit, not a measured jitter benefit. [S55]

PRINCIPLE TO REVEAL: Recovery behavior must account for shared capacity and the possibility that an earlier attempt already had an effect.

STUDENT PDF TREATMENT: Print the baseline and bounded-retry model, assumptions, and reminder of Week 6 idempotency.

06. RESTORING A USEFUL SERVICE AND EXPLAINING A CAUSE ARE DIFFERENT JOBS

Bridge scene | ID: mitigate-and-communicate

VISUAL IDEA: An incident timeline separates detect, assess impact, mitigate, verify recovery, communicate, and investigate.

SITUATION, PREDICTION, AND MECHANISM: Ask whether the team must know the complete cause before reducing harm. Reveal a safe exposure rollback or capacity protection, then show the evidence used to verify improvement. Keep user communication accurate about known impact and uncertainty. Identify the incident coordinator, the operator applying the mitigation and the communicator. Small teams may combine these responsibilities, but an explicit handoff prevents contradictory changes and unsupported status messages.

CHANGE ONE CONDITION AND COMPARE: Make code rollback unsafe because of state compatibility. Compare an alternative mitigation that limits exposure or degrades a nonessential capability without breaking the booking invariant.

PRINCIPLE TO REVEAL: Incident response should reduce harm, preserve evidence, and keep claims proportionate to what is known.

STUDENT PDF TREATMENT: Print the timeline and the reason the selected mitigation was safe in this scenario.

07. AN ALERT SHOULD HELP SOMEONE DECIDE WHAT TO DO

Bridge scene | ID: actionable-alerts

VISUAL IDEA: A noisy stream of low-level alerts is contrasted with a user-impact signal and a short response guide.

SITUATION, PREDICTION, AND MECHANISM: Ask whether every CPU fluctuation should interrupt someone. Reveal the user promise, the affected service, an actionable condition, and a next diagnostic question. Avoid prescribing a universal threshold independent of workload. [S22]

CHANGE ONE CONDITION AND COMPARE: Hold resource utilization high while user outcomes remain acceptable, then lower it while correctness fails. Compare resource observations with the actual service objective. Then show an alert tied to the promise: a sustained burn rate of 2 from the beginning of a fresh seven-day window would use its budget in 3.5 days, under the fixture's steady-traffic assumption. This is not 3.5 days of remaining budget after the displayed seven-day window: that window has already exceeded its allowance. Choose urgency from impact, remaining budget and corroborating observation windows rather than treating the ratio alone as a paging rule. [S45]

PRINCIPLE TO REVEAL: Alerting should connect a meaningful condition to an appropriate response, not merely report that a number changed.

STUDENT PDF TREATMENT: Print the signal-condition-response relationship and the two counterexamples.

08. THE EXPLANATION SHOULD IMPROVE THE SYSTEM, NOT FIND A SCAPEGOAT

Bridge scene | ID: learn-after-the-incident

VISUAL IDEA: A causal timeline connects conditions, decisions, observations, and follow-through checks.

SITUATION, PREDICTION, AND MECHANISM: Ask what “someone made a mistake” fails to explain. Reveal missing safeguards, misleading feedback, or an untested workload assumption. Separate observed facts from hypotheses and identify which change would reduce recurrence or shorten detection.

CHANGE ONE CONDITION AND COMPARE: Add a proposed action that sounds reassuring but cannot be verified. Compare it with a specific change, an owner role, and evidence showing whether the risk was reduced.

PRINCIPLE TO REVEAL: Learning from an incident requires testable improvements to the conditions that shaped it.

STUDENT PDF TREATMENT: Print the factual timeline and improvement-to-verification map without personal blame.

6. TEACHING GUARDRAILS

Do not use availability, correctness, latency, and deployment recovery interchangeably. The illustrative SLO’s denominator includes authenticated, syntactically valid booking requests reaching the defined service boundary, including legitimate room conflicts; it excludes health checks. Define eligibility before observing results, not afterward to improve the score.

Percentiles require a stated convention and enough relevant observations for the claim being made. Use the tiny latency dataset to explain a concept, not to validate Q-01. Logs may contain sensitive material; use synthetic, minimal fields. Correlation can guide diagnosis but does not by itself establish causation.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Every rejection is an outage.” A correct, timely conflict decision can satisfy the stated service promise.

“Average latency describes everyone.” Unequal distributions can share an average.

“Retry until it works.” Repeated attempts can add load and duplicate effects without appropriate limits and identity semantics.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: What is the observed good-event rate for the declared fixture?

ANSWER: 980 divided by 1,000, or 98%.

QUESTION 2: How many bad events does the illustrative 99% objective allow in that window?

ANSWER: Ten of 1,000, which is the error budget; twenty are observed, a burn rate of 2.

QUESTION 3: Which signal helps locate where one request spent time?

ANSWER: A correlated trace can localize time across the observed request path, within its sampling and instrumentation limits.

QUESTION 4: Why might a high CPU reading not justify an urgent incident alert by itself?

ANSWER: Its significance depends on user impact, capacity risk, and an actionable response, not the number alone.

QUESTION 5: Does the historical burn-rate illustration tell us how much budget remains now?

ANSWER: No. Remaining budget depends on the current window and observations; the 3.5-day calculation assumes a fresh window and a sustained rate.

QUESTION 6: A batch report finishes quickly but silently omits valid input records. Is elapsed time enough to call the run successful?

ANSWER: No. Define success using correct, sufficiently complete output and the relevant deadline or latency condition. Preserve rejected-input reasons and trace missing records; fast completion alone misses the user promise.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Print the SLO classification rules and arithmetic, the two latency distributions, and the full correlated incident packet. Each chart needs axes, units, sample size, and synthetic-data labeling. Preserve the hypothesis table and recovery limitations so the student can follow the diagnosis without the animation.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect outcome definition, correlated diagnosis and retry amplification. Keep profiling and overload policy tied to the same incident. Optional depth: choose the lookup/index trade-off in scene 04 or a different observation window for alerting with its traffic assumptions stated. Each can be omitted without affecting later weeks. Avoid adding a dashboard or algorithm catalogue.

10. HANDOFF

Carry the incident’s learning and the compatibility limitation into Week 12. The system now needs to change over time while people continue to rely on its existing behavior.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S20, S21, S22, S45, S55, S59. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 12 | MAINTENANCE, EVOLUTION, AND SUSTAINABLE CHANGE

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

How can a useful system change without losing the behavior people rely on?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-12.html

1. PURPOSE, CASE STATE, AND SCOPE

Make maintenance visible as engineering under accumulated knowledge and constraints, not merely fixing old code. Reuse the stable booking service, then introduce a small data-model change and show why rolling releases, older consumers, stored records, and support knowledge all matter.

Connect maintainability with the ability to respond to necessary change. Include resource efficiency as a separate sustainability concern: compare useful work under a declared boundary, rather than equating reduced CPU time with a measured carbon reduction. [S01, S02, S23, S24, S37]

In practice (dated examples, 2026): Renovate and Dependabot propose dependency updates as reviewable changes, Flyway and Liquibase apply database migrations one step at a time, and feature flags keep old and new paths switchable during a transition.

2. OBSERVABLE LEARNING OUTCOMES

1. Distinguish observed existing behavior from intended required behavior.

2. Explain a compatibility-preserving data transition and its recovery conditions.

3. Identify the future change cost and risk represented by a specific debt item.

4. Prioritize maintenance using evidence about impact, uncertainty, and reversibility.

5. Compare resource use per successful task without overstating environmental conclusions.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare an older booking response containing durationMinutes and a revised representation containing start and end. Both describe the same campus-local same-day interval. Show old and new readers and records in a compatibility matrix. Do not introduce time-zone migration implicitly.

Prepare one behavior-preserving refactoring, one bug fix changing behavior, and a synthetic resource fixture: 100 successful searches requiring 1,000 storage reads before a change and 200 reads afterward, under the same correctness and freshness conditions. The fixture measures reads per successful search, not energy or emissions.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Existing behavior is evidence, not automatically the requirement | Bridge

week-12.html#/understand-before-changing

02 | A rewrite and a refactoring carry different uncertainties | Anchor

week-12.html#/safe-small-change

03 | Old and new versions may need to coexist | Anchor

week-12.html#/expand-migrate-contract

04 | Name the future cost, not just the disliked code | Bridge

week-12.html#/technical-debt-with-interest

05 | The loudest complaint is not always the most important change | Anchor

week-12.html#/prioritize-by-consequence

06 | Removing something is also a compatibility decision | Bridge

week-12.html#/dependency-and-interface-retirement

07 | Doing less work is useful only if the promise still holds | Bridge

week-12.html#/resource-per-useful-outcome

08 | The next maintainer needs the reason and the current state | Bridge

week-12.html#/evolution-record

5. DETAILED SCENE TEACHING NOTES

01. EXISTING BEHAVIOR IS EVIDENCE, NOT AUTOMATICALLY THE REQUIREMENT

Bridge scene | ID: understand-before-changing

VISUAL IDEA: A legacy function, observed examples, and the canonical rule appear in separate columns.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a characterization check should preserve everything the current implementation does. Reveal the actual behavior first, including a defect accepting a 121-minute booking. Distinguish documenting what exists from endorsing it.

CHANGE ONE CONDITION AND COMPARE: Change the upper-bound logic deliberately as a bug fix while preserving unrelated behavior. Compare a corrected expectation derived from R-02 with blindly freezing the defect.

PRINCIPLE TO REVEAL: Understand current behavior before changing it, then decide explicitly which behavior should remain.

STUDENT PDF TREATMENT: Print observed-versus-required examples and mark the intentional correction.

02. A REWRITE AND A REFACTORING CARRY DIFFERENT UNCERTAINTIES

Anchor scene | ID: safe-small-change

VISUAL IDEA: A change-impact graph and behavior checks accompany incremental structural changes; a full replacement appears as a comparison.

SITUATION, PREDICTION, AND MECHANISM: Ask what evidence survives when the Policy responsibility is extracted behind an interface. Reveal unchanged externally observable behavior under relevant checks and a smaller change surface. Define refactoring by preserved behavior, not by the number of edited lines.

CHANGE ONE CONDITION AND COMPARE: Replace the whole booking path at once while keeping the intended feature set fixed. Compare the larger set of assumptions needing re-establishment. A rewrite can be justified, but the decision needs reasons, transition strategy, and evidence—not contempt for old code.

PRINCIPLE TO REVEAL: Choose a change strategy by its uncertainty, constraints, and evidence needs, not by whether the code is new.

STUDENT PDF TREATMENT: Print the two change surfaces and the behavior each strategy must re-establish.

03. OLD AND NEW VERSIONS MAY NEED TO COEXIST

Anchor scene | ID: expand-migrate-contract

VISUAL IDEA: A three-stage data transition and compatibility matrix show old/new applications reading old/new representations.

SITUATION, PREDICTION, AND MECHANISM: Ask whether renaming a field instantly is safe while old consumers remain active. Reveal an expanded representation supporting both forms, a controlled migration, and checks that readers agree on the same duration. Show that mixed versions are a normal transition state to consider. [S23]

CHANGE ONE CONDITION AND COMPARE: Remove the old representation before the old reader is retired. Compare the failure with a transition that verifies migration and consumer readiness before contraction. Show why rollback possibilities narrow after the old data path is removed. Before a destructive step, distinguish having a backup from having evidence that it restores usable, consistent data. Compare the recovery point with later writes and inspect how long restoration makes the service unavailable. Introduce acceptable data loss and restoration time as requirements; do not invent recovery targets. A backup does not make already-delivered notifications reversible. [S56]

PRINCIPLE TO REVEAL: Evolution is safer when compatibility is maintained deliberately across the states that actually coexist.

STUDENT PDF TREATMENT: Print each transition stage, the reader-writer matrix, and the condition required before contraction.

04. NAME THE FUTURE COST, NOT JUST THE DISLIKED CODE

Bridge scene | ID: technical-debt-with-interest

VISUAL IDEA: A debt card links a shortcut to repeated change effort, risk, and a possible repayment path.

SITUATION, PREDICTION, AND MECHANISM: Ask whether every defect or untidy function is technical debt. Reveal a duplicated duration rule that makes each policy update require coordinated edits and checks. Describe the specific extra future work and failure risk rather than

attaching a vague debt label.

CHANGE ONE CONDITION AND COMPARE: Change the expected policy-change frequency from rare to frequent. Compare the rationale for repaying this item now, while acknowledging that estimates are uncertain and other risks may dominate.

PRINCIPLE TO REVEAL: A useful debt discussion identifies the deferred cost, the condition that triggers it, and the evidence supporting action.

STUDENT PDF TREATMENT: Print the debt record with its assumptions and revisit trigger rather than an invented monetary total.

05. THE LOUDEST COMPLAINT IS NOT ALWAYS THE MOST IMPORTANT CHANGE

Anchor scene | ID: prioritize-by-consequence

VISUAL IDEA: A small portfolio of maintenance needs is compared by user impact, exposure, change frequency, uncertainty, and reversibility.

SITUATION, PREDICTION, AND MECHANISM: Offer a failing authorization edge case, an annoying display inconsistency, a fragile dependency update path, and a slow infrequent report. Ask what information is needed to prioritize them. Reveal consequences and constraints one at a time without reducing ethics or security to a decorative score.

CHANGE ONE CONDITION AND COMPARE: Change only the exposure of the authorization flaw or the dependency's support status in the fictional scenario. Compare how the priority rationale changes. Avoid making a real unsupported vulnerability claim about an actual library.

PRINCIPLE TO REVEAL: Maintenance priority should respond to consequences and evidence, with assumptions open to revision.

STUDENT PDF TREATMENT: Print the alternatives and a reasoned ordering under each labeled condition.

06. REMOVING SOMETHING IS ALSO A COMPATIBILITY DECISION

Bridge scene | ID: dependency-and-interface-retirement

VISUAL IDEA: Usage evidence, deprecation communication, transition support, and removal conditions attach to an old interface.

SITUATION, PREDICTION, AND MECHANISM: Ask whether announcing a removal date proves consumers are ready. Reveal a remaining old client and an incomplete observation window. Connect the removal decision to actual usage, support constraints, and the earlier compatibility matrix.

CHANGE ONE CONDITION AND COMPARE: Provide verified consumer migration and a tested fallback, then compare the evidence for retiring the old path. Discuss records and diagnostics retained only for a stated purpose, with policy questions left explicit rather than invented as law.

PRINCIPLE TO REVEAL: Retirement requires evidence about remaining dependence and an intentional treatment of data and recovery.

STUDENT PDF TREATMENT: Print the readiness conditions and remaining unknowns for the old interface.

07. DOING LESS WORK IS USEFUL ONLY IF THE PROMISE STILL HOLDS

Bridge scene | ID: resource-per-useful-outcome

VISUAL IDEA: Two bars show storage reads per successful search, with correctness and freshness conditions fixed beside them.

SITUATION, PREDICTION, AND MECHANISM: Ask whether fewer reads is automatically an improvement. Reveal 1,000 reads for 100 successful searches versus 200 for 100: 10 and 2 reads per successful search. Verify that both return the required current availability and preserve the authoritative booking decision. Explain the reduction through avoiding repeated reads or using a cache only within a declared freshness contract. Measure useful outcomes before claiming a performance or resource benefit; correctness and the authoritative confirmation check still apply.

CHANGE ONE CONDITION AND COMPARE: Reduce reads further by serving stale results beyond the allowed freshness assumption. Compare the apparently attractive resource number with the violated user promise. Explain that a carbon claim additionally needs an appropriate energy, emissions, hardware, and system-boundary method. [S37]

PRINCIPLE TO REVEAL: Compare resource use per useful outcome under equivalent requirements, and label proxies honestly.

STUDENT PDF TREATMENT: Print counts, denominators, freshness assumptions, and the explicit statement that reads are not measured emissions.

08. THE NEXT MAINTAINER NEEDS THE REASON AND THE CURRENT STATE

Bridge scene | ID: evolution-record

VISUAL IDEA: A compact evolution record links changed requirement, migration stage, checks, rollout evidence, and remaining retirement conditions.

SITUATION, PREDICTION, AND MECHANISM: Ask what would let a new maintainer continue the transition safely. Reveal which representations exist, which readers remain, how correctness is checked, and when rollback ceases to be safe. Keep the record synchronized with actual state rather than the original intention.

CHANGE ONE CONDITION AND COMPARE: Change one migration condition and update the record. Compare an obsolete confident diagram with an accurate record of the partial transition. AI beat: a repository instruction file written for automated assistants is documentation with the same obligation. If it still describes the pre-migration state, a new maintainer and an assistant will both act on stale facts.

PRINCIPLE TO REVEAL: Maintainability includes preserving enough shared understanding to continue a change responsibly.

STUDENT PDF TREATMENT: Print the final transition record with completed, pending, and uncertain conditions distinguished in text.

6. TEACHING GUARDRAILS

Do not call a change behavior-preserving when it changes required outputs or permissions. Characterization captures observations but does not decide which observations are desirable. A data transition needs actual compatibility checks; the words “expand-migrate-contract” are not a

substitute for them. [S23]

Treat technical-debt estimates as contextual, not an objective universal currency. Resource reductions do not automatically imply equal carbon reductions. The SCI reference provides a broader accounting method; this lecture uses a clearly labeled resource proxy and does not claim to calculate an SCI score. [S37]

7. MISCONCEPTION AND DIAGNOSIS CLINIC

“Old code should always be replaced.” Age alone does not establish change risk or value.

“A characterization test makes the old behavior correct.” It records behavior; requirements still decide what should happen.

“Fewer database reads prove lower emissions.” The proxy omits energy, hardware, carbon intensity, and system-boundary information.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: When may the old representation be removed safely in the depicted transition?

ANSWER: After relevant consumers and stored data have migrated, compatibility evidence is sufficient, and recovery implications are understood.

QUESTION 2: Is fixing an incorrect boundary a pure refactoring?

ANSWER: No. It intentionally changes externally observable behavior to satisfy the rule.

QUESTION 3: What makes a debt item actionable?

ANSWER: A specific deferred cost or risk, the conditions that cause it, a possible change, and evidence for its priority.

QUESTION 4: What is the resource reduction in the declared fixture?

ANSWER: Reads per successful search decrease from 10 to 2, an 80% reduction under the same stated conditions; this is not an emissions measurement.

QUESTION 5: Why does a successful backup job not settle migration recovery?

ANSWER: The data must be restorable and consistent, later writes must be accounted for, and the restoration interruption must meet the relevant requirement.

QUESTION 6: A desktop application's update changes its document format. Does reinstalling the old application recover the documents?

ANSWER: Only if that version can read the actual saved data or a verified conversion or restore path exists. Account for edits since any backup and communicate the recovery consequences before changing the format.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve migration stages and the compatibility matrix in readable separate panels. Print the distinction between observed and intended behavior, the contextual debt record, and the resource calculation. Do not collapse the transition to a single “new system” picture that hides coexistence or

recovery limits.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect the small-change comparison, coexistence migration and consequence-based prioritization. Put restore evidence at the migration decision. Optional depth: revisit cache freshness in the existing resource comparison; no new storage-platform tour.

10. HANDOFF

Carry the requirement, checks, change rationale, and maintenance constraints into Week 13. The next lecture asks whether faster code generation changes the evidence needed to accept a change.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S23, S24, S37, S56. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 13 | AI-ASSISTED ENGINEERING AND ACCOUNTABLE AUTOMATION

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Does plausible-looking output satisfy the intended behavior, and who can verify that?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-13.html

1. PURPOSE, CASE STATE, AND SCOPE

Apply the course's established engineering discipline to AI-assisted change. Use instructor-prepared candidate code, a requirement, an independent check, and a bounded action trace. There is no live assistant interface, product comparison, prompt-writing demonstration, account login, or external model call in the student lecture.

The emphasis is neither automatic trust nor automatic rejection. A generated candidate can be useful while still requiring the same behavioral, security, integration, and maintenance evidence as any other change. Current DORA research is useful context for discussing whole-workflow effects and verification effort, but it should not be converted into a universal productivity multiplier. [S01, S02, S18, S19]

Name the working vocabulary students will meet in 2026 as each mechanism appears: specification-driven development, where the precise rule is the input to drafting; repository instruction files that carry standing rules for assistants; tool permissions and sandboxes; evaluation of output that is not deterministic; cost and latency budgets for automated steps; and the shift of the engineer's work toward specifying and checking. Scene 07 uses a fictional change record as its example.

In practice (dated examples, 2026): coding assistants include OpenAI Codex, Claude Code, GitHub Copilot and Cursor, and they reach tools through connectors such as MCP servers. Name them factually, without comparison or ranking.

2. OBSERVABLE LEARNING OUTCOMES

1. Evaluate a generated candidate against explicit requirements and independent checks.

2. Explain why agreement between generated code and generated tests can be weak evidence.

3. Identify permissions and approval boundaries appropriate to an automated engineering action.

4. Recognize untrusted content that attempts to redirect an automated workflow.

5. Distinguish increased output from improved verified delivery and maintainable understanding.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare a short candidate implementing the duration rule with an exclusive 120-minute upper bound. Prepare a second candidate that trusts an owner identifier supplied by a request. Both are fictional teaching snippets, not claims about the behavior of any particular model or vendor. Include corrected versions and independently derived checks.

Prepare a bounded action diagram with read, edit, local-check, and propose-change stages. Production access and secrets are outside its allowed scope. A repository note containing a misleading instruction is displayed as untrusted input, never executed. Reuse the Week 4 review-capacity model for the output-versus-flow comparison. For scene 07, prepare the compact change record of the corrected cancellation change, which takes the owner from the signed-in user instead of the request. The record lists the requirement it implements, that a coding assistant drafted it, the checks that ran, the reviewer's acceptance and one unresolved limitation. It names no product.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Readable code can implement the wrong rule | Anchor

week-13.html#/plausible-candidate

02 | Two agreeing outputs can share one mistaken premise | Anchor

week-13.html#/shared-assumption

03 | Missing context can become an invisible design decision | Bridge

week-13.html#/context-and-unknowns

04 | A useful assistant does not need every possible permission | Anchor

week-13.html#/bounded-actions

05 | A file being read is not automatically allowed to command the reader | Bridge

week-13.html#/untrusted-instructions

06 | Faster drafting can move the constraint somewhere else | Bridge

week-13.html#/output-and-flow

07 | Someone must be able to explain the accepted change | Bridge

week-13.html#/provenance-and-understanding

08 | Using AI to build software is not the same as building an AI-dependent service | Bridge

week-13.html#/two-different-engineering-questions

5. DETAILED SCENE TEACHING NOTES

01. READABLE CODE CAN IMPLEMENT THE WRONG RULE

Anchor scene | ID: plausible-candidate

VISUAL IDEA: A polished candidate function sits beside R-02; input cards reveal behavior while the explanation text remains visible.

SITUATION, PREDICTION, AND MECHANISM: Ask whether the confident explanation and tidy structure justify acceptance. Reveal the 120-minute case rejected by “duration < 120.” Compare the prose claim with the actual observable result. Do not imply that only AI-assisted code contains this kind of defect.

CHANGE ONE CONDITION AND COMPARE: Correct the boundary while keeping formatting and explanation style unchanged. Replay 120 and 121, then compare how independent evidence—not appearance or origin—changes the judgment.

PRINCIPLE TO REVEAL: Accept a change for the behavior and evidence it establishes, not for how fluently it is presented.

STUDENT PDF TREATMENT: Print the requirement, candidate, counterexample, correction, and independently expected outcomes.

02. TWO AGREEING OUTPUTS CAN SHARE ONE MISTAKEN PREMISE

Anchor scene | ID: shared-assumption

VISUAL IDEA: A candidate and its accompanying tests both depend on an incorrect interpretation card; an independent requirement-derived check enters from another direction.

SITUATION, PREDICTION, AND MECHANISM: Ask whether generated tests passing the generated implementation settle the issue. Reveal that both exclude the 120-minute boundary. Trace their shared assumption and distinguish agreement from independence.

CHANGE ONE CONDITION AND COMPARE: Add a check derived directly from the canonical inclusive rule and a separate ownership matrix. Compare the evidence when the evaluator is not simply reproducing the candidate's logic. Multiple tools or agents are not automatically independent if they share the same mistaken specification.

PRINCIPLE TO REVEAL: Independent verification needs a justified basis for expected behavior, not merely a second source of confident text.

STUDENT PDF TREATMENT: Print the assumption graph and the independent counterexample with its derivation.

03. MISSING CONTEXT CAN BECOME AN INVISIBLE DESIGN DECISION

Bridge scene | ID: context-and-unknowns

VISUAL IDEA: A compact change brief links relevant rules, interfaces, constraints, and explicitly unresolved questions to the candidate.

SITUATION, PREDICTION, AND MECHANISM: Ask how a candidate would know whether a staff role may cancel any booking. Reveal that the baseline grants no such override. Compare a candidate inventing permission with one that leaves an explicit question for the responsible decision-maker. Use a compact task specification containing the intended outcome, relevant files, invariants, non-goals, available checks and a stop condition. Useful bounded work includes explaining unfamiliar code, proposing boundary examples and drafting a small refactoring. Inspect one accepted change against its independent evidence so the lecture shows productive use as well as failure analysis.

CHANGE ONE CONDITION AND COMPARE: Change the brief to an approved, narrowly defined delegation policy. Compare the code and checks that legitimately change with those that should remain fixed. Restore R-03 after the variant.

PRINCIPLE TO REVEAL: Automation should not silently convert an unknown policy into authority.

STUDENT PDF TREATMENT: Print the baseline, labeled policy variant, and the evidence required for the changed permission.

04. A USEFUL ASSISTANT DOES NOT NEED EVERY POSSIBLE PERMISSION

Anchor scene | ID: bounded-actions

VISUAL IDEA: An action path passes through explicit read, edit, verify, propose, and approve boundaries; available capabilities are displayed beside it.

SITUATION, PREDICTION, AND MECHANISM: Ask whether generating a patch requires access to production booking data or release credentials. Reveal a narrower path using synthetic fixtures and local checks. Show that proposing a change and applying a high-impact change are different authorities; the permission list and the sandbox that enforce this live outside the assistant. [S33] Add a row for an unreviewed tool connector, for example an MCP server: it is third-party code acting with the assistant's permissions, so it is both a dependency and a trust boundary. OWASP's list for agents names the risks it creates, tool misuse and identity and privilege abuse. [S44]

CHANGE ONE CONDITION AND COMPARE: Give the same imperfect candidate broad unattended production access, then compare the possible consequences with the bounded version. Keep it a diagram, not a functioning dangerous workflow. An approval button needs informative evidence to be meaningful.

PRINCIPLE TO REVEAL: Limit capability, permission, and autonomy to the intended action, and enforce those limits outside the model’s judgment.

STUDENT PDF TREATMENT: Print the capability matrix and the approval boundary with the evidence available at that point.

05. A FILE BEING READ IS NOT AUTOMATICALLY ALLOWED TO COMMAND THE READER

Bridge scene | ID: untrusted-instructions

VISUAL IDEA: A repository note is tagged as content; a separate trusted task and permission boundary remain visible.

SITUATION, PREDICTION, AND MECHANISM: Show a note claiming that the verification step should be skipped. Ask whether its location in a repository makes it authoritative. Reveal the attempt to cross from task data into workflow instruction. Introduce prompt injection as a trust-boundary problem. [S34] Name the agent-level result as OWASP's agentic list does: goal hijack, content redirecting what the agent is trying to achieve. [S44]

CHANGE ONE CONDITION AND COMPARE: Keep the note unchanged but enforce the external requirement that checks cannot be bypassed by retrieved content. Compare the attempted redirection with the allowed action path. State that text filtering alone is not a complete guarantee.

PRINCIPLE TO REVEAL: Treat retrieved material as untrusted input and preserve authority boundaries around consequential actions.

STUDENT PDF TREATMENT: Print the content-versus-authority diagram and the enforced check, without operational attack details.

06. FASTER DRAFTING CAN MOVE THE CONSTRAINT SOMEWHERE ELSE

Bridge scene | ID: output-and-flow

VISUAL IDEA: The Week 4 flow board shows candidate changes, verification capacity, accepted changes, and rework separately.

SITUATION, PREDICTION, AND MECHANISM: Ask whether doubling candidate output doubles useful delivery. Reveal a fixed review constraint and the resulting queue in the declared toy model. Distinguish generated lines, accepted changes, time to verified behavior, and downstream defects. Connect the flow to specification-driven drafting: the precise rule from Week 2 makes verification better specified; whether it saves total time must be observed across drafting, review, correction and downstream work.

CHANGE ONE CONDITION AND COMPARE: Improve context and verification capacity rather than only generation rate. Compare whole-flow outcomes under the same arrival conditions. Use current research as context, while identifying this chart as a constructed mechanism illustration rather than measured industry results. [S19] Compare a survey, a controlled task study and this simulation as different kinds of evidence. METR's February 2026 follow-up reports selection and time-measurement problems; its May 2026 survey concerns self-reported impact. Neither establishes a universal productivity multiplier. The DORA AI Capabilities Model provides organizational context without prescribing a tool ranking. [S53, S54, S57]

PRINCIPLE TO REVEAL: Evaluate automation across the workflow it changes, including verification, rework, and maintainable understanding.

STUDENT PDF TREATMENT: Print the model assumptions, before-and-after flow, and the limits of the numerical example.

07. SOMEONE MUST BE ABLE TO EXPLAIN THE ACCEPTED CHANGE

Bridge scene | ID: provenance-and-understanding

VISUAL IDEA: A change record collects source context, reviewed dependencies, checks, residual questions, and a plain-language explanation.

SITUATION, PREDICTION, AND MECHANISM: Ask whether “the tool wrote it” answers a maintainer’s question about an authorization branch. Reveal the rationale and relevant evidence required to maintain the change. Any uncertain origin or licensing concern is marked for appropriate review rather than guessed away. Include one dependency question: the candidate imports textenc-turbo, a name marked unavailable in the fictional registry fixture; this is not a claim about every real public registry. Code assistants are documented to invent package names, and anyone could publish a package under the invented name, so the inventory check blocks the build until a person confirms or removes the dependency. [S50] Use the fictional record: the cancellation change was drafted by a coding assistant, checked by the requirement tests and an independent check, and accepted by a named reviewer, with its unresolved limitation stated.

CHANGE ONE CONDITION AND COMPARE: Replace a confident but unsupported explanation with one that accurately states a limitation. Compare whether the reviewer can make a better decision, even though the wording sounds less certain.

PRINCIPLE TO REVEAL: Accountability requires an understandable change record and honest limits, regardless of how the draft was produced.

STUDENT PDF TREATMENT: Print the change record and its unresolved question.

08. USING AI TO BUILD SOFTWARE IS NOT THE SAME AS BUILDING AN AI-DEPENDENT SERVICE

Bridge scene | ID: two-different-engineering-questions

VISUAL IDEA: Two system boundaries contrast an offline-generated code change with a runtime model inside the service.

SITUATION, PREDICTION, AND MECHANISM: Ask whether Campus Rooms needs an AI dependency because an assistant helped author some code. Reveal that the shipped behavior can remain ordinary deterministic software. Contrast this with adding a runtime model whose outputs and dependencies become part of the user-facing system.

CHANGE ONE CONDITION AND COMPARE: Insert a hypothetical model that suggests room choices, while leaving authoritative booking and permissions outside it. Compare the additional evaluation and failure questions without expanding this course into model training. Name two of them: an evaluation set of room-choice cases with expected answers, and a regression check that reruns it whenever the prompt or the model version changes, because the output is not deterministic. NIST's AI profile concerns AI-model and AI-system development and should not be mislabeled as a blanket coding-assistant rulebook. [S35] For the recommendation extension, compare against a simple deterministic baseline on held-out scenarios. Use task-appropriate relevance, unsupported-claim, permission, latency and cost criteria, allowing more than one acceptable suggestion. Record model, prompt, retrieval data and evaluation-set versions; repeat observations where variability matters and define fallback behavior. A pass on a fixed evaluation set does not make an action authorized.

PRINCIPLE TO REVEAL: The engineering obligations follow the actual system boundary and consequences, not the presence of AI somewhere in the workflow.

STUDENT PDF TREATMENT: Print both boundaries and the distinct claims each would require.

6. TEACHING GUARDRAILS

Do not make the lecture a contest between brands or use a live external model whose output may change during teaching. Preserve the prepared evidence and display its fictional nature. Do not imply that generated code is inherently untrustworthy while human code deserves no checks; apply the same acceptance discipline.

Research findings about reported AI use or productivity are contextual, often observational or survey-based, and do not prove a causal improvement for every learner or team. Keep current research in dated notes and avoid unverified percentages. Agent safeguards need external permission enforcement, useful review evidence, and appropriate isolation; a prompt alone is not a security boundary. [S18, S19, S33, S34]

7. MISCONCEPTION AND DIAGNOSIS CLINIC

"Fluent explanation means correct implementation." Compare the explanation with the boundary result.

"A second generated answer is independent verification." It may inherit the same wrong premise or expected-result logic.

“Human approval solves everything.” Approval without understandable evidence and restricted authority can become a meaningless click.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: What reveals the first candidate’s defect?

ANSWER: An independently expected 120-minute valid booking is rejected by its exclusive upper bound.

QUESTION 2: Why not grant production access merely to prepare a patch?

ANSWER: It is unnecessary for the intended local action and increases possible consequences if the action is mistaken or redirected.

QUESTION 3: What should happen when a repository note tells the workflow to skip required checks?

ANSWER: Treat it as untrusted content; it must not override the authoritative task or externally enforced verification boundary.

QUESTION 4: What distinguishes using an assistant during development from a runtime AI feature?

ANSWER: In the first case the accepted artifact can operate without a model; in the second the model is part of the deployed behavior and introduces its own evaluation and dependency questions.

QUESTION 5: What evidence is needed before claiming that an assistant improved engineering productivity?

ANSWER: Relevant comparable work, explicit measurement boundaries, accepted behavior, review and rework costs, and limitations such as task selection or self-report bias.

QUESTION 6: An assistant proposes a repository-wide rename with a script that can write anywhere on disk. What authority does that task need?

ANSWER: Limit writes to the intended checkout, inspect the proposed changes and run relevant checks before acceptance. Repository text must not expand that permission; independent scope enforcement and recoverable history bound a useful automated change.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

Preserve the exact candidate, failed boundary, shared-assumption graph, and capability matrix. The static version must make the independent expected result visible. Include the trust-boundary explanation and the difference between development assistance and runtime AI. No assistant interface or external service is required to understand the student PDF.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect candidate evaluation, independent expectations and externally enforced permissions. Reuse familiar boundary and queue cases briefly so context, useful delegation and evaluation receive depth. Optional depth: a runtime recommendation evaluation; omit multi-agent orchestration and model architecture catalogues.

10. HANDOFF

Carry the full requirement-to-evidence discipline into Week 14. The final lecture brings the same system through one new operational need and asks which connected decisions must change together.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S18, S19, S33, S34, S35, S44, S50, S53, S54, S57. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

WEEK 14 | CONNECTING THE LIFECYCLE THROUGH ONE ENGINEERING DECISION

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

Can we explain a change from human need to operational evidence without losing the original promise?

Instructor lecture plan | flexible pacing | 8 scenes: 3 Anchors + 5 Bridges | planned HTML: weeks/week-14.html

1. PURPOSE, CASE STATE, AND SCOPE

End with an instructor-led engineering decision clinic, not a list of all previous definitions. A staff member reports that C101 must be closed for maintenance during a future interval. The viewer follows one bounded change across requirements, models, architecture, checks, release, operation, and learning.

Use familiar representations and identifiers so the difficulty lies in connecting decisions, not decoding a new domain. Preserve explicit disagreement where evidence does not determine one universal design. The goal is a coherent explanation of what should change, what must remain true, and what would justify the decision. [S01, S02]

In practice (dated examples, 2026): the closure change travels through the tools named in earlier weeks, an issue in GitHub Issues or Jira, a reviewed pull request, CI gates in GitHub Actions, a feature flag for exposure, and OpenTelemetry signals after release.

2. OBSERVABLE LEARNING OUTCOMES

1. Trace a new user need through a requirement, model, design decision, verification, and operational observation.

2. Identify which established invariants remain unchanged during a new feature decision.

3. Compare two change strategies under the same constraints and evidence.

4. Explain a release or recovery decision with its assumptions and consequences.

5. State a general engineering principle using a concrete counterexample from the course.

3. INSTRUCTOR PREPARATION AND ENTRY CHECK

Prepare a maintenance-closure request, a future room calendar containing one confirmed booking, the current architecture and requirement map, and two candidate approaches. Proposed Week 14 extension R-06: once a closure is effective, new confirmations overlapping its room and interval are rejected; existing bookings are retained, visibly marked as affected, and handled through an explicit resolution process. Do not silently grant staff cancellation authority or claim that the room remains usable.

For the teaching scenario, accept this policy after the opening clarification. Owner cancellation remains governed by R-03. Notification status remains separate under R-04. Prepare one release with a missing interface cue and one dependency-failure variant. The extension is a labeled addition, not a retroactive change to earlier fixtures.

4. ORDERED CHAPTER AND PAGE MAP

Opening situation and recall/outcomes come first. The eight scenes below follow in this order. Principles/checks and sources/handoff close the deck. The normal twelve-page starting layout may expand where readability requires it.

01 | Close the room—but what happens to the people already relying on it? | Anchor

week-14.html#/new-need-old-promises

02 | Each representation should answer the next unanswered question | Bridge

week-14.html#/trace-the-change

03 | The interface can explain the closure; the decision boundary must enforce it | Bridge

week-14.html#/locate-authority

04 | One new rule needs more than one friendly example | Bridge

week-14.html#/verification-weave

05 | What would make this change ready to expose? | Anchor

week-14.html#/release-with-a-fault

06 | Two reasonable designs can optimize different constraints | Anchor

week-14.html#/compare-defensible-options

07 | Name the principle after remembering the counterexample | Bridge

week-14.html#/course-principles

08 | Finish with a claim the evidence can actually support | Bridge

week-14.html#/engineering-explanation

5. DETAILED SCENE TEACHING NOTES

01. CLOSE THE ROOM—BUT WHAT HAPPENS TO THE PEOPLE ALREADY RELYING ON IT?

Anchor scene | ID: new-need-old-promises

VISUAL IDEA: A maintenance notice overlays a future calendar, with an existing confirmed booking and a new request shown separately.

SITUATION, PREDICTION, AND MECHANISM: Ask what “disable C101” leaves ambiguous. Reveal the closure interval, who may record a closure, what new requests should see, and how existing booking holders learn that action is required. Separate preventing new promises from resolving promises already made.

CHANGE ONE CONDITION AND COMPARE: Compare silent deletion of the existing booking with preserving its record and making its affected status explicit. Ask which outcome supports honest communication, traceability, and authorized resolution. Accept R-06 as the scenario’s stated policy only after showing these choices.

PRINCIPLE TO REVEAL: A new requirement must address the consequences for existing users, not merely the next successful action.

STUDENT PDF TREATMENT: Print the ambiguous request, accepted extension, and treatment of new versus existing bookings.

02. EACH REPRESENTATION SHOULD ANSWER THE NEXT UNANSWERED QUESTION

Bridge scene | ID: trace-the-change

VISUAL IDEA: A linked strip changes from requirement card to state annotation, decision table, dependency graph, and verification matrix.

SITUATION, PREDICTION, AND MECHANISM: Ask which representation would show who is permitted to record a closure and which would show request ordering. Reveal the appropriate model for each question. Connect R-06 to the existing R-01 through R-05 without inventing a one-diagram solution.

CHANGE ONE CONDITION AND COMPARE: Change only the closure interval so it ends exactly when a candidate booking begins. Compare the boundary expectation with the half-open interval convention already used throughout the course.

PRINCIPLE TO REVEAL: Traceability is useful when it carries meaning across representations, not when it merely adds connecting lines.

STUDENT PDF TREATMENT: Print the linked trace and the adjacency example with the relevant rule labels.

03. THE INTERFACE CAN EXPLAIN THE CLOSURE; THE DECISION BOUNDARY MUST ENFORCE IT

Bridge scene | ID: locate-authority

VISUAL IDEA: Two alternatives show closure checking only in the browser or at the authoritative booking decision with an informative interface.

SITUATION, PREDICTION, AND MECHANISM: Ask whether a hidden room card is sufficient to prevent a new confirmation. Reveal stale client state and another valid request path. Reuse Week 9’s trust boundary and Week 6’s concurrency lesson.

CHANGE ONE CONDITION AND COMPARE: Introduce a closure while a booking request is in progress. Compare an interface-only check with an authoritative decision that defines ordering between closure and confirmation. Do not pretend the exact storage mechanism is settled by drawing a lock icon.

PRINCIPLE TO REVEAL: Shared promises need an enforcement point that survives alternate paths and concurrent change.

STUDENT PDF TREATMENT: Print both alternatives, the interleaving, and the decision-ordering requirement.

04. ONE NEW RULE NEEDS MORE THAN ONE FRIENDLY EXAMPLE

Bridge scene | ID: verification-weave

VISUAL IDEA: A traceability matrix links R-06 to interval boundaries, permission decisions, concurrent behavior, interface states, and notification failure.

SITUATION, PREDICTION, AND MECHANISM: Ask which checks would distinguish a plausible incomplete implementation from the intended behavior. Reveal a nonoverlap case, a closure-boundary case, a forbidden staff action, an in-flight booking, and an affected-owner message.

CHANGE ONE CONDITION AND COMPARE: Make all function checks pass while omitting the affected-booking interface cue. Compare the evidence with a keyboard-accessible user journey that reveals the missing information. [S25]

PRINCIPLE TO REVEAL: Build evidence around the risks and user promises that cross layers, not just the easiest function to test.

STUDENT PDF TREATMENT: Print the check matrix with expected outcomes and a clear gap in the incomplete version.

05. WHAT WOULD MAKE THIS CHANGE READY TO EXPOSE?

Anchor scene | ID: release-with-a-fault

VISUAL IDEA: A compact release path combines the verified artifact, compatibility state, selected exposure, and user-outcome observations.

SITUATION, PREDICTION, AND MECHANISM: Ask whether passing interval and permission checks justify full exposure when existing users cannot see their affected status. Reveal the incomplete release record and keep the decision paused. Add the missing accessible cue and the evidence that it is observable. Before the fix, change only the release pressure: a sponsor now asks to proceed despite the known missing cue. Identify who bears the consequence, explain the concern, propose a safer release or deferral and escalate the unresolved decision to its responsible owner. Professional judgment includes affected people beyond the sponsor; approval does not remove a known user harm. This is an application of responsibility and harm reduction, not a new booking permission. [S58]

CHANGE ONE CONDITION AND COMPARE: Introduce notification failure after closure is recorded. Compare a system that loses the affected-user task with one that records pending communication and continues to display the affected status. Preserve recovery responsibility without undoing the closure or deleting booking history. Carry a compact decision record with the responsible owner, the permitted action, the independent result, an observation after exposure and the recovery condition. Let one missing item narrow the release claim, then reveal exactly the evidence that resolves it.

PRINCIPLE TO REVEAL: Release decisions must consider the whole user promise and the behavior of incomplete or failed work.

STUDENT PDF TREATMENT: Print the blocked decision, corrected evidence, and notification-failure outcome.

06. TWO REASONABLE DESIGNS CAN OPTIMIZE DIFFERENT CONSTRAINTS

Anchor scene | ID: compare-defensible-options

VISUAL IDEA: A side-by-side decision record compares a targeted change inside the modular application with a separately deployed closure service.

SITUATION, PREDICTION, AND MECHANISM: Ask which new requirement actually needs independent deployment. Reveal benefits and costs under the fictional small-team, single-campus baseline. Compare change surface, operational boundaries, authority, and maintenance burden without using an arbitrary winner score. Include continuing operation, support effort and exit or migration cost in the qualitative comparison. Make the displaced work visible and retain the option to defer the distributed extension when its benefit is not established.

CHANGE ONE CONDITION AND COMPARE: Change one condition: several independent campus operators now need separate release schedules. Revisit the rationale and identify the new contract and ownership questions. Do not announce that this single condition automatically proves the distributed option is best.

PRINCIPLE TO REVEAL: A defensible choice explains its assumptions, trade-offs, evidence, and conditions for reconsideration.

STUDENT PDF TREATMENT: Print both decision records and the one changed condition, preserving unresolved questions.

07. NAME THE PRINCIPLE AFTER REMEMBERING THE COUNTEREXAMPLE

Bridge scene | ID: course-principles

VISUAL IDEA: Six concise principle cards connect to small images from earlier scenes rather than a wall of vocabulary.

SITUATION, PREDICTION, AND MECHANISM: Ask which counterexample taught each principle: false confirmation, ambiguous requirement, clean-but-wrong merge, shared test assumption, unsafe rollback, and overpowered automation. Reveal the principle only after the viewer recognizes the situation.

CHANGE ONE CONDITION AND COMPARE: Change the domain label from room booking to a generic reservation service while keeping the underlying relationship. Compare which principles transfer and which policies must be elicited again.

PRINCIPLE TO REVEAL: Conceptual understanding transfers relationships and reasoning—not an unexamined copy of another system's rules.

STUDENT PDF TREATMENT: Print each principle beside its concrete counterexample and scope.

08. FINISH WITH A CLAIM THE EVIDENCE CAN ACTUALLY SUPPORT

Bridge scene | ID: engineering-explanation

VISUAL IDEA: A final one-page decision narrative links need, rule, design, checks, release, observation, and unresolved uncertainty.

SITUATION, PREDICTION, AND MECHANISM: Ask the viewer to explain why the selected closure behavior is justified. Reveal a coherent account using the evidence already shown, with explicit uncertainty about the multi-campus extension. Avoid ending with "everything is solved."

CHANGE ONE CONDITION AND COMPARE: Remove one important check or change one assumption. Compare how the final claim must narrow. Close on the habit of revising confidence when the supporting conditions change. End with a fresh transfer question: which reasoning

survives in another reservation domain, and which duration, fairness, identity and harm assumptions must be elicited again? Keep uncertainty as a concrete next question with an owner, not a generic disclaimer.

PRINCIPLE TO REVEAL: Good engineering explanations connect decisions to evidence and remain honest about what has not been established.

STUDENT PDF TREATMENT: Print the complete decision narrative and its bounded conclusion as the closing student reference.

6. TEACHING GUARDRAILS

Do not add a large new business context or introduce advanced distributed algorithms in the final lecture. The closure extension must remain clearly distinguished from the earlier baseline. Existing confirmations affected by closure are not silently canceled, and staff closure authority is not a general permission to cancel someone else's booking.

Keep the final alternatives comparable: same initial requirements, same workload, same evaluation criteria. When the condition changes, label it and recompute or reconsider only the implications it actually affects. The conclusion should not imply full security, accessibility, or reliability certification from this teaching example.

7. MISCONCEPTION AND DIAGNOSIS CLINIC

"A final diagram means the design is finished forever." It records a decision under current assumptions.

"New requirements replace old guarantees automatically." Identify which promises remain and explicitly resolve conflicts.

"There is always one best architecture." A justified choice depends on context, evidence, and trade-offs.

8. IN-LECTURE CHECKS WITH ANSWERS

QUESTION 1: Why retain the existing affected booking rather than silently deleting it?

ANSWER: Its owner already relied on the promise; explicit affected status, history, and an authorized resolution preserve information needed to address the consequence.

QUESTION 2: Where must closure and booking ordering be enforced?

ANSWER: At the authoritative decision boundary with a mechanism that preserves the stated concurrent behavior.

QUESTION 3: What is missing when function checks pass but users cannot perceive an affected booking?

ANSWER: Evidence that the complete user-facing requirement, including accessible status information, has been satisfied.

QUESTION 4: What should happen to the final engineering claim when an assumption changes?

ANSWER: Reconsider the affected decision and narrow or revise the claim until appropriate evidence is available.

QUESTION 5: Does a sponsor's approval resolve the known absence of accessible affected-booking information?

ANSWER: No. The user consequence remains. Explain who is affected, offer a safer alternative and escalate the unresolved decision rather than treating approval as a repair.

QUESTION 6: For a device-configuration update, which parts of this course's release reasoning transfer, and what must be learned again?

ANSWER: Trace the need to behavior, model failure, verify the candidate, control authority and plan recovery. Elicit the device's interruption behavior, safe operating state, update constraints and consequences of failure; room-booking policies and web-service rollout assumptions do not transfer automatically.

9. PDF CONTINUITY AND LECTURE ELASTICITY

WHAT THE PRINTED EXPLANATION MUST PRESERVE

The final student PDF is a compact connected explanation of the scenario. Preserve R-06, the treatment of existing bookings, the traceability matrix, release evidence, alternative decisions, and the concluding principles. It should be readable without earlier animations, while referring to stable scene identifiers for students returning to the HTML decks.

CORE AND OPTIONAL DEPTH

Close with the answered transfer question in Section 8, using its small contrasting setting. Preserve the question and explanation in the reading copy.

Protect existing-user consequences, release evidence and the architecture trade-off. Compress the principle recap into retrieval of earlier counterexamples. Optional depth: compare another non-reservation context; do not introduce a new framework in the finale.

10. HANDOFF

The course closes with a reusable reasoning pattern: identify the promise, make assumptions visible, choose a representation, inspect a mechanism, challenge it with a changed condition, and justify the conclusion with evidence.

11. SOURCES AND CURRENCY NOTES

Primary references: S01, S02, S25, S58. Full titles, URLs, dated status, and access limitations appear in the source register. Curriculum review: 22 September 2026. See each source entry for its actual retrieval date; unchanged references retain earlier dates. The examples and visual teaching sequences are original proposals; no live service behavior is implied.

CURRICULUM CURRENCY, COVERAGE, AND BOUNDARIES

Curriculum review: 22 September 2026; source retrieval dates are recorded individually

STABLE FOUNDATION; DATED CONTEMPORARY LAYER

Use durable concepts as the course spine and date the examples, terminology, and supporting references. SWEBOK V4.0a is the identified current IEEE Computer Society guide; CS2023 is the joint ACM/IEEE-CS/AAAI curriculum reference. These support scope selection rather than a claim that fourteen lectures cover every knowledge area or satisfy a formal accreditation mapping. The SWEBOK landing page was accessible through indexed official excerpts, not an unrestricted full-guide audit. [S01, S02]

The original instructional scenarios, questions, diagrams, fixtures, and weekly sequence are authored for this course. They are not reproduced from these references. The reference list supports definitions and technical guardrails; it does not imply that a source endorses every pedagogical choice.

CURRENT MATERIAL INCORPORATED

Security: use OWASP Top 10:2025, including the expanded attention to software supply chains and exceptional-condition handling. Teach the underlying risks, not the order of a memorized list. NIST SSDF 1.1 remains the final baseline used here; the retrieved revision proposing SSDF 1.2 is explicitly an initial public draft dated 17 December 2025. Its closed comment period does not make it final. [S13, S14, S15]

Delivery: the current DORA guide uses five measures. The course uses the current names and distinguishes failed deployment recovery time from a generic all-incident recovery measure. It includes deployment rework rate rather than presenting the historical four-key formulation as unchanged. [S17]

AI assistance: incorporate independent verification, shared-assumption risk, bounded actions, untrusted instructions, review capacity, and accountable maintenance. Use the 2025 DORA report and the March 2026 analysis as dated research context, not universal causal claims or fixed model rankings. Distinguish assisting development from placing a model inside the shipped service. [S18, S19, S33, S34, S35] From this revision the through-line is stated in Week 1 and carried by one short beat inside an existing scene in Weeks 2, 7, 8, 9, 10 and 12, so Week 13 deepens a familiar claim instead of opening a new topic. Agents that call tools and act with delegated permissions have their own OWASP risk list, published 9 December 2025, which Week 13 uses beside the LLM list; a dependency name invented by a code assistant is a documented supply-chain risk and appears in Week 13's change record. [S44, S50]

Reliability: name the error budget and its burn rate with the definitions of the Site Reliability Workbook, so the Week 11 objective connects to the alerting practice students will meet. [S45]

Quality vocabulary: ISO/IEC 25010:2023 names the product quality characteristics behind the quality attributes of Weeks 1 and 2, including its renamed interaction capability and flexibility and the added safety characteristic. [S46]

Law as context: Week 9 names KVKK (Law No. 6698), the GDPR and the EU Cyber Resilience Act as jurisdiction-specific examples connecting engineering choices with applicable duties, without assuming the viewer's location or the system's legal scope. CRA reporting starts on 11 September 2026; its main obligations apply from 11 December 2027. The lecture distinguishes those phases and leaves scope and exemptions explicit. The course explains engineering and gives no compliance advice. [S47, S48, S49]

Tools: each week's Section 1 closes with an "In practice" paragraph naming current tools as dated examples. The names help students connect concepts to everyday work; no scene depends on them.

Workflow and accessibility: use the November 2020 Scrum Guide and May 2025 Kanban Guide editions identified in the register. Make accessible interaction and clear status communication recur throughout, with WCAG 2.2 as the cited accessibility reference. [S04, S05, S25]

COVERAGE ACROSS THE FOURTEEN WEEKS

Weeks 1–3 establish engineering scope, stakeholders, professional consequences, verifiable requirements, quality attributes, traceability, validation, state, data, interaction models, and accessible feedback.

Weeks 4–6 connect work organization, team accountabilities, blameless process feedback, estimation, risk, completion criteria, cohesion, coupling, contracts, refactoring, architecture views, deployment boundaries, concurrency, retries, and compatibility.

Weeks 7–9 address shared history, integration, review etiquette, debugging by bisection, configuration and dependency inputs, documentation, test design, debugging a failing case, test scope, independent expectations, properties, mutation, controlled fixtures, trust boundaries, authorization, privacy, supply-chain evidence, and secure failure behavior. Week 9 also carries a licensing beat on the transitive dependency, using SPDX identifiers as vocabulary. [S42]

Weeks 10–12 follow verified changes into delivery, static analysis and its scope, artifact identity, configuration, exposure, recovery, service-level objectives, error budgets and burn-rate alerting, telemetry, diagnosis, latency, incident learning, maintenance, migrations, debt, retirement, and honest resource-efficiency measures.

Weeks 13–14 apply those ideas to AI-assisted change and an integrated new requirement, reinforcing that the engineering evidence does not disappear when drafting becomes easier.

Quality assurance is distributed across the sequence rather than confined to Week 8. Configuration management spans Weeks 7, 10, and 12. Security, accessibility, ethics, documentation, and uncertainty appear before and after their most concentrated scenes.

FOUNDATION COVERAGE AND DEPTH

This is an instructor planning map to the eighteen SWEBOK knowledge-area headings, using the official indexed topic outline. It is an original mapping of this course, not an IEEE assessment of it. Worked coverage means a decision is explained through a changed condition and an answered question; selected foundations means only the concepts needed here are introduced. A topic appearing in this map is not a claim of complete knowledge-area coverage. [S01, S02]

Software Requirements: worked in Weeks 2-3 and revisited in Week 14 through elicitation, observable acceptance, stakeholder validation and traceability. The request/observation/prototype comparison shows how an intended solution can conceal a different user need.

Software Architecture: worked in Weeks 6 and 14 through deployment boundaries, quality trade-offs, concurrency and uncertain outcomes. Multi-region operation and consensus remain further study.

Software Design: worked in Weeks 3, 5 and 6 through models, responsibilities, interfaces and change propagation. A pattern catalogue is outside the core.

Software Construction: selected foundations in Weeks 5, 7 and 8 through readable pseudocode, validation, explicit results, controlled effects, integration and debugging. This is not a first programming course; explain the short examples as they appear.

Software Testing: worked in Week 8 and revisited through requirements, contracts, delivery and AI candidate evaluation. Boundary cases, independent expectations and test scope are the central decisions. A guided exploratory comparison shows how investigation discovers an expectation for a later regression check; usability research asks a separate question about people completing the task. [S60]

Software Engineering Operations: worked in Weeks 10-12 through release, telemetry, incidents, overload and usable restoration. Infrastructure administration is outside the core.

Software Maintenance: worked in Week 12 and applied in Week 14 through understanding existing behavior, compatible evolution, debt and retirement.

Software Configuration Management: worked across Weeks 7, 10 and 12 through history, resolved dependencies, configuration, artifact identity and version coexistence.

Software Engineering Management: selected decisions in Weeks 4, 7, 11 and 14 cover scope, capacity, risk, responsibility, communication and release. Organizational staffing and contract administration are further study.

Software Engineering Process: worked in Weeks 1, 4 and 10 through feedback, integration, completion policies and improvement experiments. Method names serve those decisions.

Software Engineering Models and Methods: worked in Weeks 3, 6 and 8 through state, data, interactions, invariants and counterexamples. Bounded exploration is optional; advanced formal verification is further study.

Software Quality: worked throughout, especially Weeks 1-2, 8 and 10-12, through explicit promises, preventive practices and evidence about actual behavior. Quality is broader than passing tests.

Software Security: worked in Week 9 and reinforced in Weeks 2, 6, 10 and 13 through trust boundaries, permission, privacy, dependencies and bounded automation.

Software Engineering Professional Practice: selected decisions across Weeks 1, 7, 9, 11, 13 and 14 address affected people, collaboration, honest communication, licensing context and accountable release.

Software Engineering Economics: selected decisions in Weeks 4-5, 12 and 14 cover lifecycle cost, opportunity cost, acquisition and exit. Formal financial modeling is further study.

Computing Foundations: selected concepts in Weeks 3, 5-6 and 11 connect identity, data integrity, program behavior, network boundaries and concurrency. The lookup/index comparison is optional; full algorithms, operating systems, networking and database courses remain further study.

Mathematical Foundations: selected concepts in Weeks 2-4, 6, 8 and 10-11 include conditions, interval boundaries, graphs, state transitions, rates and distributions. Explain each in place; advanced probability, calculus and proof techniques are not prerequisites.

Engineering Foundations: worked across Weeks 2, 4-6, 11 and 14 through assumptions, prototypes, controlled comparisons, measurement, trade-offs and feedback. A simulation illustrates its declared model; it is not an empirical result about a real team.

Cross-cutting threads: accessibility and usability recur in Weeks 1-3, 8-9 and 14; resource efficiency in Week 12; AI assistance in the existing through-line and Week 13. The answered transfer examples extend these decisions beyond the reservation setting. These threads enrich the relevant scenes rather than creating additional weeks.

SEPTEMBER 22 REVIEW: DEPTH, TRANSFER, AND WORKLOAD

The original plan already covers requirements, modeling, process, modularity, distributed failure, review, testing, security, delivery, operation, maintenance and AI assistance. Retain that sequence and the Campus Rooms example. The review strengthens thin mechanisms inside the existing scenes rather than adding a fifteenth week or a new technology survey.

Construction becomes explicit in Week 5 through validation, a deterministic policy and explicit results. Economics becomes explicit in Weeks 4, 5, 12 and 14 through lifecycle cost, opportunity cost, reuse/acquisition, maintenance and exit decisions. Week 3 adds the invariant-versus-progress distinction; optional bounded exploration connects it to Week 8 without claiming formal proof. These changes address breadth of engineering reasoning, not just currently popular tools. [S01, S02]

Week 6 makes an HTTP contract concrete and follows durable notification work across crashes. Week 7 supplies the missing working-tree-to-shared-history bridge and evidence-based disagreement resolution. Week 8 connects controlled doubles with actual contract evidence. Security verification gains a small ASVS mapping; delivery gains an explicit build/release authority distinction. Week 11 connects profiling and overload decisions to user outcomes. Week 12 asks whether a backup actually restores consistent, usable state. [S51, S52, S55, S56]

Week 13 includes productive bounded delegation and compares types of productivity evidence. Its runtime-AI extension names a deterministic baseline, held-out scenarios, variability, relevant evaluation criteria, versioned inputs and fallback. The newer METR material is interpreted within its study limits, and only the public DORA capabilities overview is used. Tool names remain examples rather than the organizing structure. [S53, S54, S57]

Depth is managed in each week's Section 9. The three Anchor scenes remain the core; optional depth is clearly separate and never required by the next week. New content replaces part of a repeated definition, recap or tool-name tour. A week should leave the learner able to explain the decision and the decisive evidence, including a changed condition. An unanswered prediction followed by a reasoned reveal has more educational value than another dense page. Production timing and narration are determined from the actual deck in the tutorial workspace.

Use the general-audience and flexible-pacing decisions in 00_START_HERE.txt. The delivered first lecture remains the starting point; no new prerequisite or recording revision is introduced. The course offers a strong introductory foundation with selected deeper connections, not every specialist topic in software engineering or professional mastery from viewing alone.

TRANSFER BEYOND THE RECURRING CASE

Each remaining lecture closes with an answered transfer question. The examples include a desktop editor, mobile synchronization, an image library, a batch importer, a local file converter, a command-line tool and a device configuration. They change the setting while keeping the week's decision small enough to explain immediately. These are original qualitative scenarios, stored in the transfer_checks fixture block, not measurements or a second implementation track. The room-booking rules do not become defaults for these other systems.

Week 14 also makes professional responsibility actionable: identify who bears a foreseeable consequence, explain the concern honestly, propose a safer alternative and escalate an unresolved release decision. The ACM ethics reference supports consideration of affected people and harm; the course's release scenario and response are original applications. [S58]

WHY THIS ORDER IS DELIBERATE

Requirements precede model and design choices so students have something concrete to judge. State and interaction models precede concurrent failure so “unknown result” has meaning. Modularity precedes architecture so distribution is not confused with good separation. Review precedes deeper testing so students understand why checks must accompany changes. Security precedes delivery so release evidence includes permission and dependency concerns. Operation precedes maintenance so evolution responds to actual feedback. The concentrated AI-assistance week appears after students can evaluate ordinary changes independently; earlier beats establish the same expectations throughout.

Foundational topics are introduced early and revisited with a stronger question. For example, no-overlap begins as a promise, becomes a requirement, appears in a sequence, becomes a concurrent invariant, receives tests, and returns in the closure decision. This is intentional reinforcement, not fourteen unrelated mini-courses.

DELIBERATE LIMITS

Do not stretch these lectures into exhaustive UML, a design-pattern catalog, a full distributed-systems course, database-administration training, a penetration-testing course, or a vendor certification. Advanced formal verification, consensus algorithms, model training, infrastructure orchestration, and detailed environmental accounting remain outside the core sequence.

An optional illustration may acknowledge one of those areas when it clarifies a boundary, but it must not become prerequisite knowledge for the next week. Laws appear only as named context, never as compliance guidance. The course should produce a coherent conceptual foundation and evidence-based reasoning, not a claim of complete professional mastery from lecture exposure alone.

Worked examples and questions with their answers follow the public lecture-only decision in 00_START_HERE.txt.

REFRESH POLICY FOR FUTURE RELEASES

Week 7 authoring, 28 October 2026: rechecked S01, S02, S06 and S07; S01 returned HTTP 403 and keeps its last recorded status. Added primary references S61 (Git bisection), S62 (npm package-lock.json) and S63 (uv lockfile); these additions support Week 7. The In practice examples remain factual, dated examples and require no accounts or installations. No curriculum mechanism changed.

Before a new run, check the official source status for SWEBOK, curriculum guidance, OWASP (including the agentic list), NIST, workflow guides, accessibility guidance, ISO/IEC 25010, the delivery-measure definitions, and the application dates of the laws named in Week 9. Record edition, publication or update date, retrieval date, and final-versus-draft status. Review each week's "In practice" tool names at the same time and replace names that have fallen out of common use. Preserve the conceptual lesson unless the evidence gives a reason to change it. The DORA publication index is the entry point for newer delivery and AI-assistance research; the re-verification task template is ../tasks/50_source_reverification.md. Recheck volatile references again immediately before authoring their lecture; do not silently update the dated research claims. [S36]

During visual authoring, verify supported stable versions of the selected local libraries and a supported Node.js LTS release if Node is used. Pin the tested versions for the course release instead of silently following floating “latest” dependencies. Recheck the affected scenes and PDF output after a dependency update. Sources are dated evidence, not a promise that their live pages can never change. [S26–S32, S38, S39]

ANALYSIS OF THE SUPPLIED BLENDER PLAN

Research and curriculum date: 16 September 2026

WHAT WAS EXAMINED

The supplied ZIP contains fourteen weekly TXT plans, an orientation file, supporting audit and instructor guides, a source register, an asset, and backup copies of several files. The analysis used the current week1.txt through week14.txt files rather than treating .bak files as additional course weeks. It examined the shared structure, weekly progression, chapter maps, and representative detailed teaching flows.

The reference is built around the recurring Lumen Field Station scene. This course's overall duration target and flexible pacing follow the user's decision in 00_START_HERE.txt. Its eleven-section weekly format links purpose, outcomes, preparation, chapter order, detailed teaching notes, guardrails, diagnosis, answered checks, supplementary depth, continuity, and sources. This analysis concerns its instructional structure. It does not independently validate the Blender release or feature claims inside the supplied material.

STRENGTHS RETAINED

A persistent example reduces the need to explain a new context every week. The new course retains that advantage through Campus Rooms while allowing the same behavior to be viewed through requirements, states, dependencies, tests, and operations.

Observable outcomes remain at the front of every week. The instructor receives both a concise ordered map and enough detail to understand the explanations. Predictable errors and misconceptions are prepared in advance rather than discovered by accident during teaching. The explicit handoff keeps later lectures connected to earlier decisions.

The source register and currency notes remain separate from student-facing explanation. This allows an instructor to verify terminology and technical details without turning the lecture into a citation tour. Core explanations and optional depth remain separate, so an explanation can expand or contract according to its teaching needs.

WHAT NEEDED TRANSFORMATION

The Blender plan naturally emphasizes manipulating a software interface to produce an observable result. Software engineering needs a different unit of explanation: a decision and its consequences. A click sequence therefore becomes a controlled causal scene, such as changing a rule and observing which components and checks must change.

The cumulative artifact in the reference becomes a cumulative case with stable rules and evidence. Students can revisit the same room-booking promise through different representations without needing to complete a prior technical construction before following the next lecture.

Technical recipes become semantic guardrails and authoring intent. Exact SVG coordinates, animation APIs, component architecture, and library-specific code are deliberately left to the authoring stage. The plan specifies the claim, visible mechanism, comparison, and static interpretation—not every implementation detail.

The weekly supplementary section becomes PDF continuity and lecture elasticity. Closing questions are in-lecture prediction and reasoning checks, with answers supplied for the instructor. The package concentrates on the lecture experience and its student reference version.

ELEVEN-SECTION CORRESPONDENCE

1. Purpose and scope remains purpose, case state, and scope. 2. Observable learning outcomes remains observable learning outcomes. 3. Instructor preparation remains prepared fixtures, recalled knowledge, and entry prompt. 4. Chapter map becomes the ordered eight-scene map with stable identifiers. 5. Detailed teaching notes becomes visual, prediction, mechanism, changed condition, principle, and PDF treatment. 6. Technical guidance becomes conceptual and simulation guardrails. 7. Troubleshooting becomes a misconception and diagnosis bank. 8. Answered closing checks remains in-lecture checks with answers. 9. Supplementary depth becomes PDF continuity and pacing flexibility. 10. Weekly handoff remains the next question and shared case state. 11. Sources remains dated source keys and explicit access/status notes.

ADDITIONAL IMPROVEMENTS FOR THIS MEDIUM

Separate page navigation from scene state so a clicker does not accidentally reveal the answer or skip a mechanism. Define reset, replay, direct links, and re-entry once for the entire site. Include a print interpretation within every scene rather than postponing it until export.

Limit each week to three substantial interactions. The remaining scenes can be matrices, simple annotated comparisons, model galleries, or short reveals. This preserves time for prediction and explanation and avoids building 112 elaborate simulations merely because the medium permits it.

Use deterministic fictional fixtures and an explicit case contract. The same boundary must remain inclusive in a requirement table, test scene, AI example, and printed caption. This protects the coherence that a polished animation can otherwise hide.

Not transferred: the Blender package's assessment rubric, milestones, recommended practice and student actions. This course is lecture-only and its public-facing materials carry no assessment or administration content; the decision is recorded in 00_START_HERE.txt.

SHARED VISUAL LECTURE AND PDF BRIEF

Research and curriculum date: 16 September 2026

THE EXPERIENCE TO BUILD

Build a reusable local lecture website with a course index and exactly one student HTML deck per week. The index should show week number, central question, concise topic scope, and links to the corresponding deck and student PDF. Within each deck, provide clear page position and a scene overview. The instructor should be able to move around the lecture without losing the meaning or state of the demonstration.

Use a coherent editorial style: generous spacing, large readable labels, a restrained palette, consistent diagram grammar, and a clear hierarchy between question, evidence, and principle. Most screens should feel like an explanation under examination, not a document pasted into presentation software. Keep instructor notes outside the student reading surface and do not expose the authoring assistant's interface.

Decisions in force (00_START_HERE.txt) shape the shell: English only; dark theme by default with a light print stylesheet; Python-like pseudocode for every snippet; the viewer is addressed as "you"; no assessment or administration content; every week is recorded, so the shell has a recording mode. Student-facing wording follows the channel style rules in F:/0_tutorial_videos_project/docs/channel_style_guide.md (public audience and useful content; instructional, not defensive). The plan's instructor prose is not bound by them; the deck text and the narration are.

RECOMMENDED PAGINATION, NOT A RIGID SLIDE QUOTA

Start from twelve pages per week: an opening situation; recall and outcomes; the eight ordered scenes; a principles-and-checks page; and a sources-and-handoff page. This yields separate navigation for every meaningful scene without turning each animation beat into another slide.

Allow roughly twelve to sixteen pages when a dense matrix, paired sequence, or readable code comparison genuinely needs separation. Preserve the primary scene identifier and give continuations stable suffixes. Do not squeeze a diagram to unreadable size to satisfy a number. The PDF may need additional pages for selected animation states and is not required to have the same page count as the HTML deck.

Treat the scene titles in the plan as instructor labels. Student headings should usually express a question, surprising situation, or claim under examination. Reveal the formal term after its mechanism becomes understandable.

CHOOSE THE REPRESENTATION FOR THE QUESTION

Use custom SVG for dependencies, commit graphs, state diagrams, sequence diagrams, highlighted paths, and controlled comparisons. Use semantic HTML tables for requirements, traceability, permissions, compatibility, and check coverage when a table is the clearest representation. Use plain readable code snippets only when examining actual behavior.

Use Chart.js selectively for real numerical comparisons such as queue lengths, latency distributions, and resource counts. Give every quantitative chart units, a baseline, an observation window where relevant, and an accessible data table. Canvas content does not acquire useful text alternatives automatically. [S29, S30]

Three.js is optional. It is appropriate only when spatial relationships actually improve the explanation; a rotating 3D architecture graph is not intrinsically better than a labeled 2D dependency map. Every 3D or WebGL scene needs a clear static 2D interpretation for printing and low-capability viewing. [S31]

Modern browser animation with CSS, JavaScript, and the Web Animations API is sufficient for many controlled scenes. The authoring agent may choose a small additional local library when it solves a genuine need. Prefer the smallest understandable solution over a framework-heavy application shell. [S38]

A STABLE CONTROL CONTRACT

Previous slide and Next slide always change the lecture page. They never mean “reveal the next animation beat.” Next slide is still allowed when a scene is unfinished, so an instructor can deliberately skip ahead.

Step advances one meaningful state within the current scene. Back step returns to the previous scene state. At the final state, Step becomes disabled or clearly finished; it never silently advances to another slide. A continuous Run/Pause control is optional only for scenes such as queue flow, and the initial state is paused.

Replay returns to the first step under the currently selected condition or comparison preset. Reset restores the canonical baseline, all default conditions, and the first paused step. These actions must not be visually or semantically interchangeable.

Default re-entry behavior is deterministic: returning to a scene restores its canonical baseline, paused. A direct hash link does the same. Any optional resume behavior must be a clearly chosen mode, not hidden persistence. Leaving a scene cancels its timers and animation work so nothing continues changing offscreen.

Changing a condition returns that scenario to its first paused state. Keep the comparison baseline visible or make it easy to restore. Display the selected condition in text so the viewer knows exactly what changed.

SCENE CONTRACT FOR TESTS, RECORDING, AND PRINT

The attribute names, query modes and the window.lecture interface that implement this control contract are defined once in repo_kit/docs/scene-contract.md (docs/scene-contract.md in the repository). The tests in tests/test_deck_contract.py, the PDF export script and the recording agents read those names; change them there first. The query modes are record=1 for deterministic timing without autoplay, motion=off for instant states, and print=1 for the print view, which declares html[data-print-ready="true"] when its panels have rendered. Every control and every element a pointer may operate carries a pointer-target attribute; the deck declares no cue lists or narration (22_recording_track.txt).

NAVIGATION AND ACCESSIBILITY

Use stable names such as weeks/week-05.html#/dependency-example. Prefer a single horizontal lecture sequence with an optional overview rather than deep nested navigation. Reveal.js supplies useful navigation, numbering, progress, and hash-link capabilities, but its default fragments can share the next-navigation action. Custom scene state therefore needs an explicitly separate control path rather than assuming fragments automatically meet this brief. [S26, S27]

All controls must work by keyboard and have meaningful accessible names, visible focus, and adequate target sizes. Do not capture arrow keys while a user is adjusting a slider or input. Do not intercept Space so broadly that native buttons stop working. Any single-letter shortcuts need a suitable focus boundary, a disable/remap option, or another conforming approach; document the actual mappings after implementation. [S25]

Motion should clarify causality. Offer reduced-motion and instant-state options that preserve the same information. Never depend on color alone: use text, shape, line style, or labels. Ensure comparisons remain meaningful when printed without color. A low-motion route is a first-class explanation, not an empty scene.

Lay out at 1920 by 1080 logical pixels and render correctly at device scale factor 2; the recording captures the resulting native frame under the capture_scale rule of F:/0_tutorial_videos_project/docs/tutorial_layout_policy.json. Use the whole canvas. Nothing is reserved for a presenter; the avatar overlay yields to content as the avatar block of that policy defines. Where a composition allows, leave the bottom-right corner empty so the overlay can appear more often.

LOCAL OPERATION AND VERSION CHOICES

Use the latest suitable stable library versions verified at authoring time, then pin the tested versions. A supported Node.js LTS release may provide local serving and export automation, but the curriculum must not depend on knowing Node. Record what was actually used rather than asserting a version from memory. [S39]

The finished site must function with internet access disabled. Store scripts, styles, icons, permitted assets, and library distributions locally; do not depend on a CDN, analytics endpoint, hosted font, embedded video, online diagram renderer, external model call, or remote image. Initial dependency acquisition during authoring is different from runtime network dependence.

Provide a simple documented local-server route. A local HTTP server is not an external service. Do not promise that opening every deck through file:// will support all chosen browser-module or asset-loading behavior; either test that route or state that the supplied local launcher is the supported route. The student PDF should require no server at all.

Keep the authoring workspace and private instructor material outside the distributed student directory. Include appropriate notices for redistributed libraries and any third-party assets. Use system fonts or fonts that can be lawfully supplied; this plan package contains no font files.

SUGGESTED EVENTUAL WEBSITE LAYOUT

software-engineering-course/ repository root (23_build_plan_and_schedule.txt) AGENTS.md CLAUDE.md README.md rules for the authoring agents docs/scene-contract.md DOM and JavaScript contract the tests read plan/ this package: TXT files, assets/demo-fixtures.json, plan_manifest.json, tools/, tests/ site/ the student distribution and nothing else index.html weeks/week-01.html … week-14.html shared/theme.css print.css lecture-controls.js visual-components.js assets/diagrams/ assets/fixtures/ vendor/ THIRD_PARTY_NOTICES.txt pdf/week-01.pdf … week-14.pdf README.txt notes/week-01.scene-guide.md … instructor scene guides and completion records scripts/serve.py export_pdf.py tasks/ task templates for the recurring authoring jobs .github/workflows/ci.yml

This is a suggested output structure, not a prescription for every source module. One HTML per week may reference shared local assets and scripts. Keep explanatory ownership clear: shared code defines repeated control meanings and visual primitives, while each week defines its own scenes and content.

A STUDENT PDF IS AN EXPLANATION, NOT AN ANIMATION CAPTURE

Design print states at scene-authoring time. A typical Anchor needs baseline, pivotal mechanism, changed condition, and comparison-with-principle panels. Some can combine into two or three pages; a simple Bridge may need only one annotated final diagram. Follow the weekly scene's stated PDF treatment rather than exporting every tiny motion frame.

Retain the situation, prediction question, relevant inputs, labeled arrows, changed assumption, expected outcome, principle, and essential uncertainty. Place questions before their revealed explanations or clearly distinguish them. Hide navigation buttons, sliders, authoring notes, and operational controls. Print the values selected by those controls as readable labels.

Custom canvas and WebGL content needs a deliberate print path. Numeric charts should become stable readable renderings with their underlying values. Spatial content should have a prepared 2D fallback. The printed artifact must not depend on whichever animation frame happened to be visible when export began.

Reveal.js has a print stylesheet and fragment export behavior, but it does not automatically understand the intended educational states of custom simulations. Browser automation such as Playwright can drive a local export process once those states are defined. Use an explicit print view and wait for its declared ready condition rather than relying on a guessed delay. [S28, S30, S32]

Maintain one source for the scene's facts, principles, conditions, and values so HTML and PDF do not drift. Include stable scene IDs and meaningful page numbers in the PDF. Favor selectable text, readable captions, vector diagrams where practical, and a print-friendly high-contrast theme. Inspect reading order and content alternatives; a successful browser export alone does not establish an accessible tagged PDF. Keep accessible HTML available alongside the PDF.

FREEDOM DURING AUTHORING

The plan fixes learning intent, case semantics, core comparisons, control meanings, and evidence. The authoring agent can improve composition, transitions, visual metaphors, typography, component organization, and small supporting examples. It may split a crowded scene or simplify an animation when that improves comprehension.

Do not let visual invention change a rule, invent a measurement, hide a failure, introduce a runtime external dependency, or replace a comparison with decoration. When choosing between more movement and a clearer static matrix, choose the matrix. The desired finish is elegant and interactive, but the measure of success is whether a student can explain why the outcomes differ.

PRIMARY SOURCE REGISTER

Register updated: 28 October 2026; retrieval dates are recorded per entry

The original references were checked on 16-17 September 2026. Selected status checks and new references were reviewed on 22 September 2026 as recorded below; unchanged entries retain their earlier retrieval dates. Living documentation can change. Source keys identify supporting concepts and technical guardrails, not copied lesson text. Most sources were read as HTML; access limitations are recorded explicitly. No full external reference PDF audit is claimed.

Authoring-library references are instructor resources. They are not dependencies students need to learn. Numerical fixtures throughout the course are original teaching data unless explicitly stated otherwise. Each entry ends with its use: curriculum sources support lecture content and are cited by at least one week; authoring sources support the build and are cited by the support files.

S01 | IEEE COMPUTER SOCIETY — GUIDE TO THE SOFTWARE ENGINEERING BODY OF KNOWLEDGE

https://www.computer.org/education/bodies-of-knowledge/software-engineering

V4.0a identified in official indexed material; version update indicated 25 September 2025. Rechecked 22 September 2026: the official /v4 indexed page still identifies V4.0a as newest; direct retrieval failed. Used for high-level curriculum scope; no claim of a full guide audit. The official indexed topic outline below was also checked on 22 September 2026 for the knowledge-area coverage map; direct retrieval returned HTTP 403. The depth judgments and week mapping are the course author's.

Week 7 check, 28 October 2026: the main landing page again returned HTTP 403 Forbidden. Retain the last recorded edition; no new edition claim is made from this retrieval.

https://www.computer.org/education/bodies-of-knowledge/software-engineering/topics

Use: curriculum.

S02 | ACM / IEEE-CS / AAAI — COMPUTER SCIENCE CURRICULA 2023, FINAL REPORT

https://csed.acm.org/final-report/

Final curriculum reference, published in 2024. Final-report landing page rechecked 22 September 2026; the official indexed HTML report supports teamwork, construction, requirements and professional-context coverage. Full HTML retrieval exceeded the browser text size limit. Used for broad scope and competencies; not a formal course accreditation mapping or a full-report audit. Do not substitute the older gamma-draft knowledge-area download for the final report.

Week 7 check, 28 October 2026: the official final-report landing page was accessible and still identifies CS2023 as the final report. The linked full report was not re-audited.

Use: curriculum.

S03 | MANIFESTO FOR AGILE SOFTWARE DEVELOPMENT — PRINCIPLES

https://agilemanifesto.org/principles.html

Foundational primary statement. Used for feedback and adaptation context, not as a complete process definition.

Use: curriculum.

S04 | SCHWABER AND SUTHERLAND — THE SCRUM GUIDE

https://scrumguides.org/scrum-guide.html

November 2020 text. Used for a small framework comparison and completion policy; the lecture is not comprehensive Scrum instruction.

Use: curriculum.

S05 | THE KANBAN GUIDE

https://kanbanguides.org/the-kanban-guide/2025.5/

May 2025 edition. Used for workflow, work-in-progress, and flow-measure boundaries.

Use: curriculum.

S06 | PRO GIT — BASIC BRANCHING AND MERGING

https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

Officially hosted book chapter. Supports ancestry, branching, and merge concepts; no hosting-platform dependence.

Rechecked 28 October 2026: the official Pro Git, second-edition chapter remains accessible and explains fast-forward integration, divergent histories, three-way merges and conflict resolution. No lesson mechanism changed.

Use: curriculum.

S07 | GOOGLE ENGINEERING PRACTICES — WHAT TO LOOK FOR IN A CODE REVIEW

https://google.github.io/eng-practices/review/reviewer/looking-for.html

Primary engineering guidance. Used for focused review questions, not a universal mandate for one organization’s workflow.

Rechecked 28 October 2026: the primary page remains accessible. Its design, functionality, tests, complexity and documentation guidance supports the focused review example. The course's people, comments and policy changes are original fictional data.

Use: curriculum.

S08 | SIMON BROWN — C4 MODEL

https://c4model.com/

Primary model reference. Supports context, container, and component distinctions; a C4 container is not necessarily an operating-system container.

Use: curriculum.

S09 | IETF — RFC9110: HTTP SEMANTICS

https://www.rfc-editor.org/rfc/rfc9110.html

Published 2022. Supports method and idempotency terminology; course operation keys are an explicit application-level mechanism.

Use: curriculum.

S10 | POSTGRESQL — TRANSACTION ISOLATION

https://www.postgresql.org/docs/current/transaction-iso.html

Living official documentation. Illustrative basis for isolation caveats; PostgreSQL is not a required course platform.

Use: curriculum.

S11 | HYPOTHESIS — PROPERTY-BASED TESTING DOCUMENTATION

https://hypothesis.readthedocs.io/en/latest/

Living primary tool documentation. Supports the testing concept; the course does not require Python or Hypothesis.

Use: curriculum.

S12 | STRYKER — MUTANT STATES AND METRICS

https://stryker-mutator.io/docs/mutation-testing-elements/mutant-states-and-metrics/

Primary mutation-testing documentation. Supports diagnostic interpretation, not a promise that a score proves correctness.

Use: curriculum.

S13 | OWASP — TOP10:2025

https://owasp.org/Top10/2025/

2025 released edition, identified as current by OWASP. Awareness guidance; not an exhaustive application verification standard.

Use: curriculum.

S14 | NIST — SP800-218, SECURE SOFTWARE DEVELOPMENT FRAMEWORK1.1

https://csrc.nist.gov/pubs/sp/800/218/final

Final publication, February 2022. Secure-development baseline used in this plan.

Use: curriculum.

S15 | NIST — SP800-218 REV.1, SSDF1.2 INITIAL PUBLIC DRAFT

https://csrc.nist.gov/pubs/sp/800/218/r1/ipd

17 December 2025 initial public draft; comment period closed 30 January 2026. Rechecked 22 September 2026: the NIST publication page still labels it an initial public draft. Currency watch item, not a replacement final standard.

Use: curriculum.

S16 | SLSA — SPECIFICATION V1.2

https://slsa.dev/spec/v1.2/

Approved Version 1.2, status rechecked on the official specification page on 22 September 2026. Used for provenance and build-integrity concepts, including distinct source and build concerns; no maturity-level certification is claimed.

Use: curriculum.

S17 | DORA — SOFTWARE DELIVERY PERFORMANCE METRICS

https://dora.dev/guides/dora-metrics/

Updated 5 January 2026; rechecked 22 September 2026. Five-measure terminology and application/service-level interpretation remain on the official guide. Course event data are original simulations. Do not use these measures to rank individual developers.

Use: curriculum.

S18 | DORA — STATE OF AI-ASSISTED SOFTWARE DEVELOPMENT2025

https://dora.dev/dora-report-2025/

2025 report overview. Dated research context, not a universal causal productivity estimate or a claim that no newer DORA research exists.

Use: curriculum.

S19 | DORA — BALANCING AI TENSIONS

https://dora.dev/insights/balancing-ai-tensions/

10 March 2026. Qualitative workflow and verification context; conclusions should not be generalized as guarantees for every team.

Use: curriculum.

S20 | OPENTELEMETRY — SIGNALS

https://opentelemetry.io/docs/concepts/signals/

Living official documentation. The lecture selects logs, metrics, and traces; the source covers additional signals.

Use: curriculum.

S21 | GOOGLE SRE — SERVICE LEVEL OBJECTIVES

https://sre.google/sre-book/service-level-objectives/

Primary practitioner reference. Supports indicator/objective distinctions; the course objective and data are fictional.

Use: curriculum.

S22 | GOOGLE SRE — MONITORING DISTRIBUTED SYSTEMS

https://sre.google/sre-book/monitoring-distributed-systems/

Primary practitioner reference. Supports symptom-oriented monitoring and actionable interpretation.

Use: curriculum.

S23 | PRAMOD SADALAGE AND MARTIN FOWLER — EVOLUTIONARY DATABASE DESIGN

https://martinfowler.com/articles/evodb.html

Original-author reference. Supports incremental schema evolution and migration reasoning, not a prescribed database product.

Use: curriculum.

S24 | SOFTWARE ENGINEERING AT GOOGLE — WHAT IS SOFTWARE ENGINEERING?

https://abseil.io/resources/swe-book/html/ch01.html

Primary chapter by Titus Winters. Context for engineering across time, scale, and change; its organizational practices are not universal requirements.

Use: curriculum.

S25 | W3C — WEB CONTENT ACCESSIBILITY GUIDELINES2.2

https://www.w3.org/TR/WCAG22/

W3C Recommendation; retrieved text dated 12 December 2024. Accessibility reference, not a claim of automatic conformance for future decks or this plan PDF.

Use: curriculum.

S26 | REVEAL.JS — CONFIGURATION

https://revealjs.com/config/

Living official documentation. Relevant to navigation, progress, fragment behavior, and paused presentation defaults.

Use: authoring.

S27 | REVEAL.JS — LINKS

https://revealjs.com/links/

Living official documentation. Supports stable slide deep links; custom scene semantics remain the author’s responsibility.

Use: authoring.

S28 | REVEAL.JS — PDF EXPORT

https://revealjs.com/pdf-export/

Living official documentation. Documents print styles, Chrome/Chromium support, notes, and fragment behavior; custom simulation print states require deliberate authoring.

Use: authoring.

S29 | CHART.JS — ACCESSIBILITY

https://www.chartjs.org/docs/latest/general/accessibility.html

Living official documentation. Canvas charts need explicit accessible names or fallback content.

Use: authoring.

S30 | CHART.JS — RESPONSIVE CHARTS

https://www.chartjs.org/docs/latest/configuration/responsive.html

Living official documentation. Includes print-layout considerations; a readable print result still needs inspection.

Use: authoring.

S31 | THREE.JS — OFFICIAL REFERENCE ENTRY

https://threejs.org/

Primary library entry. Optional local spatial rendering only; no pinned release is prescribed before authoring.

Use: authoring.

S32 | PLAYWRIGHT — PAGE API, PDF OUTPUT

https://playwright.dev/docs/api/class-page#page-pdf

Living official documentation. Possible local export automation; it does not choose pedagogically meaningful scene states.

Use: authoring.

S33 | OWASP GENAI — LLM06:2025 EXCESSIVE AGENCY

https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

Primary risk guidance. Used for capability, permission, autonomy, and approval-boundary reasoning.

Use: curriculum.

S34 | OWASP GENAI — LLM01:2025 PROMPT INJECTION

https://genai.owasp.org/llmrisk/llm01-prompt-injection/

Primary risk guidance. Used for untrusted-content and authority-boundary concepts; no live exploit workflow.

Use: curriculum.

S35 | NIST — SP800-218A, SECURE SOFTWARE DEVELOPMENT PRACTICES FOR GENERATIVE AI AND DUAL-USE FOUNDATION MODELS

https://csrc.nist.gov/pubs/sp/800/218/a/final

Final publication, July 2024. AI-model and AI-system development profile; not mislabeled as generic coding-assistant instructions.

Use: curriculum.

S36 | DORA — RESEARCH PUBLICATIONS

https://dora.dev/research/publications/

Living publication index consulted for current context. Distinguish an annual report's year from newer individual publications.

Use: authoring.

S37 | GREEN SOFTWARE FOUNDATION — SOFTWARE CARBON INTENSITY SPECIFICATION

https://sci.greensoftware.foundation/

Retrieved specification v1.1.0. Supports functional-unit and accounting-boundary distinctions; the course reports a resource proxy, not a calculated carbon score.

Use: curriculum.

S38 | MDN — WEB ANIMATIONS API

https://developer.mozilla.org/en-US/docs/Web/API/Web_Animations_API

Browser-platform reference. Optional implementation direction, not student prerequisite knowledge.

Use: authoring.

S39 | NODE.JS — RELEASES

https://nodejs.org/en/about/previous-releases

Official release/support listing. Check supported LTS status at authoring time and pin the tested runtime.

Use: authoring.

S40 | SOFTWARE ENGINEERING AT GOOGLE — TESTING OVERVIEW

https://abseil.io/resources/swe-book/html/ch11.html

Primary chapter on testing concepts. Used for evidence and scope framing; no universal test-ratio prescription.

Use: curriculum.

S41 | OPENAI — PROMPTING DOCUMENTATION, REACHED FROM CODEX DOCUMENTATION

https://learn.chatgpt.com/docs/prompting

The official Codex prompting URL redirected here when checked. Instructor-side authoring context only; no live model dependency in the student decks.

Use: authoring.

S42 | SPDX — LICENSE LIST

https://spdx.org/licenses/

License List version 3.28.0, dated 20 February 2026, retrieved 16 September 2026. Used for license identifiers and the permissive-versus-copyleft vocabulary of the Week 9 licensing beat; not legal advice and not a claim about any real dependency.

Use: curriculum.

S43 | ANTHROPIC — CLAUDE CODE DOCUMENTATION, OVERVIEW

https://code.claude.com/docs/en/overview

Living official documentation, retrieved 16 September 2026. Instructor-side authoring context for the agent context files and task templates in repo_kit; no live model dependency in the student decks.

Use: authoring.

S44 | OWASP GENAI — TOP 10 FOR AGENTIC APPLICATIONS FOR 2026

https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Published 9 December 2025 by the OWASP GenAI Security Project, retrieved 17 September 2026. Risk list for agents that plan, call tools and act with delegated permissions; its categories include agent goal hijack, tool misuse and exploitation, and identity and privilege abuse (category names confirmed through published summaries; the landing page links the full document). Used for the Week 13 bounded-action and untrusted-instruction scenes; awareness guidance, not a certification.

Use: curriculum.

S45 | GOOGLE — THE SITE RELIABILITY WORKBOOK, CHAPTER 5: ALERTING ON SLOS

https://sre.google/workbook/alerting-on-slos/

Primary practitioner reference, retrieved 17 September 2026. Defines burn rate as how fast, relative to the SLO, a service consumes its error budget; a burn rate of 1 uses the whole budget exactly at the end of the window. Recommends multiwindow, multi-burn-rate alerts. Used for the Week 11 error-budget and alerting scenes; the course objective and events are fictional.

Use: curriculum.

S46 | ISO/IEC 25010:2023 — SQUARE PRODUCT QUALITY MODEL

https://www.iso.org/standard/78176.html

Second edition, 2023. The catalogue page refused automated retrieval (HTTP 403) on 17 September 2026; the nine characteristics were confirmed through published descriptions: functional suitability, performance efficiency, compatibility, interaction capability (formerly usability), reliability, security, maintainability, flexibility (formerly portability) and safety (added). Paid standard; used as vocabulary in Weeks 1 and 2, not as a conformance claim.

Use: curriculum.

S47 | TURKISH PERSONAL DATA PROTECTION AUTHORITY — PURPOSE AND SCOPE OF LAW NO. 6698 (KVKK)

https://www.kvkk.gov.tr/Icerik/7456/Purpose-and-Scope-of-The-Personal-Data-Protection-Law-No-6698

Official English booklet of the authority, retrieved 17 September 2026 (served as a PDF). Law No. 6698 on the Protection of Personal Data, 2016. Used for one context line in the Week 9 privacy scene; not legal advice.

Use: curriculum.

S48 | EUROPEAN UNION — REGULATION (EU) 2016/679, GENERAL DATA PROTECTION REGULATION

https://eur-lex.europa.eu/eli/reg/2016/679/oj

Official EUR-Lex text, retrieved 17 September 2026. Article 5(1)(c) sets data minimisation (adequate, relevant and limited to what is necessary) and Article 5(1)(e) storage limitation. Used for one context line in the Week 9 privacy scene; not legal advice.

Use: curriculum.

S49 | EUROPEAN UNION — REGULATION (EU) 2024/2847, CYBER RESILIENCE ACT

https://eur-lex.europa.eu/eli/reg/2024/2847/oj

Official EUR-Lex text of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements, retrieved 17 September 2026. Annex I Part II requires manufacturers to identify and document components and draw up a software bill of materials. Dates rechecked 22 September 2026 against the European Commission page below (updated 7 September 2026): in force 10 December 2024, reporting obligations from 11 September 2026, main obligations from 11 December 2027. Week 9 distinguishes the reporting start from the main component-documentation obligations and leaves scope/exemptions explicit; it does not imply all websites are covered. Legal context only.

https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act

Use: curriculum.

S50 | SPRACKLEN ET AL. — PACKAGE HALLUCINATIONS BY CODE-GENERATING LLMS

https://arxiv.org/abs/2406.10279

"We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs", USENIX Security Symposium 2025 (arXiv 2406.10279), retrieved 17 September 2026. Shows generated code recommending packages that do not exist, which an attacker can publish under the invented name. Used for the Week 13 change-record example; the package name in the course is fictional.

Use: curriculum.

S51 | OWASP — APPLICATION SECURITY VERIFICATION STANDARD

https://owasp.org/projects/asvs

Retrieved 22 September 2026. The official page identifies ASVS 5.0.0 as the stable version and describes its use for application technical security verification requirements. Used in Week 9 to connect a risk with a checkable control expectation and evidence, distinct from Top 10 awareness and SSDF process practices. No complete standard audit or conformance claim.

Use: curriculum.

S52 | OPENAPI INITIATIVE — OPENAPI SPECIFICATION 3.2.1

https://spec.openapis.org/oas/v3.2.1.html

Published 10 September 2026; retrieved 22 September 2026. The official latest-published link resolved to this version. Supports Week 6's concrete HTTP interface-description example. A published specification does not establish implementation support in every tool; the course does not require this version or teach its syntax. Shape descriptions do not establish domain, authorization or concurrency correctness.

Use: curriculum.

S53 | METR — WE ARE CHANGING OUR DEVELOPER PRODUCTIVITY EXPERIMENT DESIGN

https://metr.org/blog/2026-02-24-uplift-update/

Published 24 February 2026; retrieved 22 September 2026. Primary follow-up describes selection effects and time-measurement limitations affecting estimates of AI assistance in experienced-developer work. Used in Week 13 for evidence interpretation, without claiming a universal slowdown or speedup, or treating an earlier study as the current state of all tools.

Use: curriculum.

S54 | METR — SELF-REPORTED IMPACT OF EARLY-2026 AI ON TECHNICAL WORKER PRODUCTIVITY

https://metr.org/blog/2026-05-11-ai-usage-survey/

Published 11 May 2026; retrieved 22 September 2026. Primary survey concerns self-reported impact and discusses selection bias. Used beside S53 in Week 13 to distinguish perceived benefit, controlled task evidence and course simulations. No numerical productivity result is imported into the teaching fixtures.

Use: curriculum.

S55 | GOOGLE SRE — HANDLING OVERLOAD

https://sre.google/sre-book/handling-overload/

Primary practitioner chapter, retrieved 22 September 2026. Supports overload handling, admission decisions and graceful degradation in Week 11. The course preserves its own fictional booking invariants and does not copy the chapter's service capacities or imply a universal threshold.

Use: curriculum.

S56 | GOOGLE SRE — DATA INTEGRITY

https://sre.google/sre-book/data-integrity/

Primary practitioner chapter, retrieved 22 September 2026. Supports the distinction between retaining backup data and establishing recoverability and integrity. Used in Week 12's migration decision. No recovery-time or data-loss targets are attributed to Campus Rooms without explicit requirements and fixtures.

Use: curriculum.

S57 | DORA — AI CAPABILITIES MODEL REPORT

https://dora.dev/ai/capabilities-model/report/

Official overview retrieved 22 September 2026; describes a companion to the 2025 report organized around seven capabilities that support AI adoption. Used only for organizational context in Week 13; the gated/download report itself was not audited, and no detailed model findings or universal causal claims are imported.

Use: curriculum.

S58 | ACM - CODE OF ETHICS AND PROFESSIONAL CONDUCT

https://www.acm.org/binaries/content/assets/about/acm-code-of-ethics-booklet.pdf

Official ACM indexed excerpts checked 22 September 2026. Sections 1.1 and 1.2 support considering affected people and reducing harm. Direct ACM HTML and PDF retrieval returned HTTP 403; no full-document or PDF-layout audit is claimed. Used for the original Week 14 release-pressure comparison, not as a legal rule or a substitute for context-specific professional judgment.

Use: curriculum.

S59 | POSTGRESQL GLOBAL DEVELOPMENT GROUP - INDEXES, INTRODUCTION

https://www.postgresql.org/docs/18/indexes-intro.html

Official documentation retrieved 22 September 2026 through /docs/current/indexes-intro.html, which identified version 18; the versioned link preserves that reference. Supports the optional Week 11 comparison of a table scan with an index, the query planner's choice and index maintenance overhead. The course comparison is qualitative and original; it imports no benchmark, command sequence or universal performance result.

Use: curriculum.

S60 | ISTQB - FOUNDATION LEVEL SYLLABUS, EXPLORATORY TESTING

https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Version 4.0.1, dated 15 September 2024. Section 4.4.2, page 44, retrieved 22 September 2026. Used for the relationship between exploration, learning and test design in Week 8. The status-recovery comparison is original course-authored material; no certification syllabus, exam preparation or training durations are adopted. Only the relevant section was reviewed, not the full external PDF.

Use: curriculum.

S61 | GIT - BISECT DOCUMENTATION

https://git-scm.com/docs/git-bisect

Official living documentation retrieved 28 October 2026. Supports narrowing a history between known good and bad commits by testing intermediate revisions. The linear h1-h8 example, persistent-failure assumption and boundary predicate are original fictional teaching data; no Git command catalogue is required.

Use: curriculum.

S62 | NPM - PACKAGE-LOCK.JSON

https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/

Official npm CLI v11 documentation retrieved 28 October 2026. Supports the distinction between allowed dependency constraints and the resolved dependency tree recorded by package-lock.json. The lesson's teaching-range versions are invented; they describe no actual npm package or release.

Use: curriculum.

S63 | ASTRAL UV - PROJECT STRUCTURE AND THE LOCKFILE

https://docs.astral.sh/uv/concepts/projects/layout/#the-lockfile

Official living documentation retrieved 28 October 2026. Supports uv.lock as a record of resolved dependency selections used to reproduce an environment. It is a dated tool example, not a requirement to use Python or uv.

Use: curriculum.

CAMPUS ROOMS: RECURRING CASE AND TEACHING FIXTURES

Research and curriculum date: 16 September 2026

USE OF THE CASE

Audience and pacing follow 00_START_HERE.txt. The transfer_checks object in assets/demo-fixtures.json holds the original qualitative closing question and answer for each of Weeks 2-14. Those independent illustrations transfer a principle, not Campus Rooms identities or policies. They introduce no numerical measurement and require no viewer implementation. The weekly TXT remains the content source; the fixture entries carry the matching display text.

DOMAIN AND IDENTITIES

Campus Rooms is a fictional room-booking service, not a real university system. Ari is U-01 and Bo is U-02. Use room C101 for the main examples and C202 when a second room is needed. B-001 is a representative booking owned by U-01. Campus staff may record room closures in the explicitly defined scenario, but do not have an unstated right to cancel every booking.

The recurring logical components are Booking, Policy, Room Catalog, Notification, and Storage. The default architecture is a modular application with an authoritative booking decision and separately tracked notification work. Logical modules do not automatically imply separate network services. No real personal data, credentials, or campus integration is used.

CANONICAL BEHAVIORAL RULES

R-01 — No overlapping confirmed bookings for the same room. Intervals are half-open: [start,end). Thus 10:00–11:00 and 11:00–12:00 are adjacent, not overlapping. Different rooms do not conflict solely because their times match. R-02 — Booking duration is 30 through 120 minutes inclusive. Core examples use whole minutes, require end later than start, and do not impose a 30-minute slot grid. A temporary 90-minute upper bound must be labeled as a variant and reset afterward. R-03 — An authenticated requester may cancel their own future confirmed booking. “Future” means its start is later than the explicit current time. Already-started and other-owner cases are denied under this baseline. No general staff cancellation override is implied. R-04 — Notification failure does not undo a confirmed booking. Pending or failed notification work must remain visible and recoverable. Confirmation and notification are separate facts. R-05 — Users must be able to complete the core task with keyboard interaction and perceive meaningful status and error information. Examples demonstrate requirements and checks; they do not claim comprehensive accessibility certification. R-06 — Week 14 only: an effective closure blocks new confirmations overlapping its room and interval. Existing bookings remain recorded and are explicitly shown as affected, with an authorized resolution process and separately tracked communication. This is a labeled policy extension, not a retroactive rule in Weeks 1–13.

Room closure is a planted thread, not a leak of R-06 into earlier weeks. It appears as a stakeholder need in Week 1, as an open question and a scheduling condition in Week 2, and as a work card in Week 4, always unresolved; R-06 is its Week 14 resolution. Authoring agents keep it an open need until then.

TIME, IDENTITY, AND STATE SEMANTICS

Use campus-local same-day time examples in the core. Do not introduce daylight-saving or cross-zone conversion silently. Any such extension needs its own explicit policy and fixtures. For cancellation checks use a fixed teaching clock, such as 09:00 for a booking starting at 10:00; the at-start case uses exactly 10:00.

The UI/request model may include Draft, Submitting, Confirmed, Rejected, and Unknown outcome. The baseline stored booking model contains Confirmed and Cancelled. A rejected request does not create a confirmed booking. A lost response can produce Unknown outcome in the UI even when the stored booking is Confirmed.

Scope operation keys to the authenticated requester and intended action. Repeating the same key with the same payload can return the original logical result; reusing it with a different payload produces an explicit conflict in the example. This is an application contract, not a claim that a timeout means no write or that every POST is inherently idempotent.

The no-overlap decision must survive concurrency. Correct local interval arithmetic is not enough. The animation must not imply that any transaction, an arbitrary row lock, or a unique room-plus-start key automatically protects every overlapping interval.

QUALITY FIXTURES WITH DISTINCT BOUNDARIES

Q-01 is the illustrative search requirement introduced in Week 2: under the declared teaching fixture and 50 concurrent users, p95 search API response time is at most 500 ms at the service boundary. It is not a claim about a browser's complete perceived experience or an industry benchmark. Actual performance evidence requires a specified dataset, request mix, environment, run conditions, and percentile calculation; the later twenty-point example illustrates the concept only.

Q-02 is the separate Week 11 illustrative objective: over seven days, at least 99% of eligible booking requests receive a correct outcome within two seconds. Eligible events are authenticated, syntactically valid booking requests reaching the stated service boundary, including legitimate room-conflict requests; health checks are excluded. A timely correct conflict rejection is good service. The fixed fixture contains 980 good events, 10 correct-but-slow results, 5 timeouts, and 5 unexpected server failures out of 1,000. The observed good-event rate is 98%, against an allowance of 10 bad events; 20 are observed. The allowance is the error budget. Observing twice the allowed bad-event rate is a burn rate of 2. Sustaining that rate under steady traffic from a fresh seven-day window would use its budget in 3.5 days. This is not remaining time after the displayed window, which has already exceeded its budget.

TEST AND AUTHORIZATION FIXTURES

Duration cases: 29 rejected; 30 accepted; 31 accepted; 119 accepted; 120 accepted; 121 rejected, provided all other conditions are valid. Reversed or malformed intervals are separate invalid cases.

Interval cases: existing C101 [10:00,11:00); candidate C101 [10:30,11:30) conflicts; candidate C101 [11:00,12:00) does not; candidate C202 [10:30,11:30) does not conflict with C101. These cases establish interval semantics, not concurrent correctness by themselves.

Cancellation matrix: authenticated U-01, own confirmed B-001, now 09:00/start 10:00 is allowed. U-02 with the same booking is denied. U-01 at 10:00 is denied because the booking has started. An unauthenticated request is denied. An already-cancelled booking has a stable explicitly defined

repeated response rather than another cancellation transition.

FLOW AND DELIVERY FIXTURES

Queue model: eight equal-sized cards arrive at round 0. Implementation and review each take one round per admitted batch, limited by their declared capacities. Review can use only cards available at its round start. Newly implemented cards join review for the next round. Baseline capacities are implementation 2/review 1; comparisons use 4/1 and2/2 with identical arrivals. The included JSON records end-of-round ready, review-waiting, and completed counts. Baseline and faster-implementation both finish at round 9; faster-review finishes at round 5. These are model results, not forecasts about developers.

Delivery fixture: one service, ten deployments in a seven-day window, IDs D01–D10. D03 and D08 fail and require immediate intervention. D04 and D09 are unplanned incident-driven corrective deployments. Lead times in hours are [2,4,3,5,2,6,4,3,2,9], with median 3.5. Recovery durations for the two failed deployments are 15 and 25 minutes, with median 20. Deployment frequency is 10 per seven days, or approximately 1.43/day. Change fail rate and deployment rework rate are each 2/10=20%, with distinct numerators. These are illustrative summaries using the current metric meanings, not a claim about a real team. [S17]

LATENCY AND RESOURCE FIXTURES

Latency Dataset A contains twenty responses at 200 ms. Dataset B contains eighteen at 100 ms and two at 1,100 ms. Both arithmetic means are 200 ms. Under nearest-rank percentile selection, p95 is the nineteenth sorted value:200 ms for A and1,100 ms for B. Print the method and sample size. This tiny fixture is not a load test validating Q-01.

Resource fixture: 100 successful searches use 1,000 storage reads before a change and 200 afterward. That is 10 versus2 reads per successful search, an80% reduction under equivalent stated correctness and freshness conditions. No energy or emissions value is supplied; the chart must not label the reduction as carbon savings.

RETRY MODEL

The Week 11 retry-amplification scene uses a toy model: one dependency serves at most two attempts per round, first come first served; six unique requests Q1 to Q6 arrive in round 1; the first attempt of Q6 is dropped in round 1 without a response. A request still pending at the end of a round is a timeout for its client. Under the naive policy every timed-out client immediately submits another attempt while its earlier attempts stay queued, and an attempt served for a request that already succeeded is a duplicate effect. Under the bounded policy a client resubmits at most once, only after waiting two rounds and never while the circuit is open, and reports Unknown outcome once four rounds have passed with its retry spent; the circuit opens at the end of any round with at least four timeouts and blocks submissions for two round-ends.

Recorded results, recomputed by the checker: naive peaks at seven queued attempts, completes the six unique requests in round 5 with seven duplicate effects, and drains only in round 7; bounded peaks at three queued attempts, completes them in round 4 with no duplicate effects, and its circuit is open at the ends of rounds 1 and 2. The retries competed with the originals for the same capacity, which is the lesson. The model is a mechanism illustration, not a measurement, and it reuses the Week 6 operation-key idea for the duplicate effects.

INVENTED DEPENDENCY

Week 13's change record uses textenc-turbo, a fictional package name marked unavailable in the course's registry fixture, to show a dependency invented by a code assistant. It is deliberately close to textenc, the fictional transitive dependency of Week 9; both names are course labels, and no assertion about their availability in public registries is made.

AUTHORING NOTES FOR ALL FIXTURES

Any additional numbers introduced during authoring should be identified as measured, sourced, or simulated. Fix seeds where randomness is used for illustration, record the inputs, and independently verify the outputs. Repeated visits to a scene should not silently generate a different lesson.

Structured data for every Anchor scene, and for the numeric Bridges, lives in the anchors block of assets/demo-fixtures.json under the key wNN/scene-id, beside the cancellation matrix and the retry model. The HTML and the PDF render from it, and tools/check_plan.py recomputes every model and checks the week text against the values the fixtures say it shows.

REUSABLE AUTHORING BRIEF FOR THE BUILD AGENTS

Research and curriculum date: 16 September 2026

HOW TO USE THIS BRIEF

The course site is built by coding agents (Claude Code, Codex or another runtime; the choice is not fixed here). The agents do not read this brief as prose. They read the context files and task templates in repo_kit, which become the repository's AGENTS.md, CLAUDE.md and tasks folder when the repository is created (23_build_plan_and_schedule.txt). This file explains what those artefacts contain and how a human uses them; the rules themselves live once, in repo_kit/AGENTS.md. [S41, S43]

Give an agent one task at a time with the materials it needs: the task template, the week's TXT file, the shared brief, the case guide and the fixtures. Do not load every week for every small change. Build and inspect one complete week before duplicating the shell across the course; after the shared controls and print path work, weeks are authored in parallel as the build plan describes.

WHAT THE CONTEXT FILES FIX

repo_kit/AGENTS.md carries the rules that must never drift between agent runs: the fixtures are the only truth; one deck per week on the shared shell; the DOM and JavaScript contract in repo_kit/docs/scene-contract.md is the interface the tests read; no network at runtime and pinned local libraries; the decisions in force from 00_START_HERE.txt; the student-facing wording rules; the layout and recording requirements; and a definition of done that lists the commands to run and the inspection to perform before completion is claimed.

repo_kit/CLAUDE.md imports AGENTS.md and adds only runtime-specific notes. A Codex run reads AGENTS.md directly. Any other runtime is pointed at AGENTS.md.

WHAT THE TASK TEMPLATES ASK FOR

repo_kit/tasks/00_shell_and_week1.md: the shared shell, Week 1 complete, and the two shell-test scenes from Weeks 5 and 11, exactly as the start-here file recommends.

repo_kit/tasks/10_author_week.md: one week on the existing shell, with the three Anchors as controlled explanations, every Bridge reveal operated by a pointer for the recording, the print panels, the PDF, the layout scan at the recording resolution and a scene guide.

repo_kit/tasks/20_revise_after_review.md: revise only what the review notes name, recheck every deck that a shared change touches, and never expand the curriculum to demonstrate a library. There is no instructor rehearsal (00_START_HERE.txt); review notes come from reading the rendered deck and PDF.

repo_kit/tasks/30_export_pdf.md, 50_source_reverification.md: the export and inspection of student PDFs, and the register re-verification before Week 1 ships. There is no cue-list task: the recording agent generates its shot list and narration at run time (22_recording_track.txt).

HOW A HUMAN CHECKS AN AGENT'S REPORT

An agent's report must list the files produced, the commands run with their results, the library versions actually tested, and the unresolved limitations. Treat "the tests pass" as a claim to be checked by rerunning the command list in repo_kit/AGENTS.md. Treat "inspected in a browser" as a claim to be spot-checked by opening two scenes through their hash links. The completion record in 21_instructor_preflight.txt is filled from verified results only.

The prompting references in the register are instructor-side context for writing these files. They are not a requirement to expose any assistant during class, and the student decks contain no live model dependency.

INSTRUCTOR PREFLIGHT AND ACCEPTANCE CHECKS

Teaching review updated: 22 September 2026

BEFORE THE FIRST AUTHORED WEEK

Test the template at the intended recording resolution and at a 1280×720 viewing size representative of a smaller playback window. Establish shared component names, type sizes, diagram legends, and control labels. Decide the documented local-launch route and verify it on the intended teaching machine. Check a keyboard-only path and low-motion path from the course index into a scene and back.

Review the canonical rule file before authoring examples. Verify that the duration boundary is inclusive, adjacent intervals are allowed, unknown outcome is not equated with absent data, and notification status is distinct from confirmation. These small semantic details recur throughout the course.

FOR EVERY WEEKLY HTML DECK

Render every page, condition and state at the recording resolution, 1920 by 1080 logical pixels at device scale factor 2 (3840 by 2160), with python scripts/inspect_deck.py --layout, and read the captures natively. The layout scan reports overlapping text, text outside the scene area and text that would be cut from a print panel. Open every important scene directly through its stable hash. Verify the documented baseline and paused state. Step through the whole scene, use Back step, then Replay and Reset. Confirm that the controls mean different things. Leave a scene mid-animation and return; it must follow the declared re-entry behavior and must not have continued running invisibly.

Check Next and Previous independently of scene state, including when the scene is unfinished. Confirm that the last Step does not advance the slide. Check sliders, buttons, and keyboard focus without shortcut conflicts. Verify visible labels at the smaller playback viewing size, adequate contrast, color-independent meaning, and reduced motion with no lost information.

Inspect every numerical chart against the recorded fixture and every diagram against the rule it claims to illustrate. Ensure the changed condition is visible and that the comparison has not also changed workload, scale, or measurement boundary unintentionally. Test false, unavailable, empty, denied, and unknown-outcome states as appropriate—not only the successful path.

TEACHING REVIEW BEFORE ACCEPTANCE

For every outcome in the weekly plan's Section 2, identify the scene that explains it and an answered question or comparison that lets the viewer reason about it. A topic label alone does not establish coverage. Record missing explanations in the existing weekly completion record and resolve them before calling the deck teaching-ready.

Check the first use of each essential term and symbol in context. The viewer should see what it means through the example before needing it in a later decision. In the three main demonstrations, inspect the baseline, prediction, decisive change, explanation and consequence; a sequence of animations without an explained causal connection is incomplete.

Read the core route with optional extensions omitted: the next lecture must remain understandable. Review the final transfer question and its answer to check which principle carries across and which domain assumptions must be reconsidered. Apply the audience and pacing decisions in 00_START_HERE.txt when judging depth; retain essential reasoning and remove repeated labels or tool lists first.

FOR EVERY STUDENT PDF

Export from the designated print state, not the current presentation frame. Inspect every page for clipping, tiny labels, missing diagrams, blank canvas output, overlapping reveals, truncated code, and repeated controls. Check that the selected panels explain the causal change rather than showing only its final result.

Verify that questions, revealed evidence, changed assumptions, values, and principles agree with the HTML. Confirm that scene identifiers remain visible and that any multi-page scene is easy to follow. Check a grayscale or low-color preview. Review text selection and reading order; keep the accessible HTML counterpart available rather than assuming the export is fully tagged or conformant.

OFFLINE AND DISTRIBUTION CHECK

Copy the complete student site to a clean folder or second machine, disconnect internet access, start it through the documented local route, and open all fourteen decks from the index. Check that required fonts, scripts, styles, diagrams, chart data, and static alternatives load locally. Confirm that no student feature depends on a development-only server endpoint or an authoring account.

Verify the fourteen PDF links and the matching HTML week numbers. Keep instructor notes, authoring prompts, secrets, and irrelevant source files out of the student distribution. Include third-party notices and the tested version manifest. A library or browser update should trigger a repeat of affected export and interaction checks.

RECOVERY DURING A LECTURE

There is no instructor rehearsal (00_START_HERE.txt). Each week's scene guide (notes/week-NN.scene-guide.md) names the baseline, the useful condition presets and the state that best exposes each misconception, and Reset always returns to the known starting point. Avoid live dependency installation or uncontrolled code generation during the lecture.

Have the week’s student PDF available locally as a recovery route if an interactive component fails. A static explanation should still communicate the essential mechanism. Use the pacing guidance to shorten Bridge scenes when discussion expands, preserving the causal comparisons rather than rushing through every animation.

AUTOMATED COVERAGE OF THESE CHECKS

The tests in tests/test_deck_contract.py and tests/test_pdf.py run the mechanical part of this checklist against a built site (pytest with SE_COURSE_SITE set). Hash entry at the baseline, paused: test_hash_entry_opens_baseline_paused. Step, Back, Replay and Reset meaning different things: test_step_and_back_move_one_state and test_replay_keeps_condition_and_reset_restores_baseline. The last Step not advancing the slide: test_last_step_never_advances_the_page. Next and Previous independent of scene state: test_next_and_previous_change_the_page_even_mid_scene. Leaving mid-animation and returning: test_leaving_a_scene_cancels_its_work. Keyboard focus and accessible names:

test_controls_are_keyboard_reachable_with_names; sliders and arrow keys: test_range_inputs_do_not_change_the_page. Reduced motion with no lost information: test_reduced_motion_reaches_the_same_final_state. Contrast and labels: test_axe_reports_no_serious_violations, partially. Charts and diagrams against the recorded fixture: test_fixture_values_are_shown. Local operation: test_deck_loads_with_no_external_requests. Export from the designated print state: test_print_view_declares_ready and the PDF tests. Overlapping text, text outside the scene area and clipped print panels at the recording resolution: python scripts/inspect_deck.py --layout.

Still manual: readability at the smaller playback viewing size, a native read of the 3840 by 2160 captures and of every PDF page, the grayscale preview, reading order, and the copied-folder second-machine check beyond its network part.

COMPLETION RECORD

For each week, record the HTML and PDF paths, scene IDs, tested versions, checks, inspections, outcome-to-scene coverage and unresolved limitations. Mark behavior verified only after observing it; file generation, a successful build and a teaching review are distinct.

RECORDING TRACK: EVERY WEEK AS A NARRATED VIDEO

Research and curriculum date: 16 September 2026

SCOPE AND WHERE THE RULES LIVE

Every week is recorded (decision in 00_START_HERE.txt). The recordings follow the tutorial workspace rules, each of which lives in one file there and is not repeated here:

F:/0_tutorial_videos_project/AGENTS.md: the rule index and the runtime mapping.
F:/0_tutorial_videos_project/docs/natural_tutorial_workflow.md: planning order (audio first: the narration is written and synthesized before the final take, and the take is recorded to it), action over static content, continuous narration.
F:/0_tutorial_videos_project/docs/channel_style_guide.md: opening structure, narration voice, public audience and useful content, instructional not defensive.
F:/0_tutorial_videos_project/docs/tutorial_layout_policy.json: canvas, capture_scale, avatar, static_content caps, narration_stream, media_formats, resolve_frame_rates, delivery_preflight.
F:/0_tutorial_videos_project/docs/production_playbook.md: pre-take desktop hygiene, showing a Command Prompt, pre-delivery preflight, frame-rate preflight, Resolve procedures, publication deliverables. Checkers in that workspace: tools/check_static_content.py, tools/check_narration_style.py, tools/check_narration_coverage.py, tools/preflight_delivery.py, tools/scan_offline_frames.py.

The lecture decks are the screen being recorded. A deck is not a slide show in the sense of those rules only when a hand operates it on camera: a picture that changes by itself is static under the action-over-static rule, and a pointer clicking Step, selecting a condition preset, hovering a node or dragging a slider is action.

NO SPEAKING CUES IN THIS REPOSITORY

The repository and this plan hold no speaking cues (decision in 00_START_HERE.txt): no cue lists or shot lists, no narration text or narration briefs, no lines about what a moment of the deck teaches, no shot-duration budgets, and no recorded route that fixes which pages are shown or skipped, what is said over a page, or how a video opens and closes. Decks, fixtures, plan files, scene guides and completion records do not carry them, and authoring agents do not add them.

The recording agent generates all of it at run time. It first analyzes the week: plan/weekN.txt in full, plan_manifest.json, the case guide (19_recurring_case_and_demo_data.txt), the fixtures the deck renders, the scene guide, and the deck itself, operated in record=1 mode and rendered at 3840 by 2160. From that analysis it decides the route through the deck, writes the shot list and the narration, synthesizes the speech and records the tutorial, in the planning order and under the checks of the workspace rules above. What it writes for one recording stays with that recording in the tutorial workspace; it does not come back into this repository as cues for later runs.

WHAT THE DECK PROVIDES FOR RECORDING

The shell implements the recording items of docs/scene-contract.md: the record=1 mode with deterministic timing and no autoplay, a pointer-target attribute (data-cue-target) on every control and every element a pointer may click, hover or drag, and the window.lecture interface, which lists the pages and reports each scene's state. The deck is laid out at 1920 by 1080 logical pixels and

rendered in Chrome at device scale factor 2, which gives the native 3840 by 2160 capture the capture_scale rule requires; takes are captured at 60 fps. The dark theme is the default (00_START_HERE.txt).

The deck uses the whole canvas. Nothing is reserved for the presenter; the avatar overlay yields wherever the footage shows content in its rectangle, as the avatar block of the layout policy defines. Where a composition allows, the shell leaves the bottom-right corner empty so the overlay can appear more often. This is a preference, not a reservation.

Every reveal is operated. Anchors advance by Step clicks. Bridge rows, cells and arrows appear on a click or hover of their pointer target, never on a timer, so a Bridge counts as action when the pointer operates it. A running model (queue flow, retry amplification) counts as a process while its progress is visible on screen and is started by a visible Run click. In record=1 mode the deck never advances on its own and never draws a synthetic cursor; the real pointer moves with real input events.

The deck is deterministic, so no separate rehearsal is recorded or performed (00_START_HERE.txt). For its analysis the recording agent can render every page, condition and state at 3840 by 2160 with python scripts/inspect_deck.py --layout --week NN --shots --out <folder> and read the captures natively, and it serves the deck with python scripts/serve.py --root site.

WORDING OF DECK TEXT AND NARRATION

We teach, so the narration is confident and sure (decision of 16 September 2026, clarified 17 September 2026: the rule is the stance, not a word list). The course's technical words are used as instruction with their ordinary meaning: verify, confirm, evidence, distinguish, establish, record. What stays out is the defensive shape: does not prove, no guarantee, another configuration may differ, we do not claim, record the evidence for a report. The viewer is addressed as "you"; never students, class, exam or grade; no production logistics in speech.

The deck text follows the same wording rules, and the style checker is run on the deck strings (python scripts/check_deck_text.py) as well as on the narration the recording agent generates.

DELIVERABLES PER RECORDED WEEK

Generated at run time and kept with the recording in the tutorial workspace: the shot list passing the static-content checker, the 3840 by 2160 state captures, the narration units passing the style checker, the synthesized units, the takes recorded to them and remuxed, the edit plan, the placement passing the coverage checker, the Resolve project and timeline exports, the separate SRT and source mapping, the pre-delivery preflight report and the post-render check reports. The week's completion record (notes/week-NN.completion.md, 21_instructor_preflight.txt) names where they are and what was actually checked.

BUILD PLAN, PARALLEL AUTHORING, AND SCHEDULE

Build sequence updated: 22 September 2026

THE REPOSITORY

The course is built as a Git repository, local for now; the user publishes it to a remote when fitting (00_START_HERE.txt). Its root holds the files staged in repo_kit (AGENTS.md, CLAUDE.md, README.md, docs/scene-contract.md, scripts, tasks, the CI workflow and .gitignore). This plan package moves into plan/ unchanged: the TXT files, assets/demo-fixtures.json, plan_manifest.json, tools/, tests/, pytest.ini and requirements-dev.txt. The student distribution is site/ and nothing else. Scene guides and completion records live in notes/; there are no recording cue lists (00_START_HERE.txt). The layout is drawn in repo_kit/README.md.

The CI workflow runs the plan checker, the deck and PDF tests and the PDF artifact upload on every push once the repository has a GitHub remote. Until then the same commands run locally; they are listed once in repo_kit/AGENTS.md under the definition of done.

THREE PHASES

Phase 0, one agent, sequential: the shared shell, Week 1 complete, and the two shell-test scenes from Week 5 (dependency-example) and Week 11 (latency-is-a-distribution) inside their eventual deck files (repo_kit/tasks/00_shell_and_week1.md). Exit: the full command list passes for those decks, the layout scan is clean at 3840 by 2160, and Week 1 has its PDF and scene guide. There is no instructor rehearsal (00_START_HERE.txt).

Phase 1, several agents in parallel: Weeks 2 to 14, one week per branch or worktree, each against the frozen shell, the contract and the tests (repo_kit/tasks/10_author_week.md). Shell changes are made only in a shell branch and rerun every existing deck's tests. A useful working set is three agents authoring three consecutive weeks while a fourth exports PDFs for finished weeks.

Phase 2, per week and once at the end: revision after review (tasks/20), PDF export and inspection (tasks/30), the copied-folder offline check from 21_instructor_preflight.txt, and source re-verification (tasks/50) before the corresponding lecture is authored.

MINIMUM VIABLE DECK

A week is ready for detailed teaching review only after it meets this baseline: all eight scenes present with their identifiers, situation, question and principle; the three Anchors interactive with Step, Back, Replay, Reset and at least the baseline and one changed-condition preset; the five Bridges as annotated diagrams or tables whose rows appear on pointer-operated reveals; print panels for every scene and the exported PDF; the contract tests passing; the layout scan reporting no overlapping, overflowing or clipped text at 3840 by 2160. Continuation pages are required wherever readability needs them; visual polish and optional rendering libraries do not substitute for a complete explanation. This baseline is not permission to omit the core reasoning or ship an unfinished lecture. Finish and verify an incomplete deck before recording it.

DEPENDENCY-BASED BUILD SEQUENCE

Week 1 and the shared shell are complete; the first lecture is recorded. Retain them as the accepted starting point. For each remaining week, refresh its volatile references, author the deck and reading copy, verify the complete explanation and layout, then produce the recording under the tutorial workspace rules. Work may prepare later weeks independently when their shared prerequisites are stable. Publication dates and production-effort estimates are separate from lecture content and are set only when needed; follow the pacing decision in 00_START_HERE.txt. Do not reduce explanation quality to satisfy a calendar offset.

ROLES OF THE TOOLS

tools/build_manifest.py derives the scene manifest that the index page, the tests and the agents read. tools/check_plan.py guards the plan: structure, links, citations, rules, spine, fixtures recomputed from the models, and manifest freshness. tests/ guards the decks against the contract, accessibility, offline operation, fixture agreement, the print signal and the PDFs. scripts/export_pdf.py produces the student PDFs from the declared print state. scripts/inspect_deck.py --layout renders every state at 3840 by 2160 and reports overlapping text, text outside the scene area and text a print panel would cut. The workspace checkers guard the shot list, the narration and the delivery that the recording agent generates at run time. tools/build_combined.py regenerates the derived Markdown and PDF copies of this plan, and its --check mode fails when they are stale.

COMPLETION RECORD

Per week, in the repository under notes/ next to the scene guide: deck file, PDF file, scene identifiers, tested browser and library versions, the commands run with their results, the inspection performed, the recording deliverables of 22_recording_track.txt, and unresolved limitations. An item is marked verified only after the behavior was observed, as 21_instructor_preflight.txt requires.